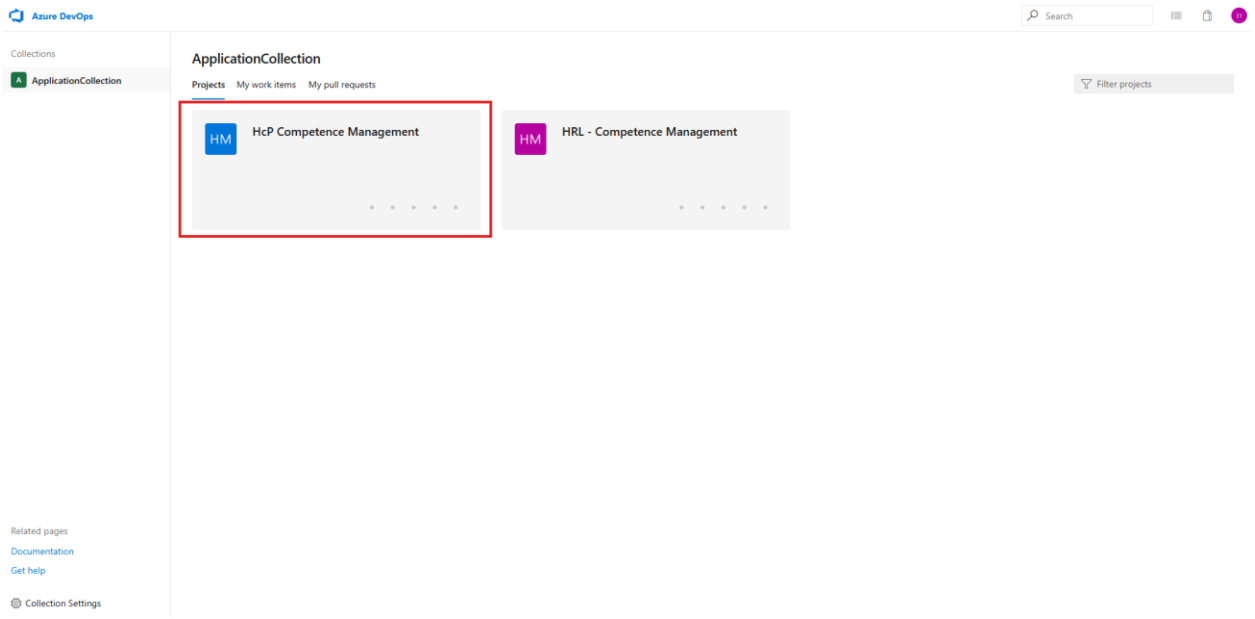
Guideline set up CICD

1. Set up Builds (CI)

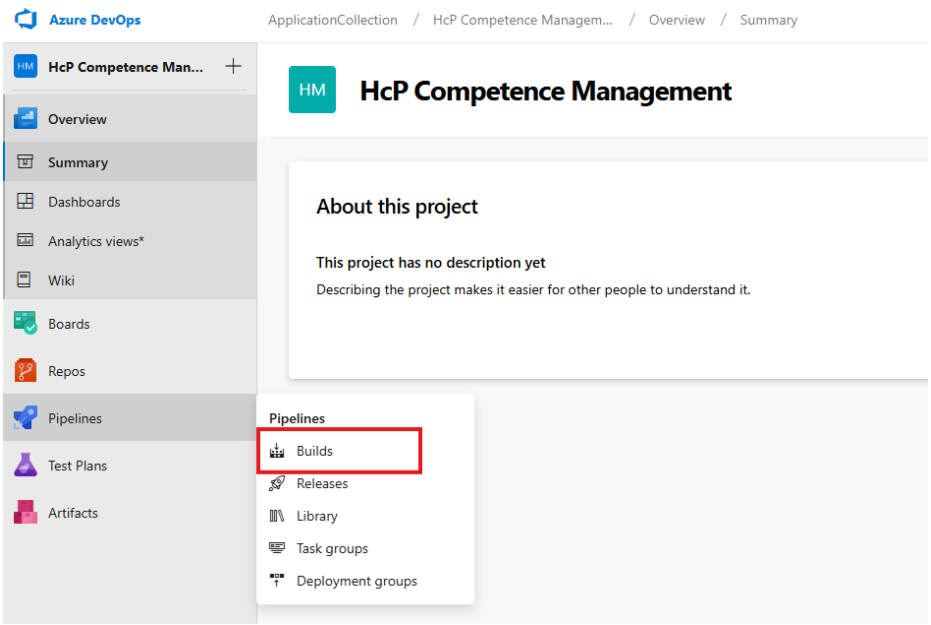
1.1. Set up CI cho backend.

Step 1. Access vào Azure DevOps theo đường link sau:
http://10.184.205.25:8080/ApplicationCollection

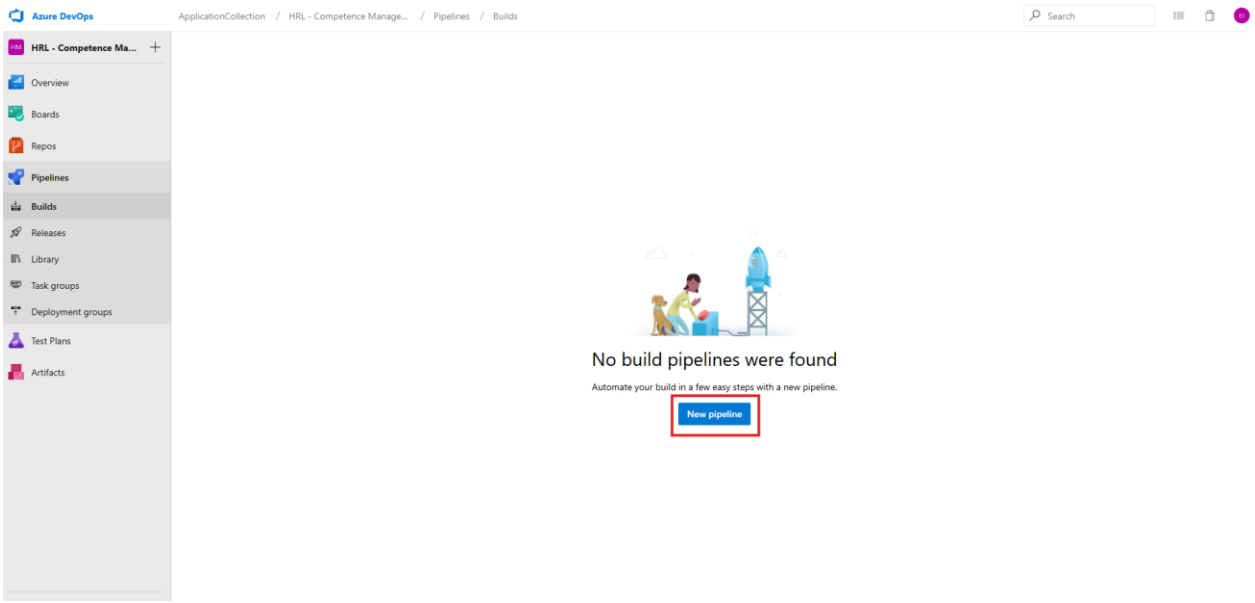
Step 2. Chọn project cần set up



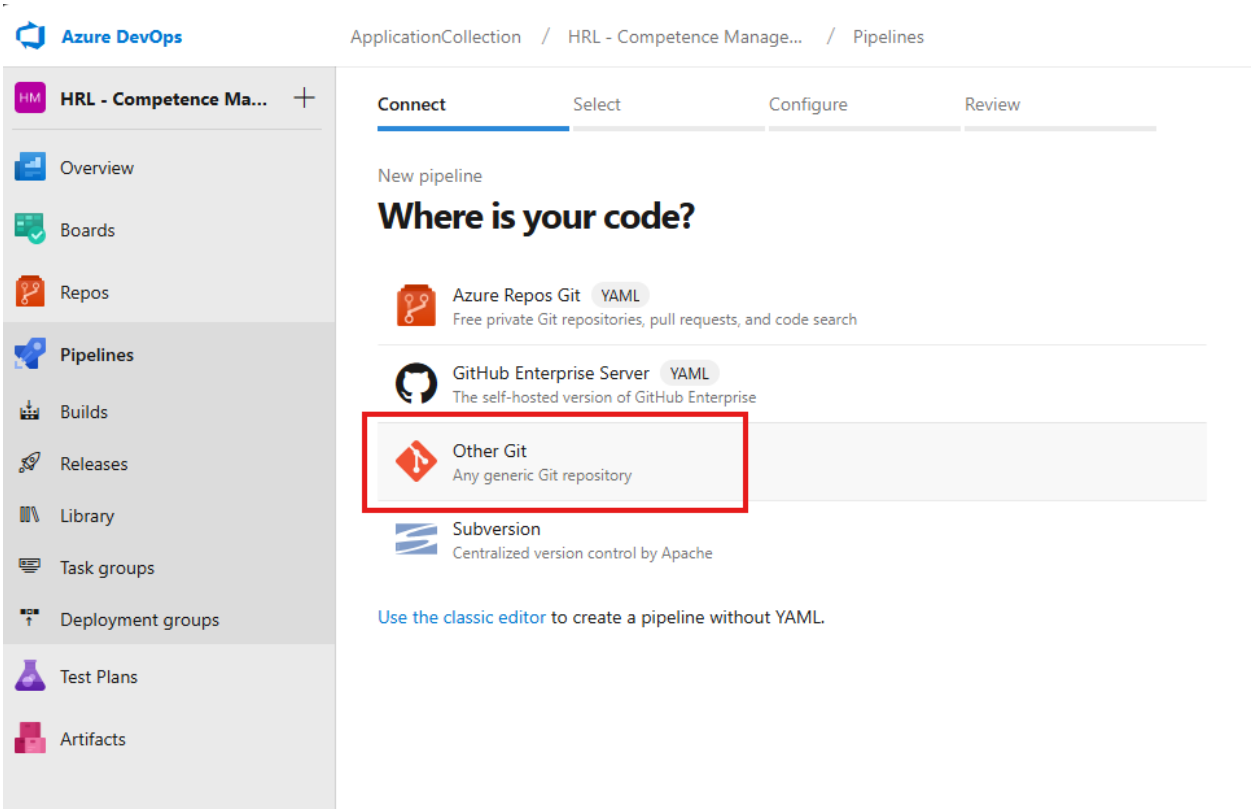
Step 3. Tại left menu, chọn Pipelines > Builds



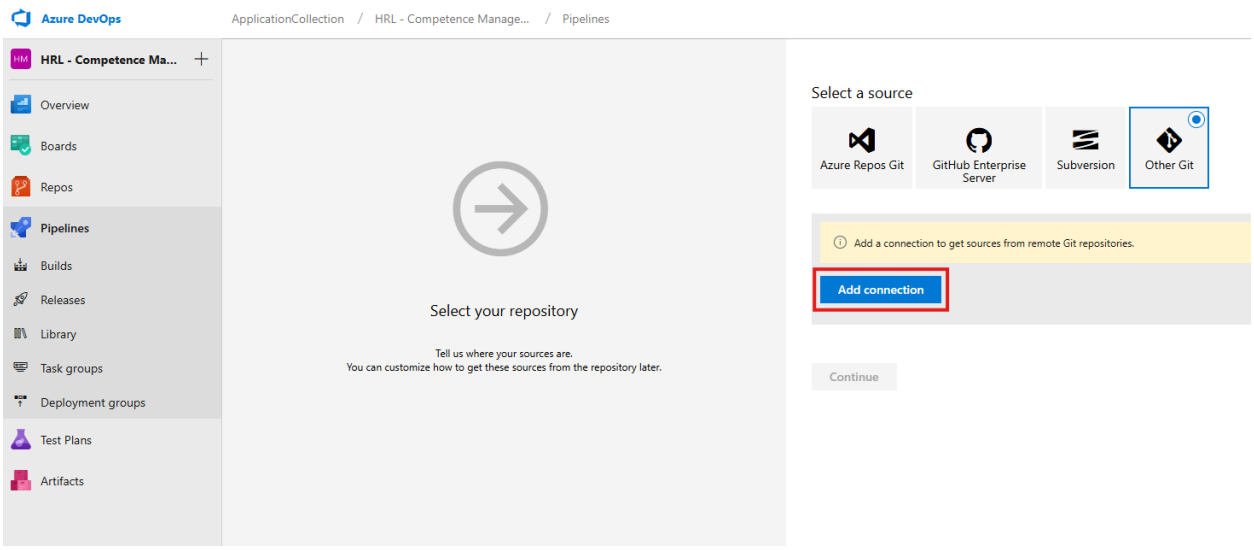
Step 4. Chọn “New pipeline”



Step 5. Chọn “Other Git”

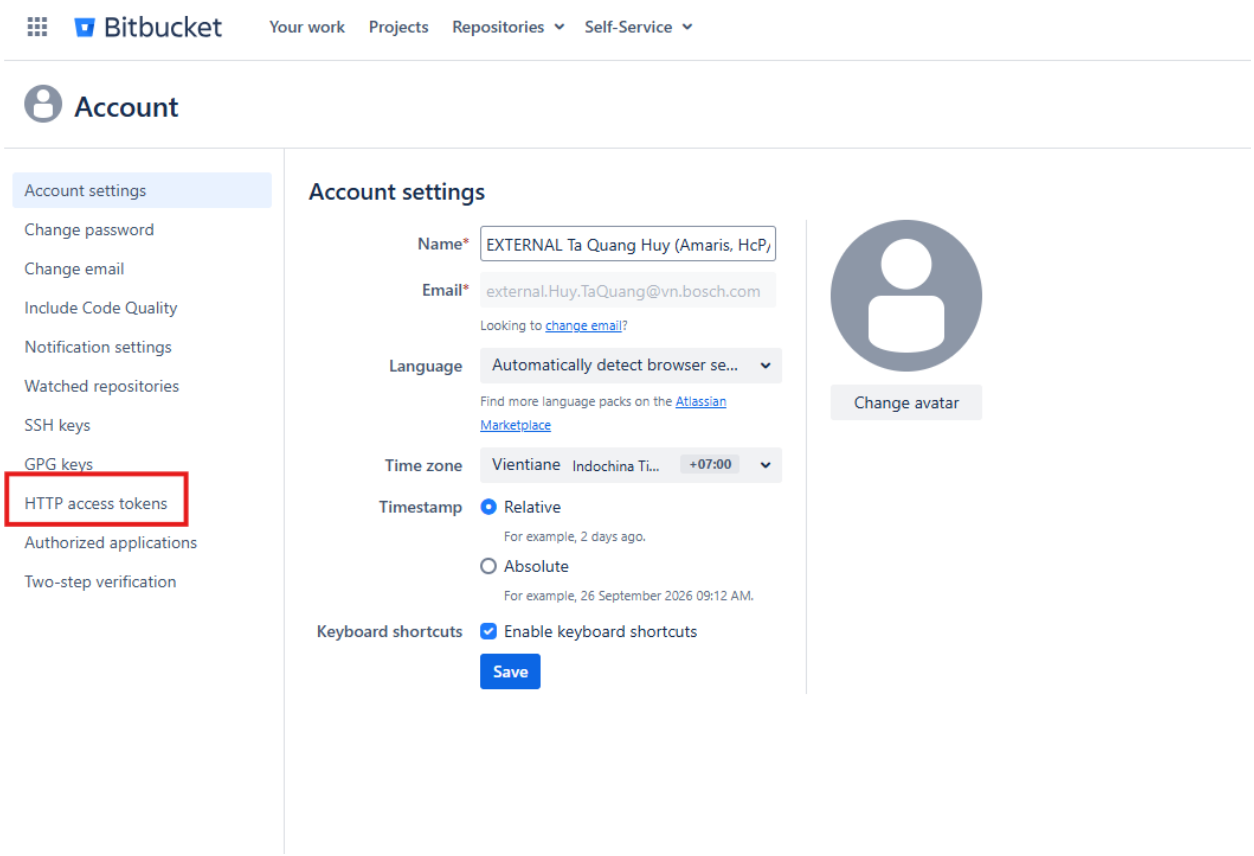


Step 6. Chọn “Add connection”

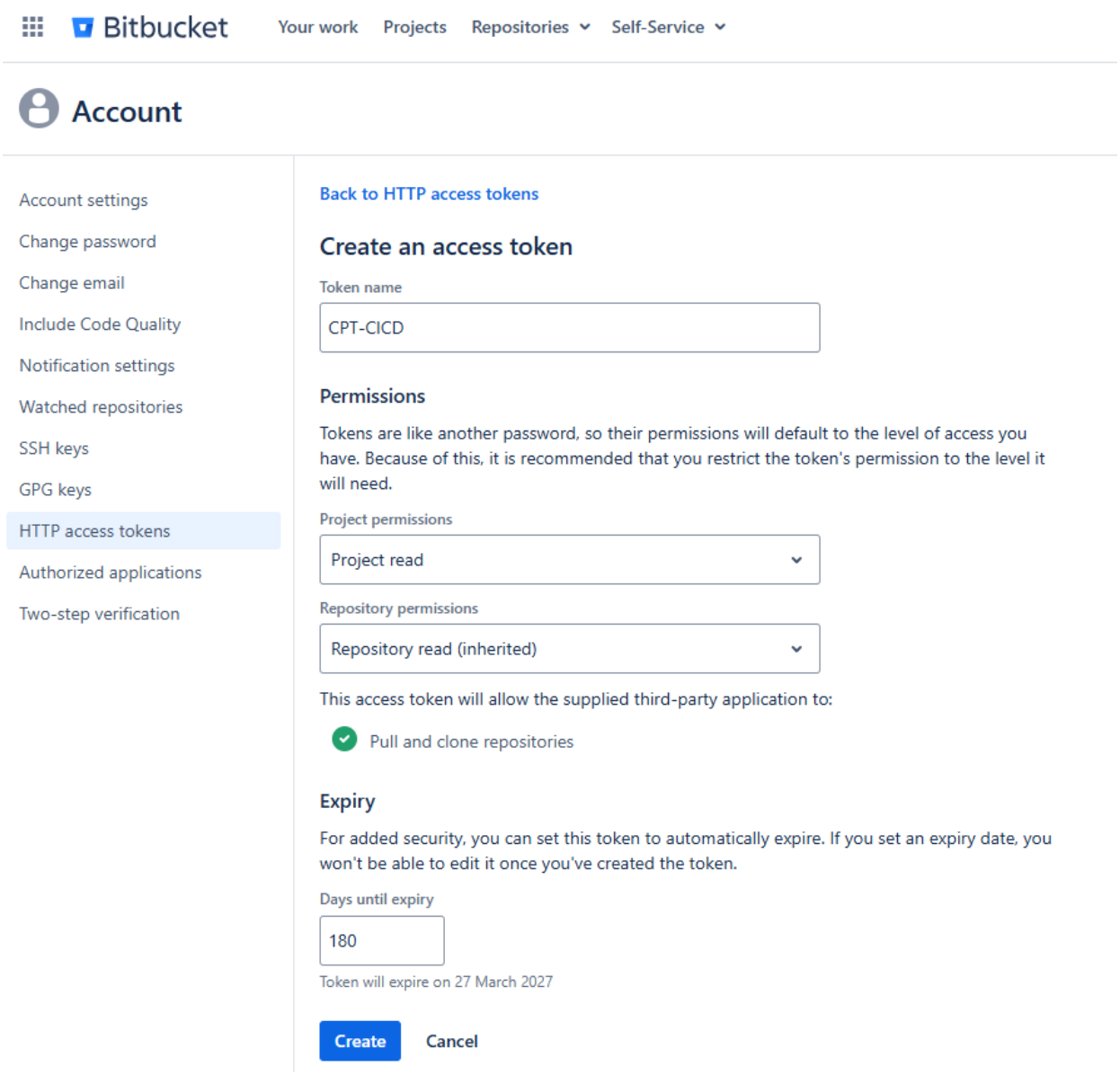


Step 7. Access vào Git với URL sau:
https://sourcecode.socialcoding.bosch.com/account

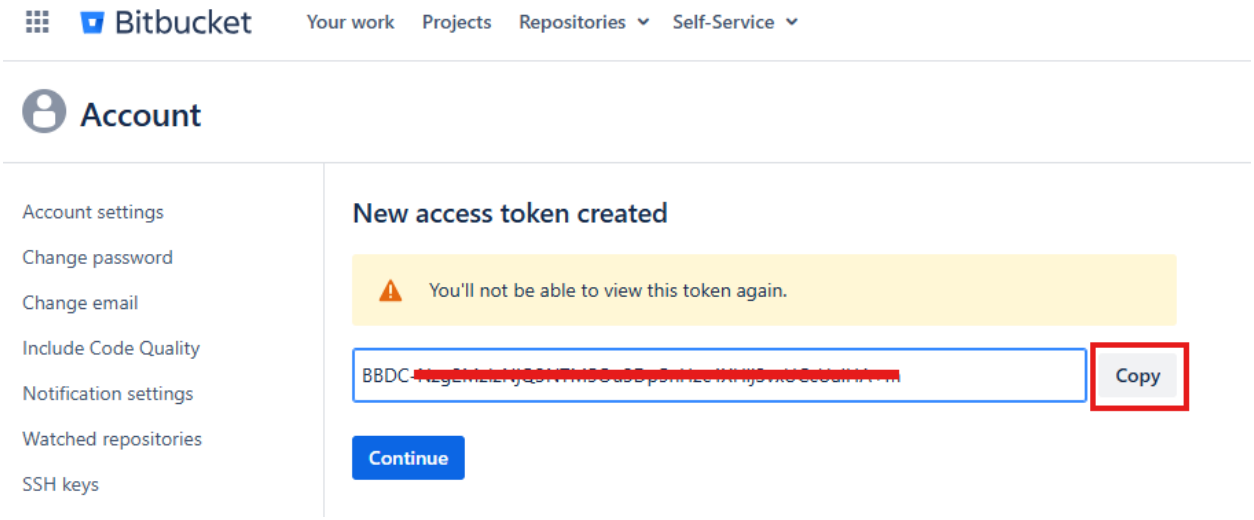
Sau đó, chọn “HTTP access tokens”.



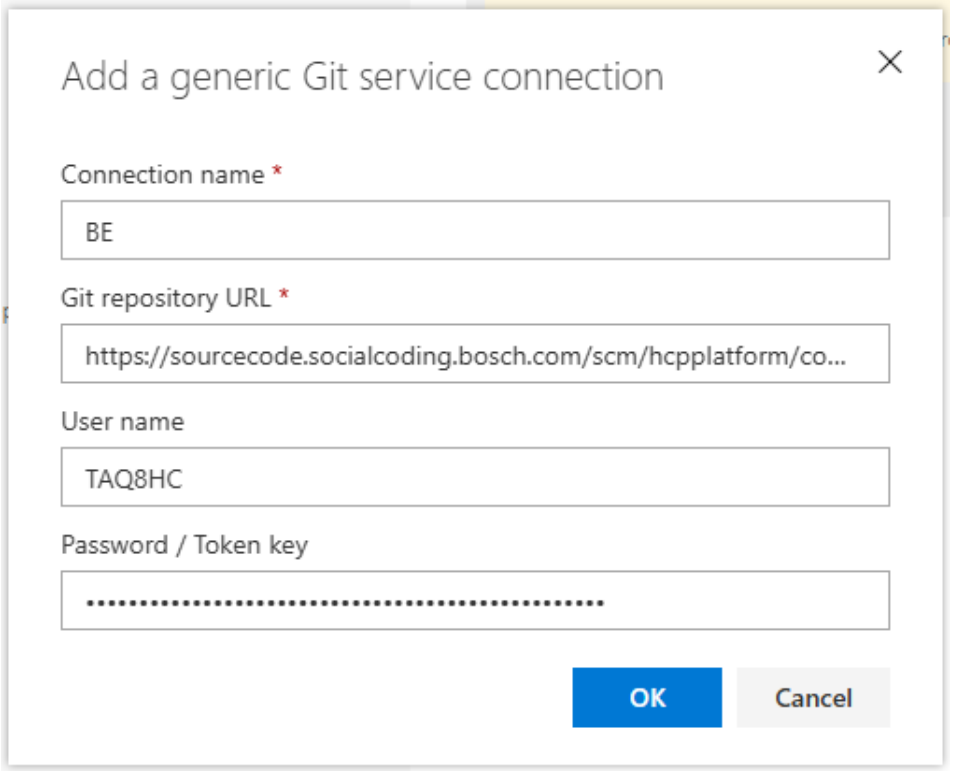
Tạo 1 access token mới với các thông tin như hình sau:



Click “Copy” và lưu lại access token để dùng cho các bước sau:



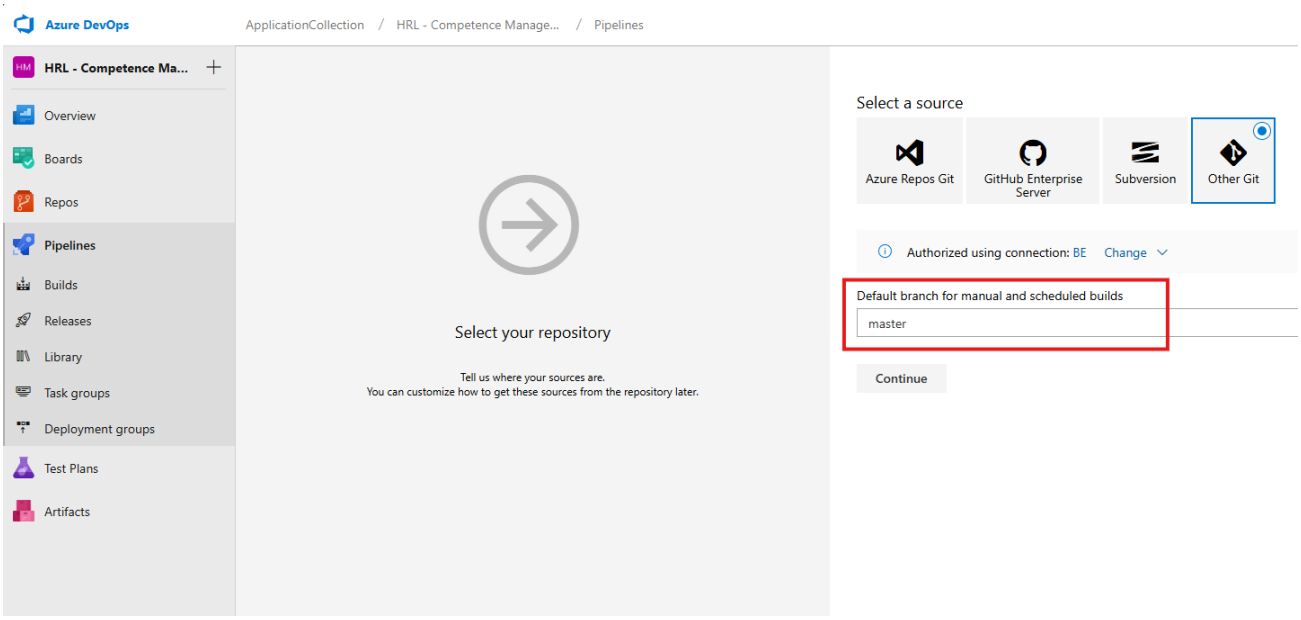
Step 8. Quay trở lại Azure DepOps, fill thông tin Git với các fields sau:



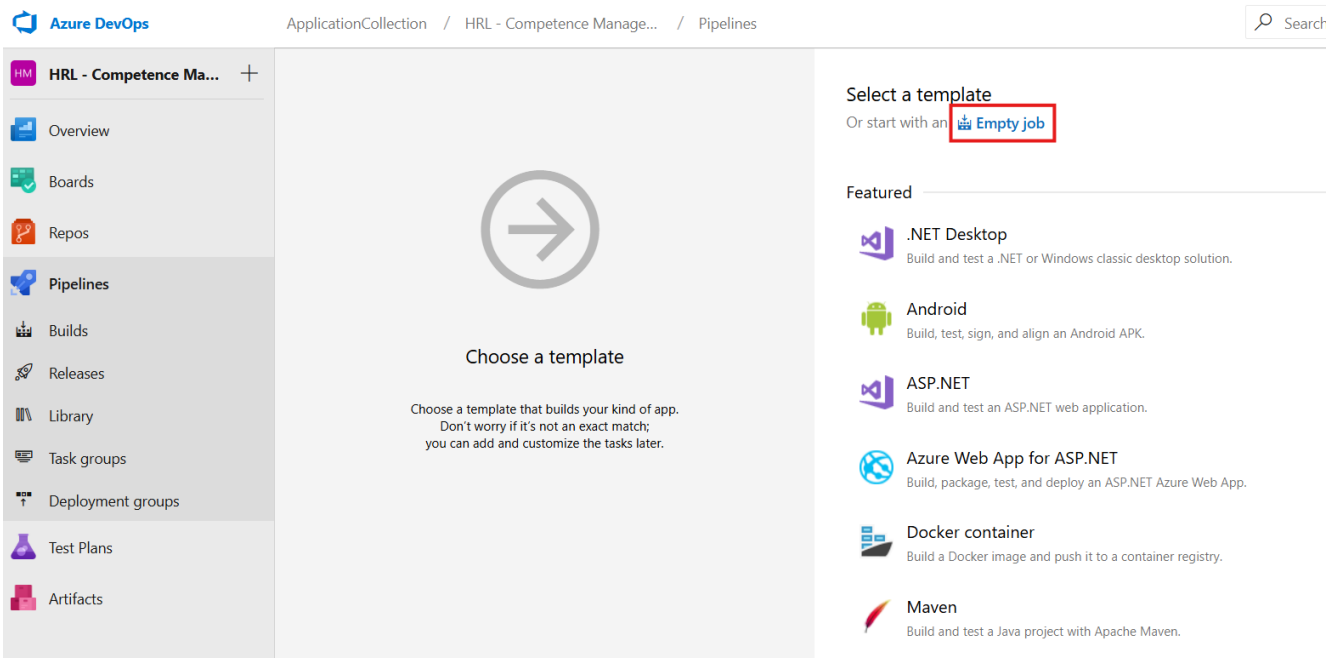
- Connection name: BE
- Git repository URL: Link repo Git của Backend
- User name: NTID
- Password / Token key: Access Token đã được tạo ở step 7.

Sau đó click “OK”.

Step 9. Chỉ định branch sử dụng để deploy và chọn “Continue”.

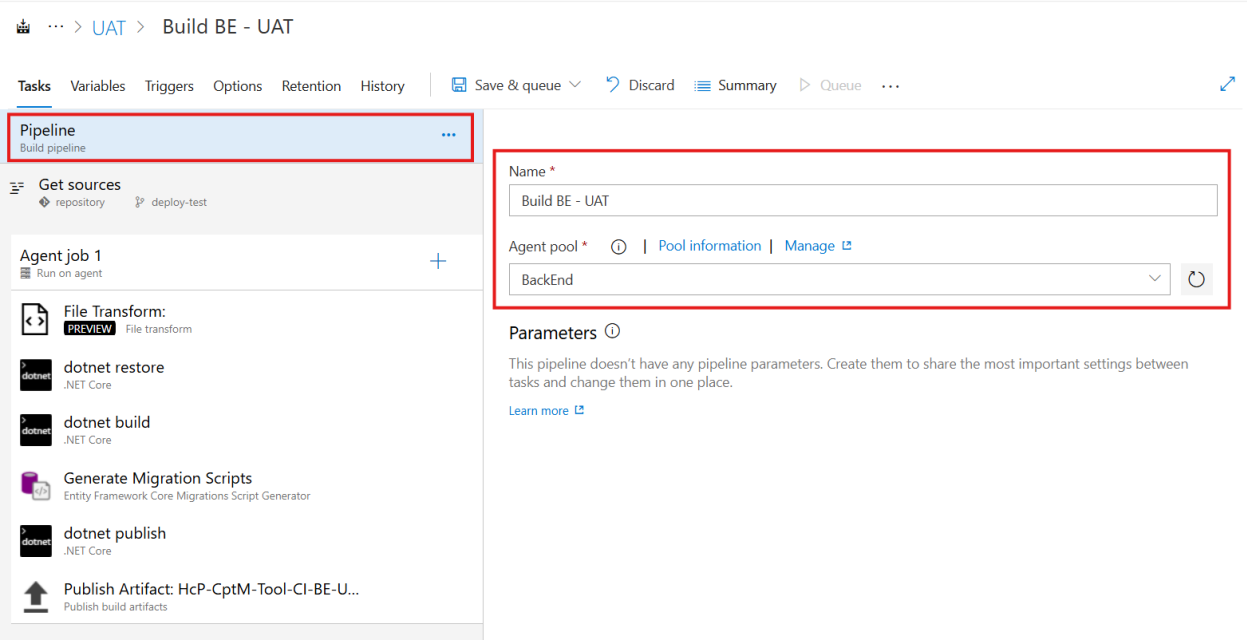


Step 10. Chọn “Empty job”.

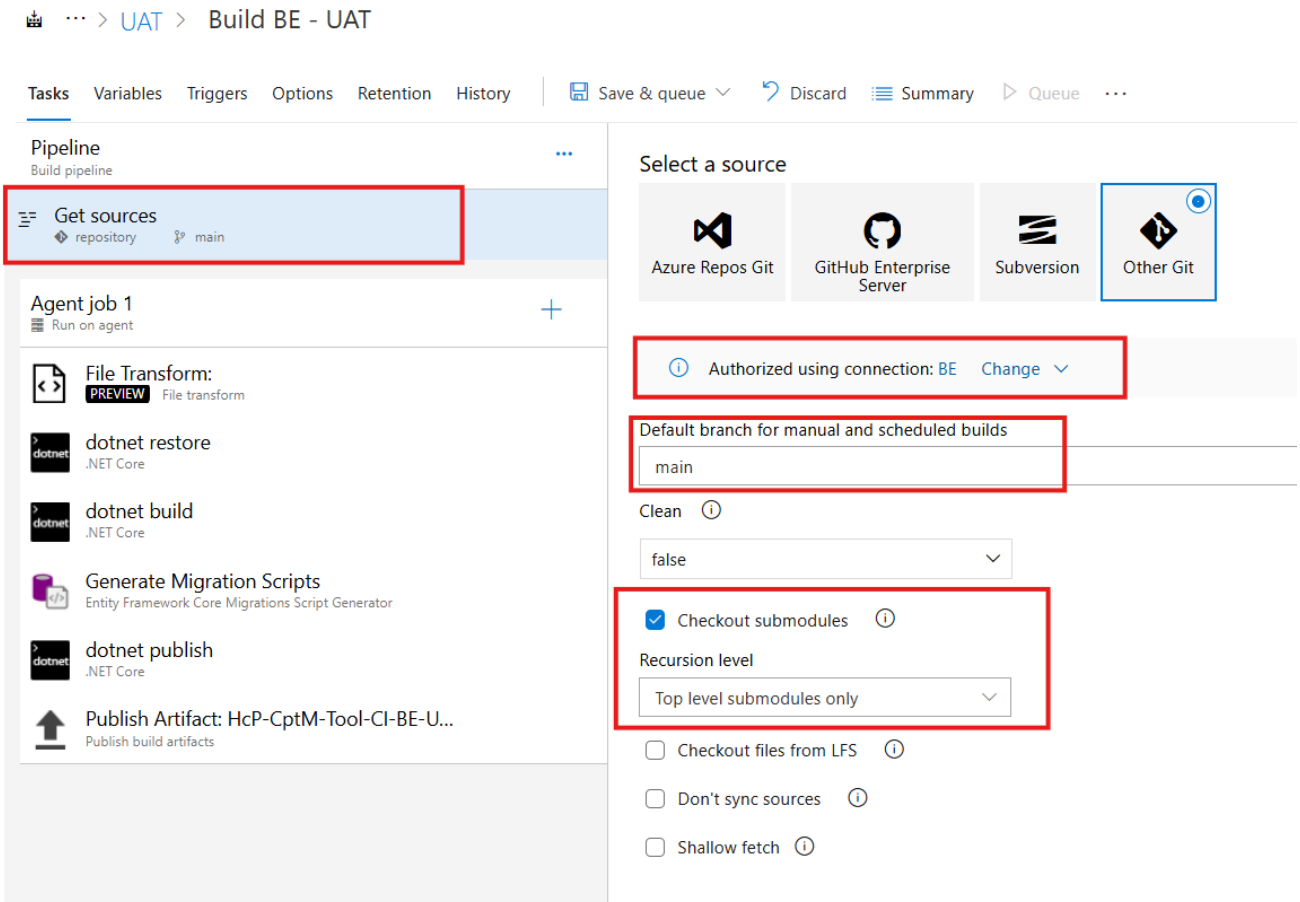


Step 11. Fill thông tin Pipeline

- Name: Build BE - <ENVIRONMENT>
- Agent pool: Backend

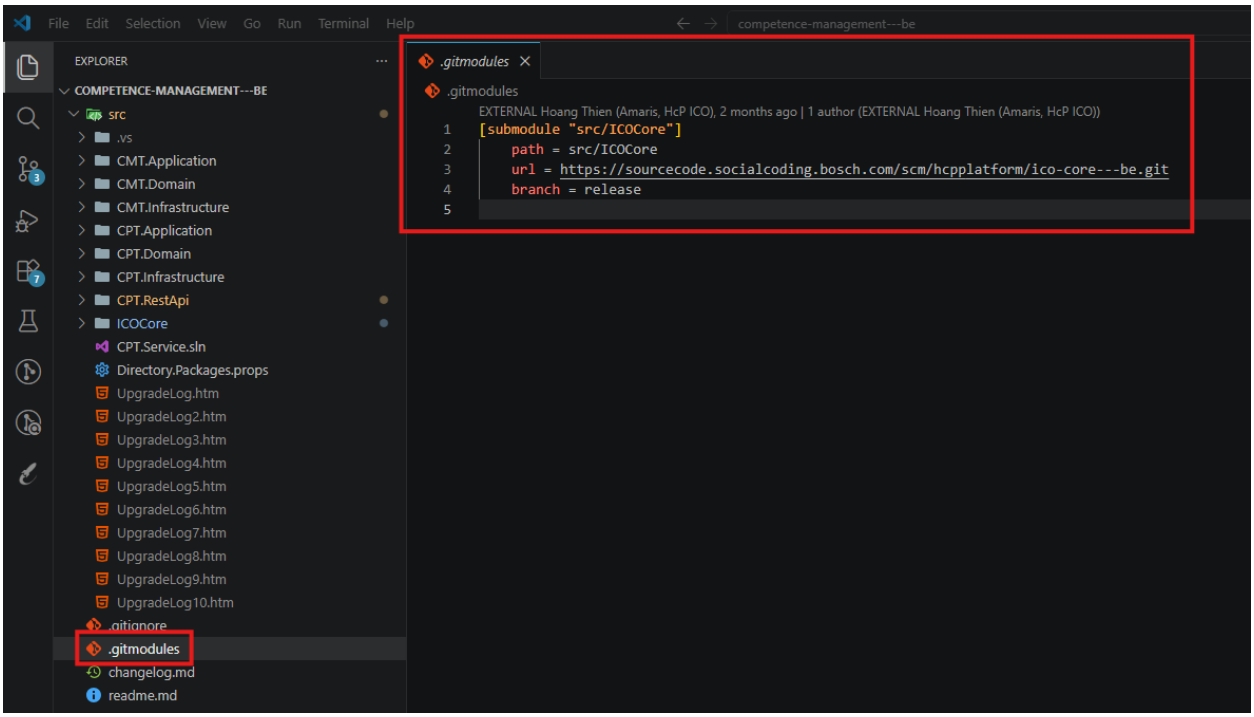


Step 12. Fill thông tin Get source

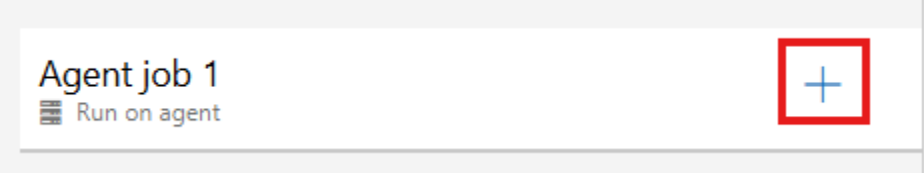


- Authorized using connection: Chọn “BE” (đã được tạo ở Step 8.)
- Select default branch sẽ được build
- Tick chọn “Checkout submodules” nếu sử dụng submodules và chọn “Top level submodules only” tại Recursion level.

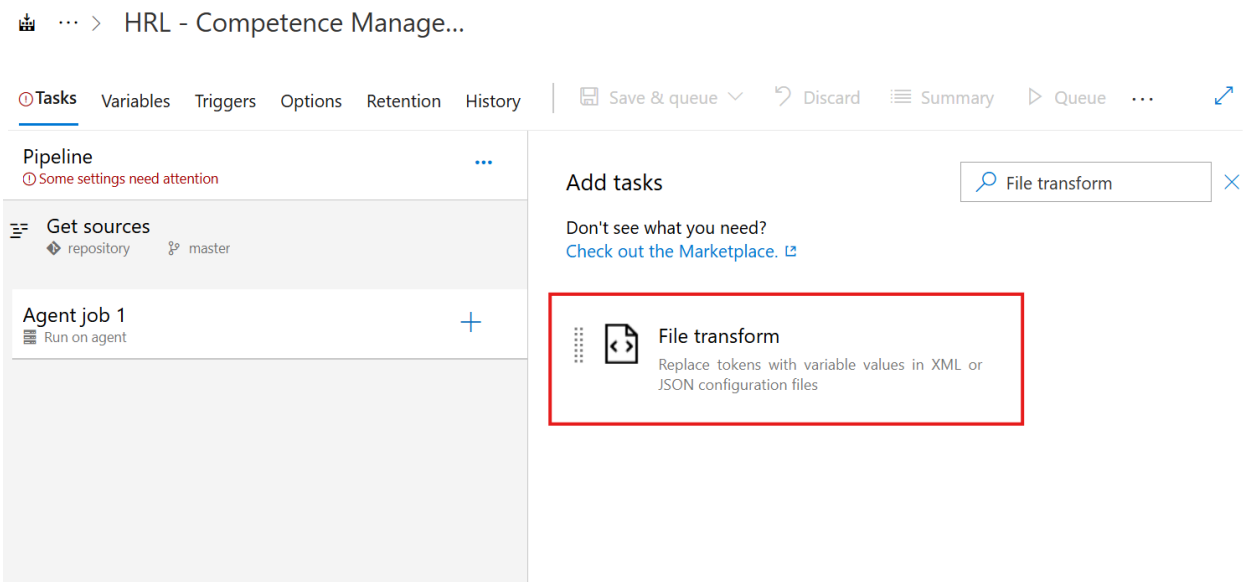
* Lưu ý: Khi sử dụng submodule, cần khai báo .gitsubmodule tại folder root.



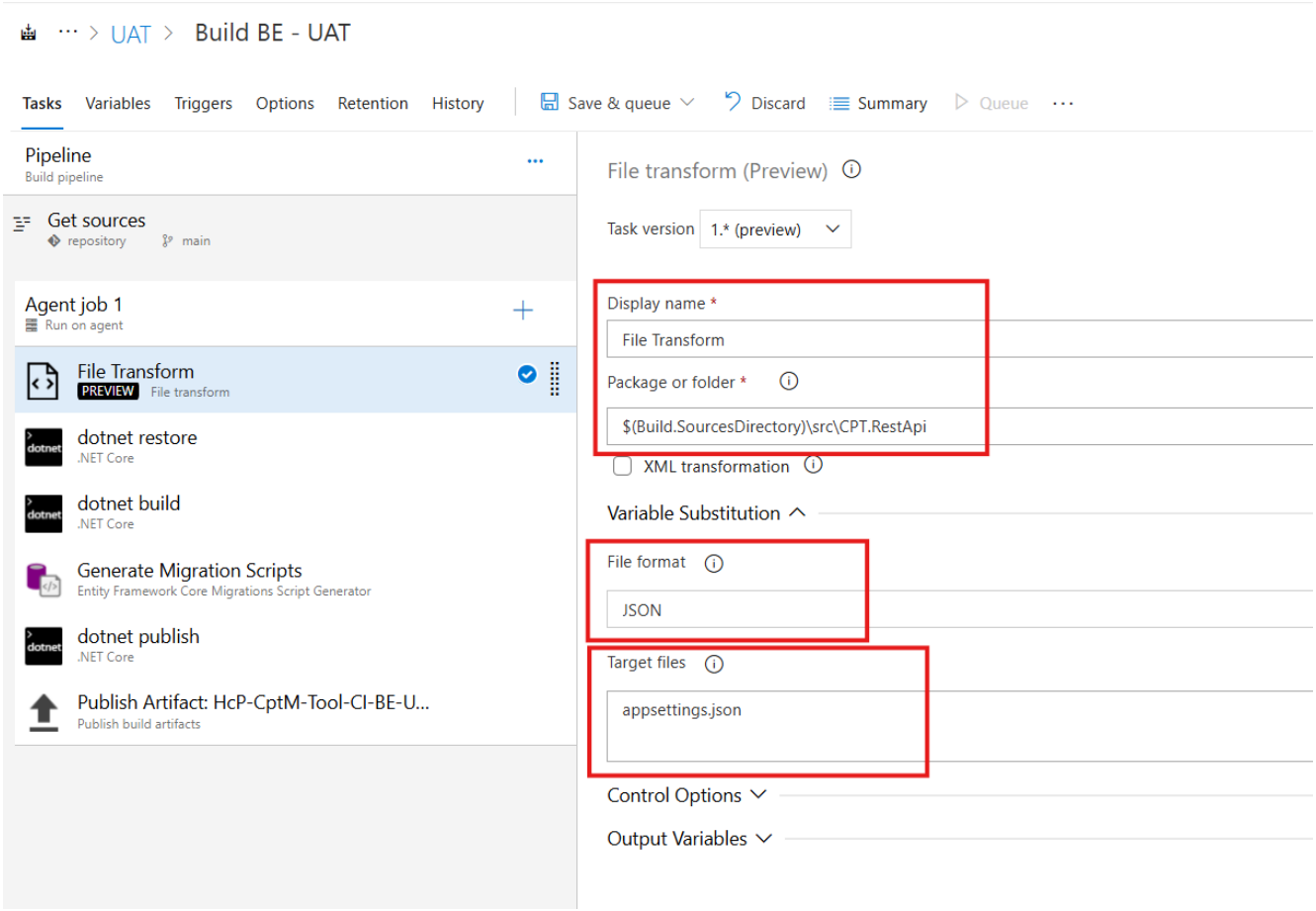
Step 13. Tại Agent job, click + button.



Tìm và add task “File transform”

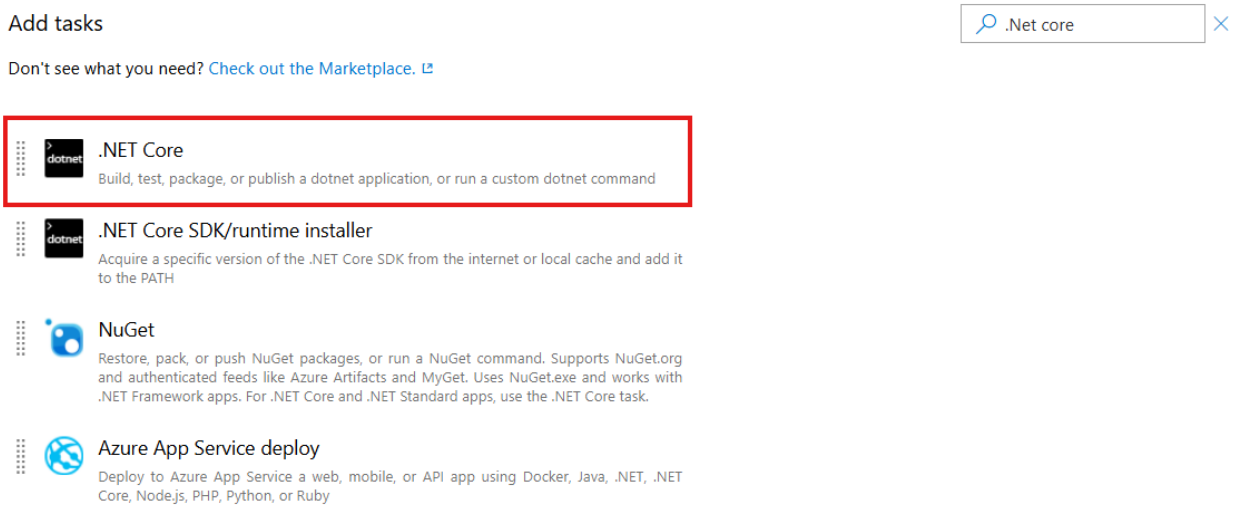


Sau đó, fill các thông tin như sau:

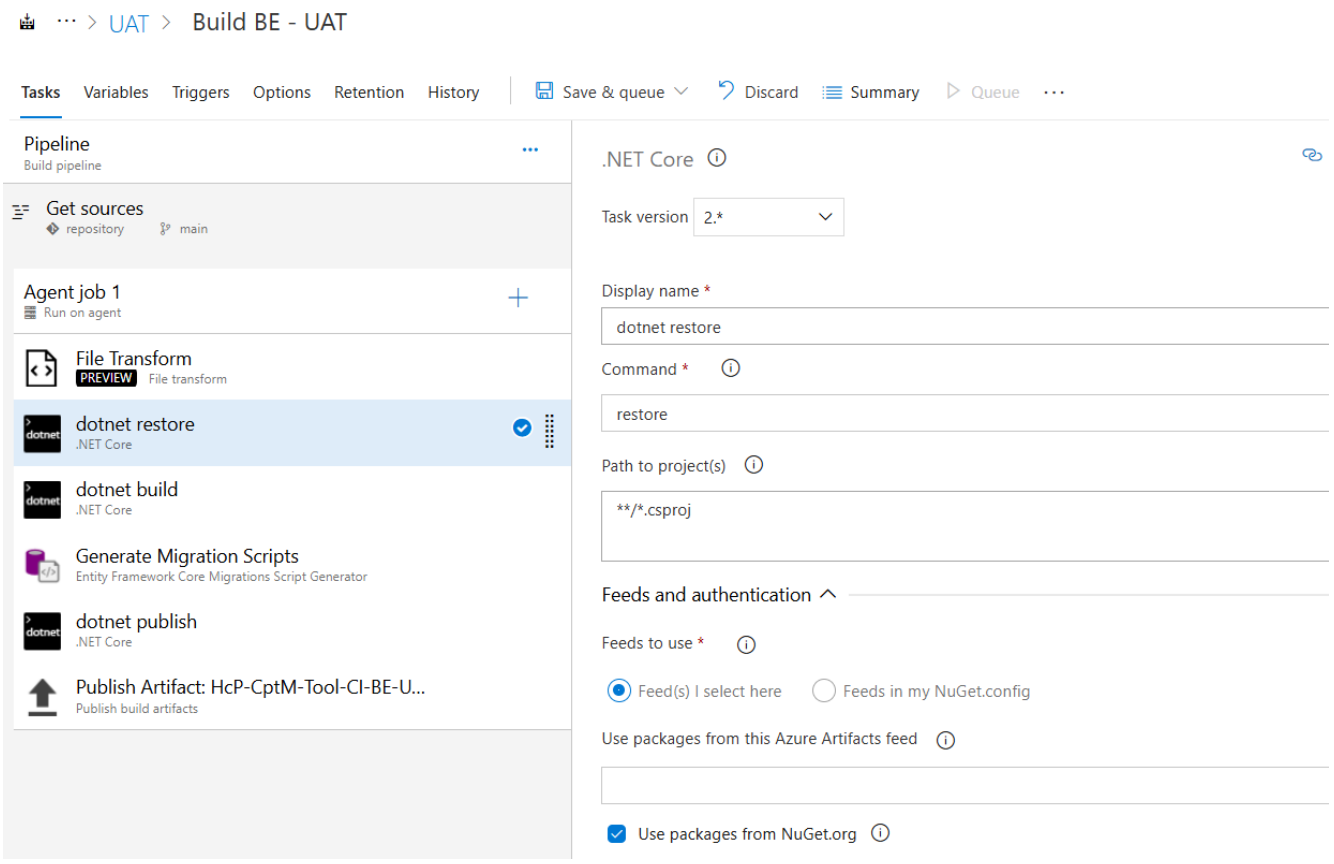


Hãy điều chỉnh “Package or folder” lại đúng với project đang được deploy.

Step 14. Tại Agent job, tìm và add task ".NET Core"

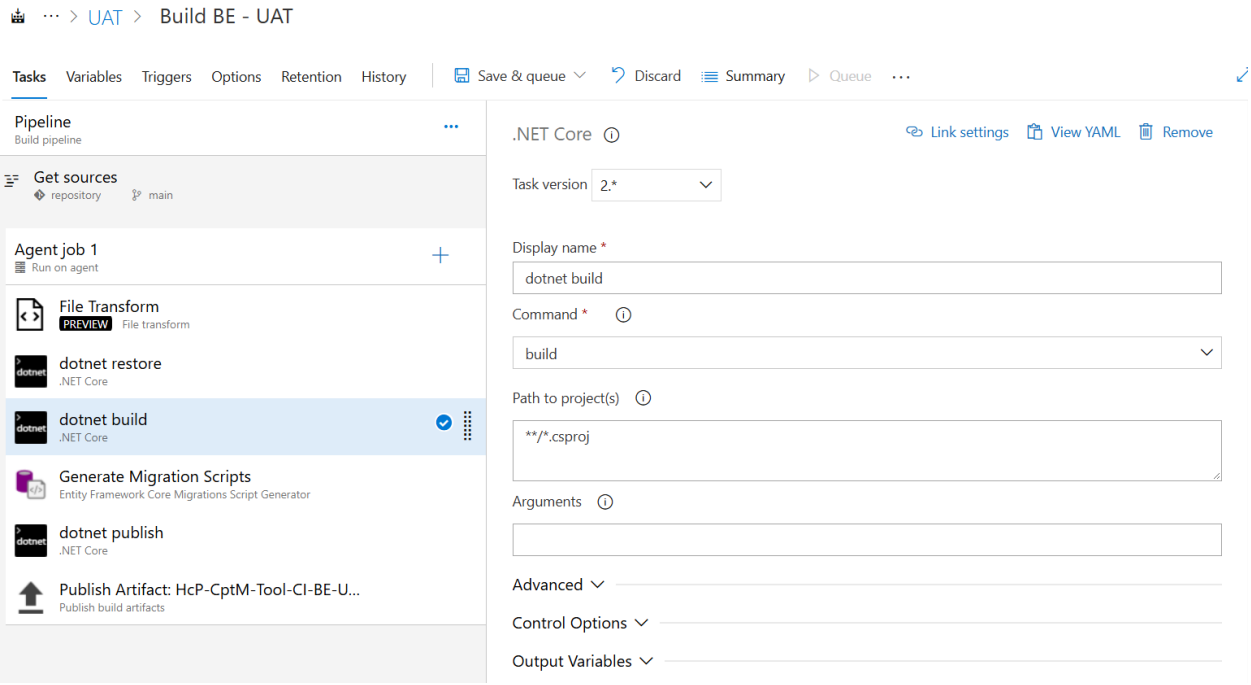


Sau đó, fill các thông tin như sau:

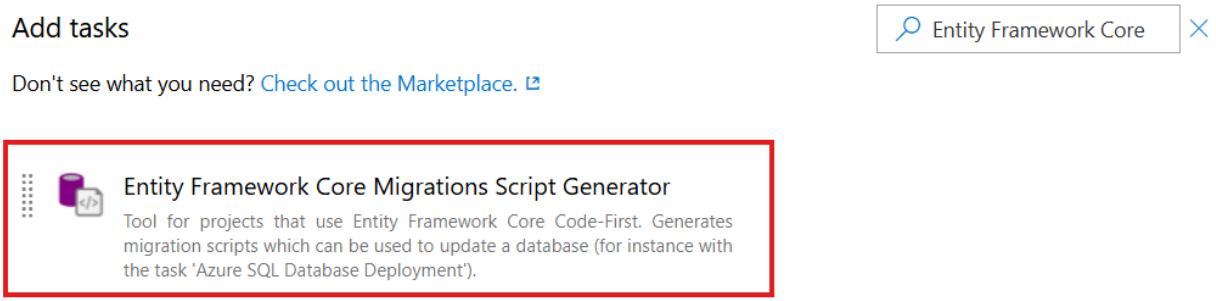


Step 15. Tại Agent job, tiếp tục tìm và add task ".NET Core"

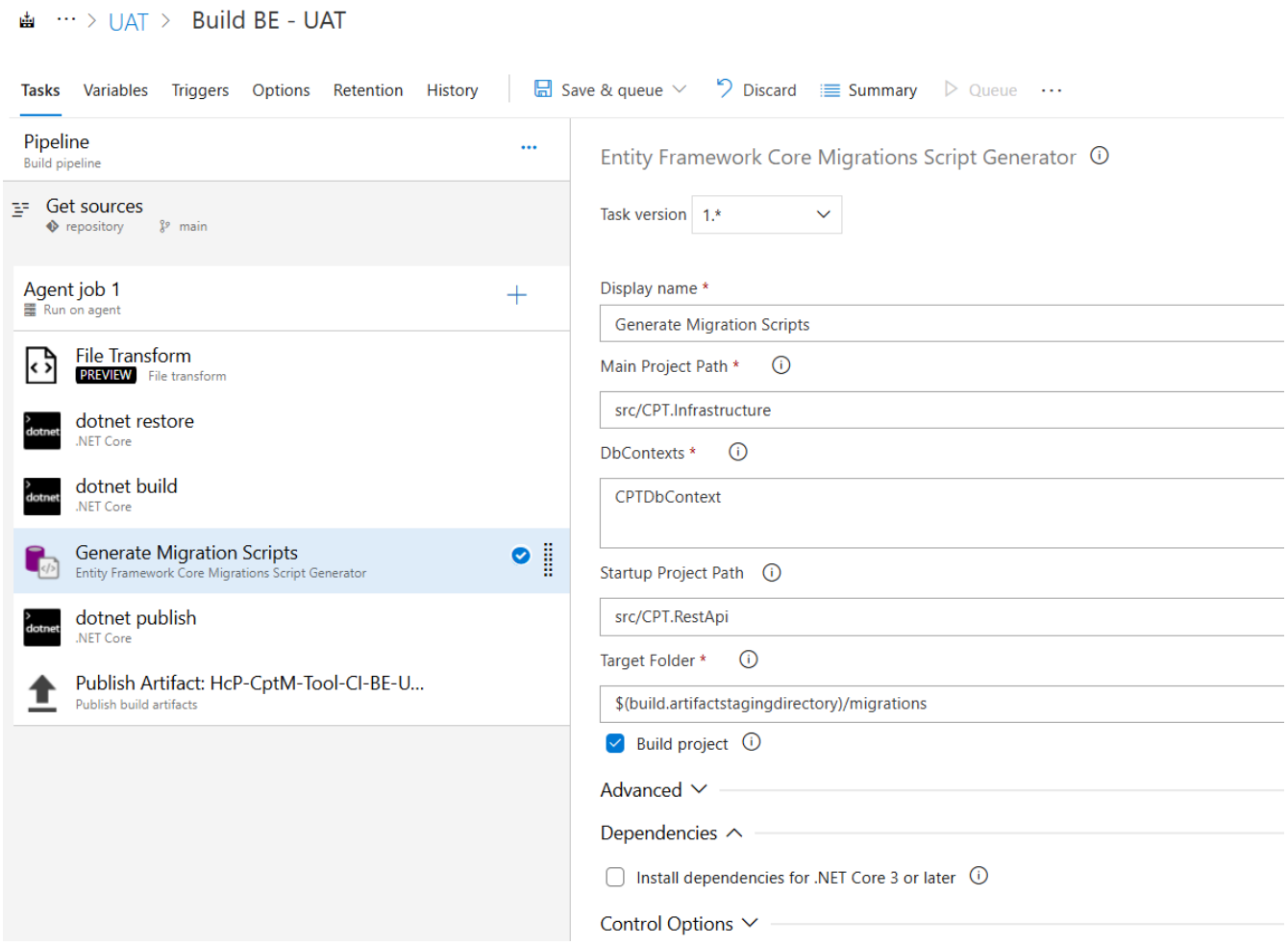
Sau đó, fill các thông tin như sau:



Step 16. Tại Agent job, tìm và add task "Entity Framework Core Migration Script Generator".



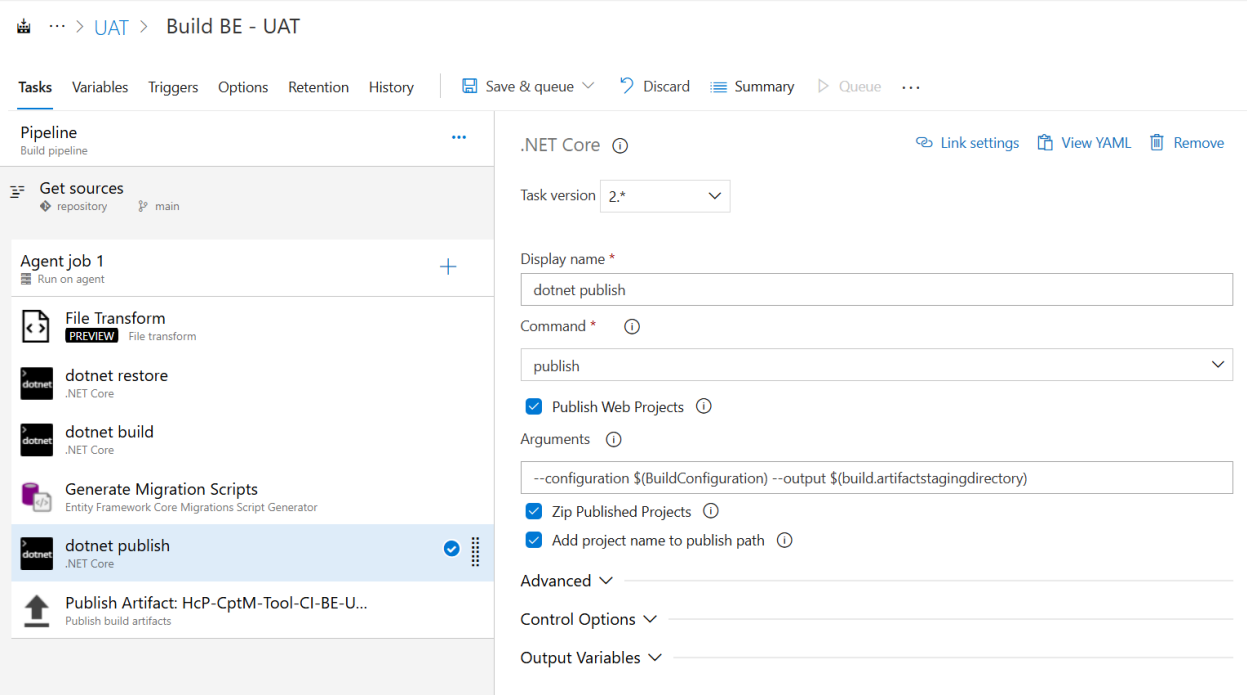
Sau đó, fill các thông tin như sau:



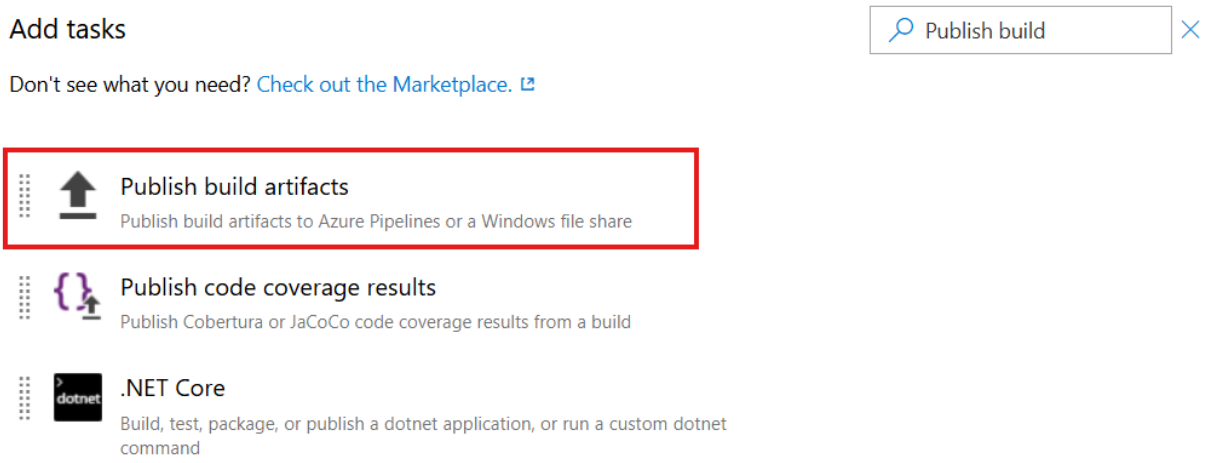
Hãy điều chỉnh lại các thông tin “Main Project Path”, “DbContexts”, “Startup Project Path” để đúng với dự án của bạn.

Step 17. Tại Agent job, tiếp tục tìm và add task “.NET Core”

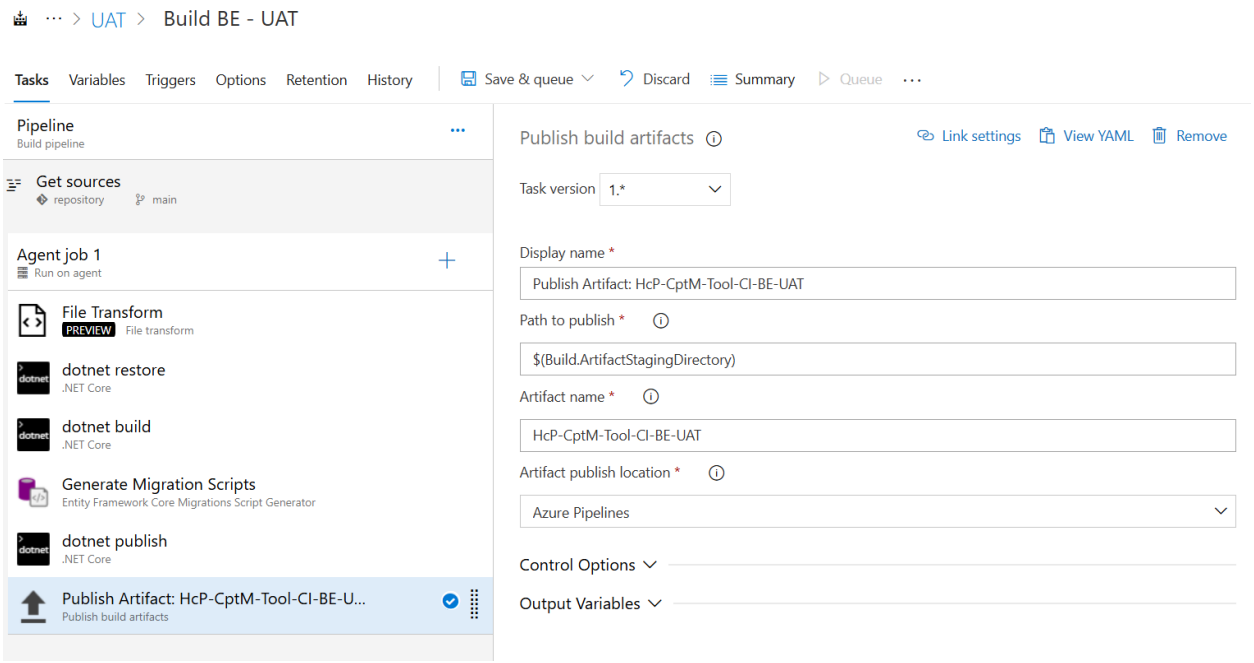
Sau đó, fill các thông tin như sau:



Step 18. Tại Agent job, tìm và add task “Publish build artifacts”.

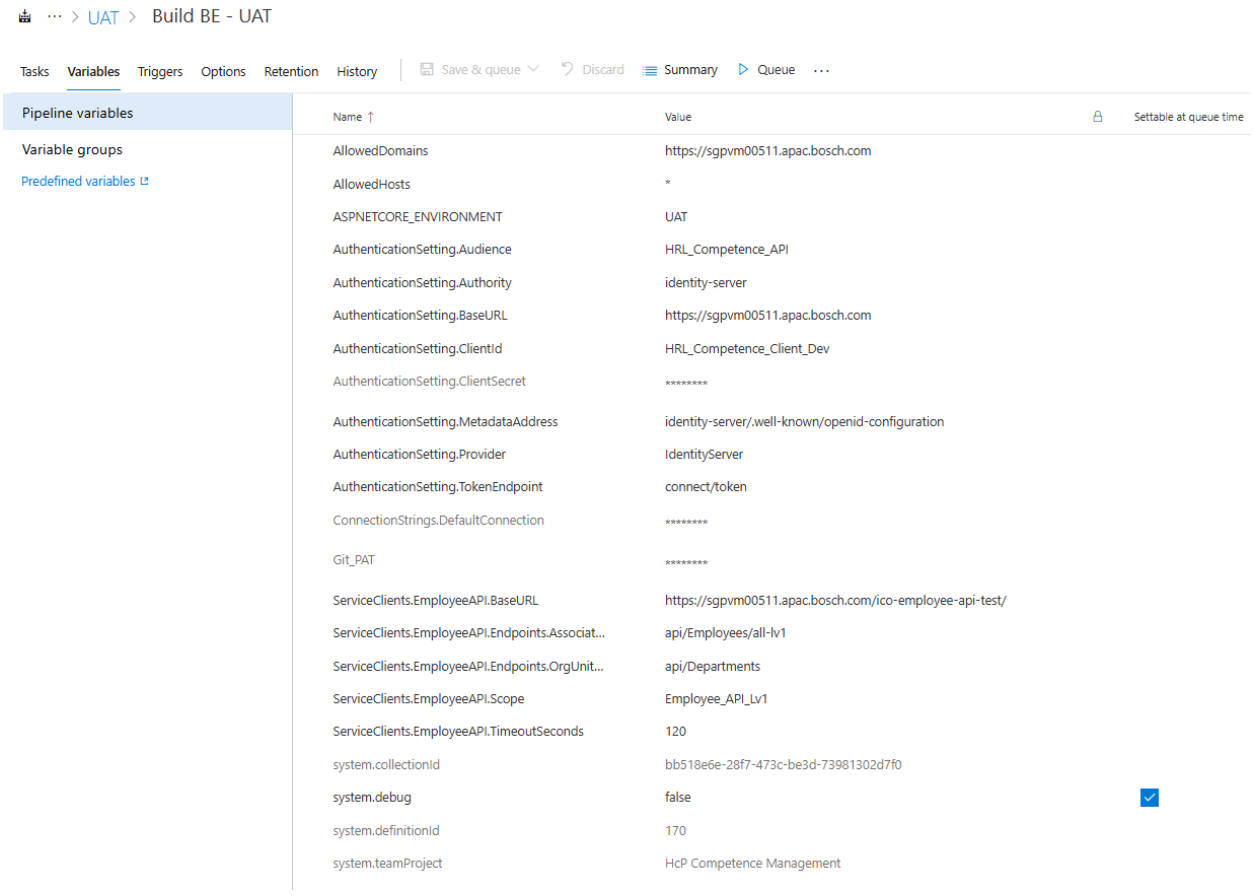


Sau đó, fill các thông tin như sau:



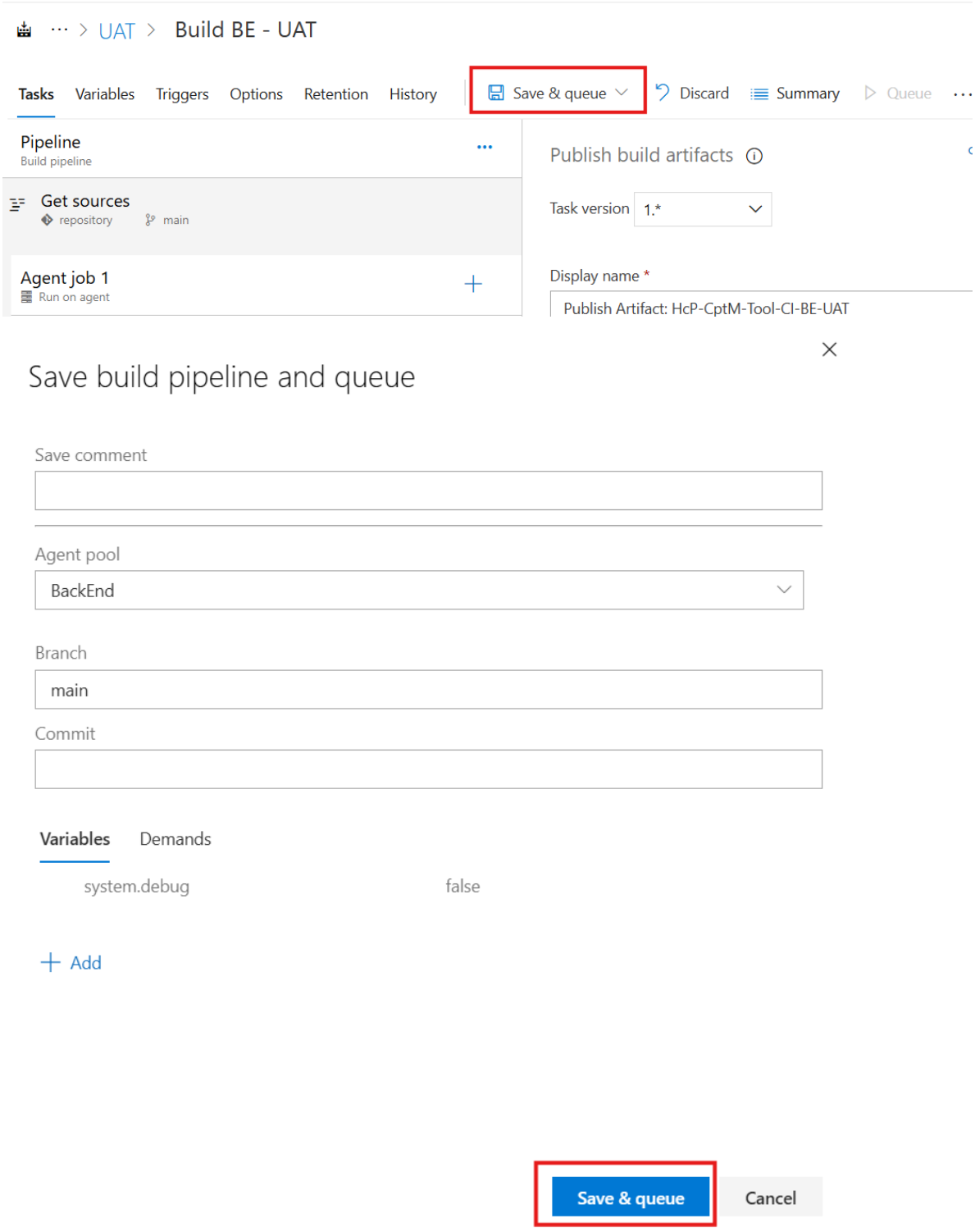
Hãy điều chỉnh lại Artifact Name theo format <Tên dự án>-CI-BE-<Environment>.

Step 19. Chọn “Variables”, sau đó fill các biến được define trong appsettings.json vào:

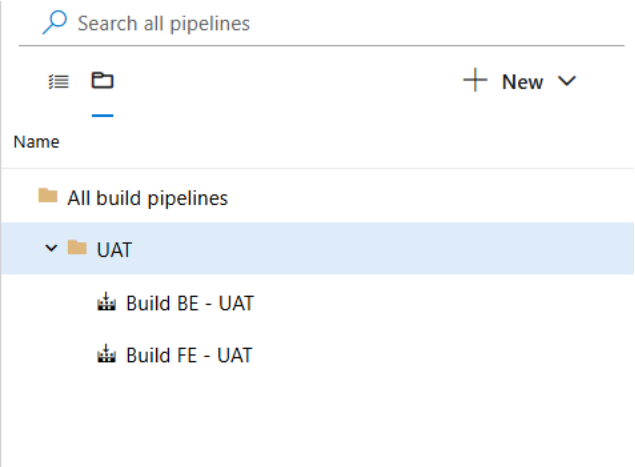


* Lưu ý: Connection String và Client Secret cần phải tick chọn “Change variable type to secret”

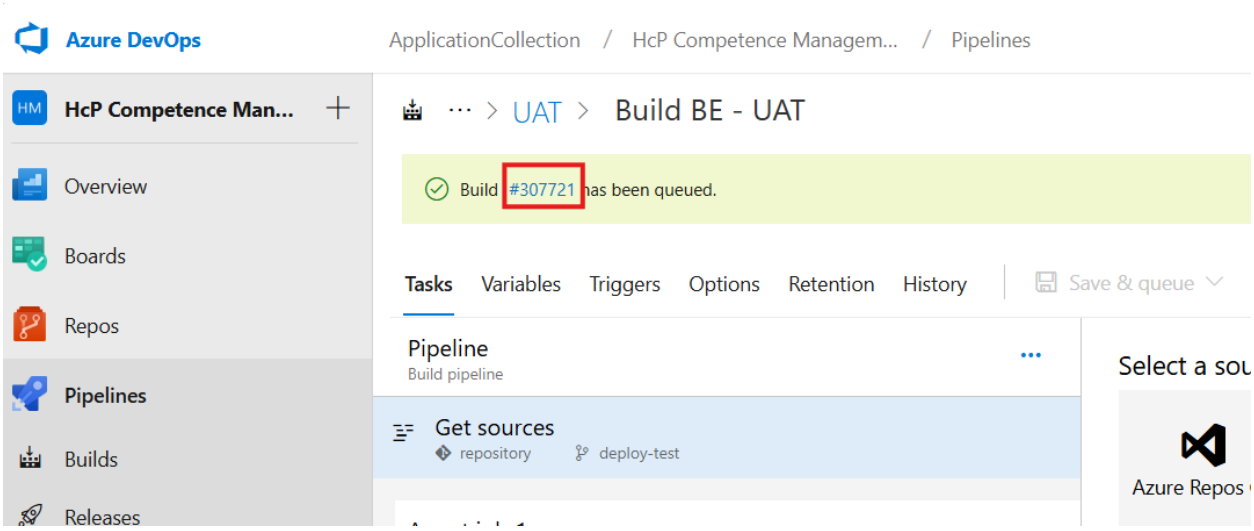
Step 20. Chọn “Save & queue”.

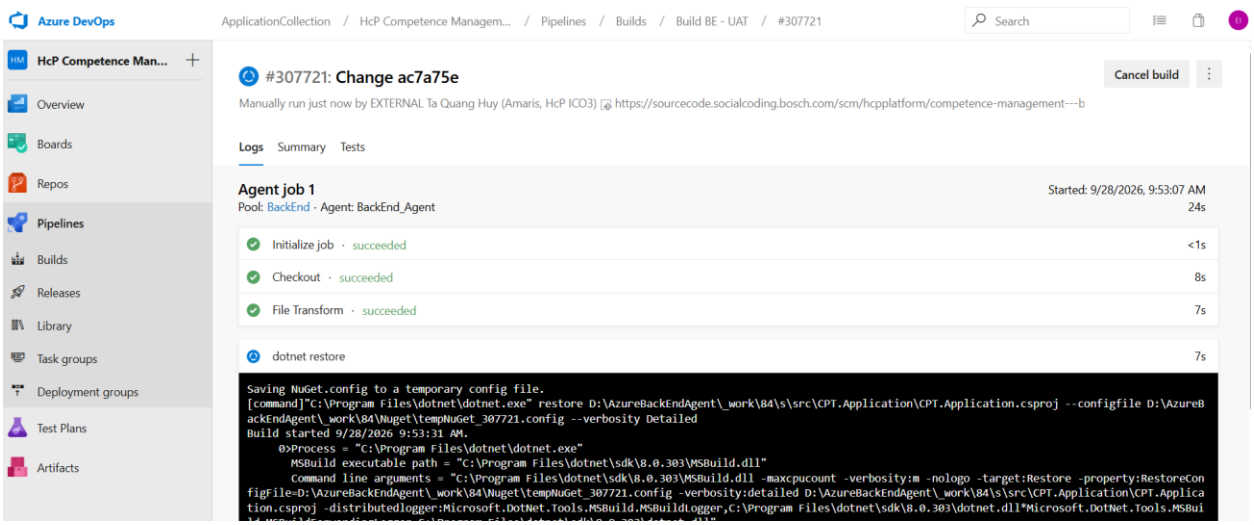


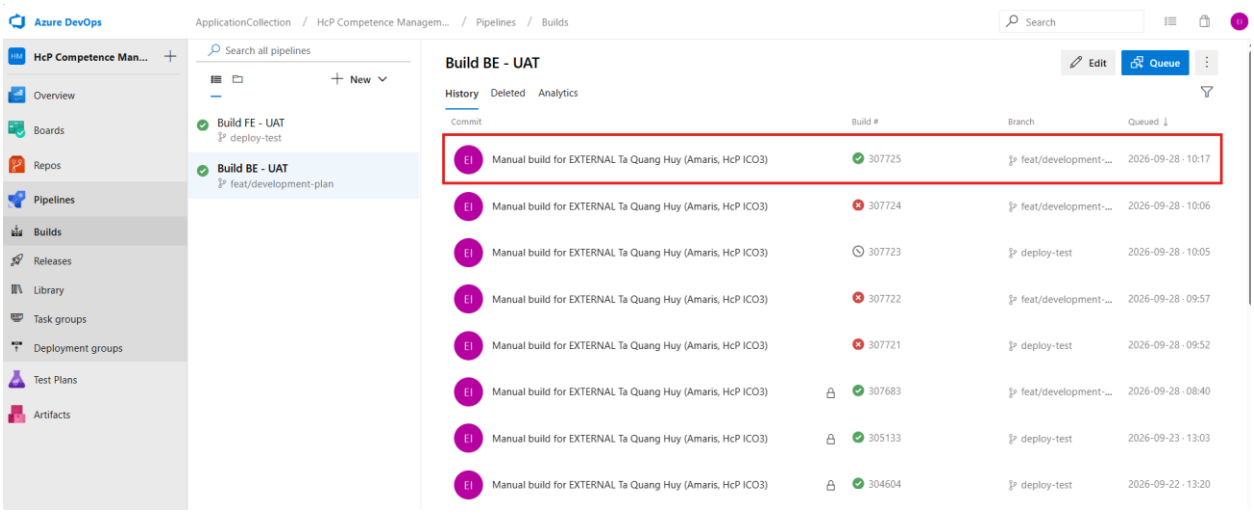
Hãy lưu folder theo format thư mục <Environment>/<Build Name>, ví dụ:



Step 20. Chọn build Id để theo dõi quá trình build.



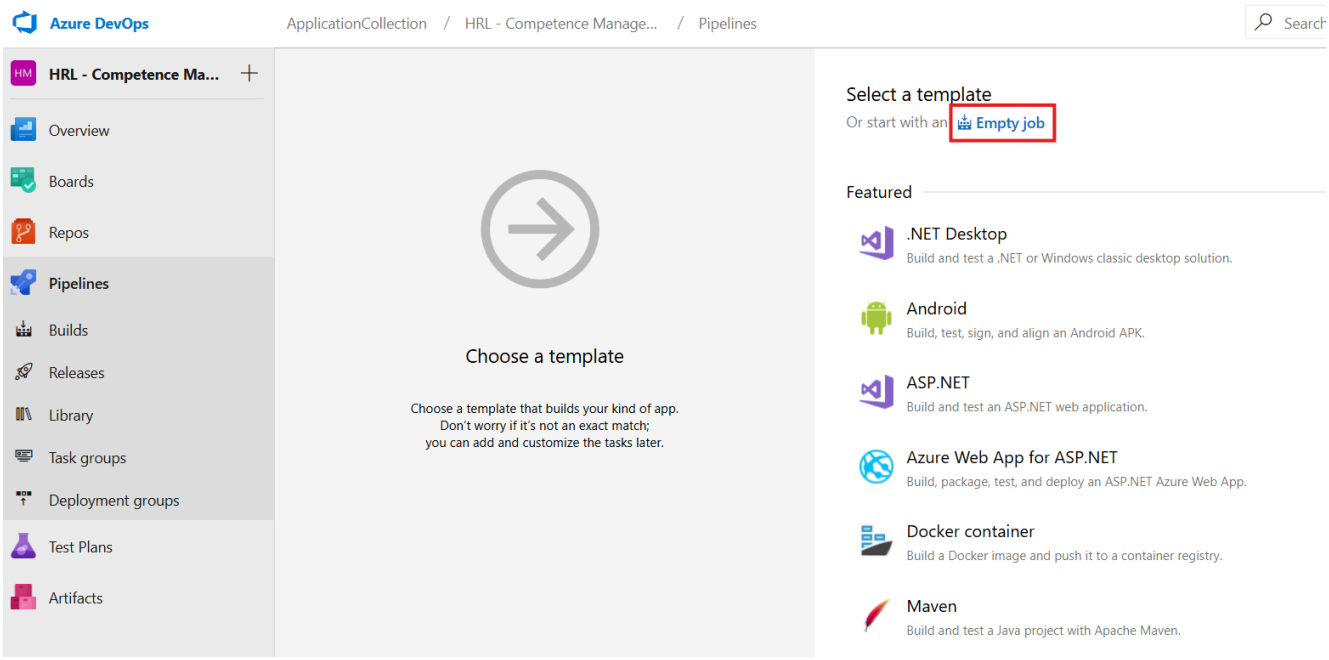




1.2. Set up CI cho FrontEnd

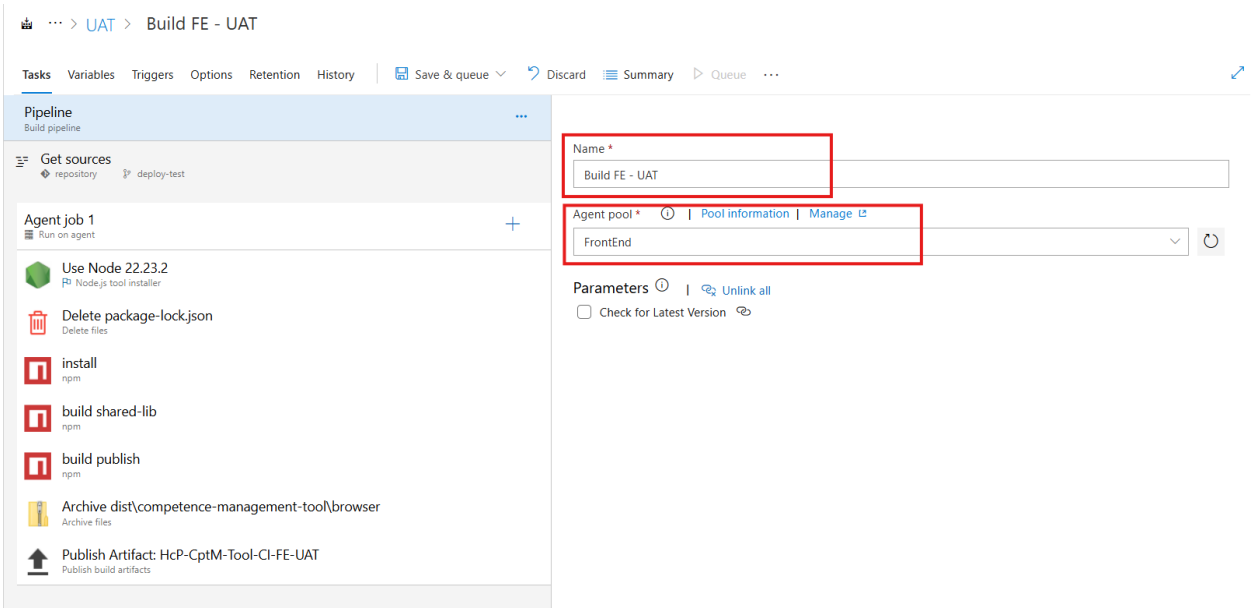
Step 1. Thực hiện tương tự step 1 đến step step 10 như mục “1.1. Set up CI cho backend” để set up Git connection cho repo Fontend. Có thể sử dụng lại Git Access Token mà không cần tạo mới.

Step 2. Tiếp tục chọn “Empty job”.

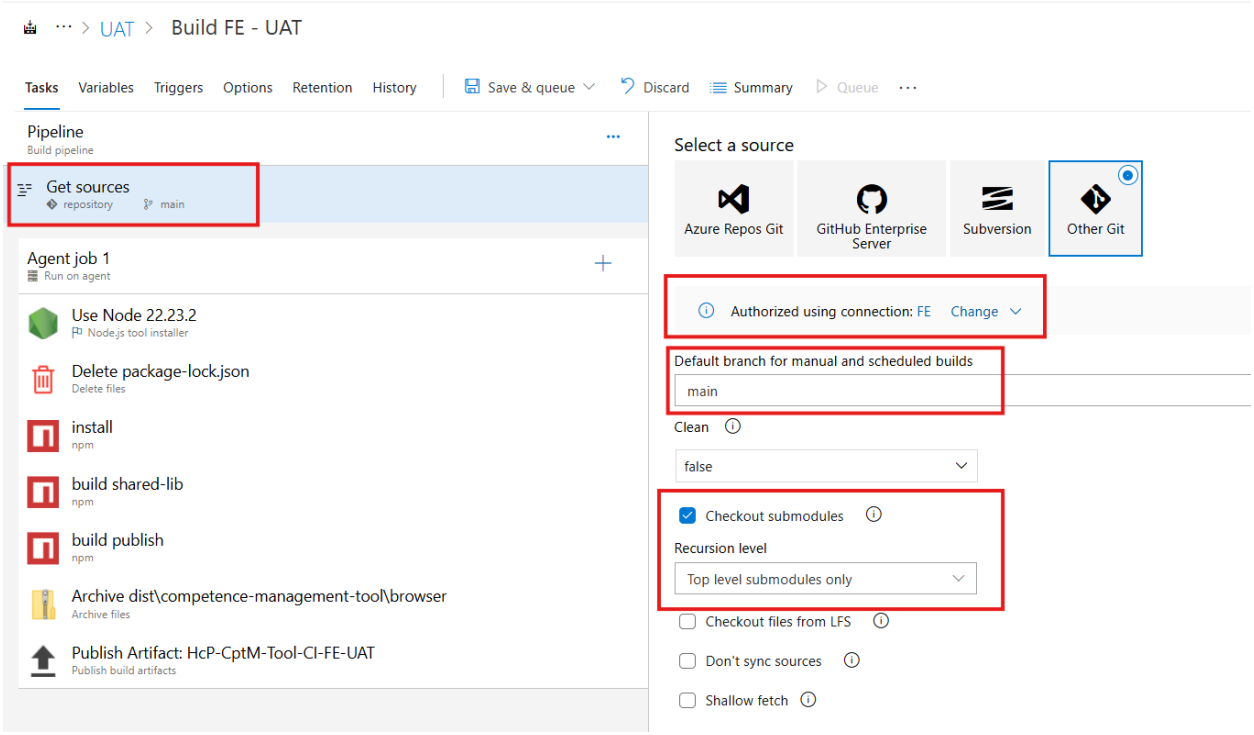


Step 3. Fill thông tin Pipeline

- Name: Build FE - <ENVIRONMENT>
- Agent pool: FrontEnd



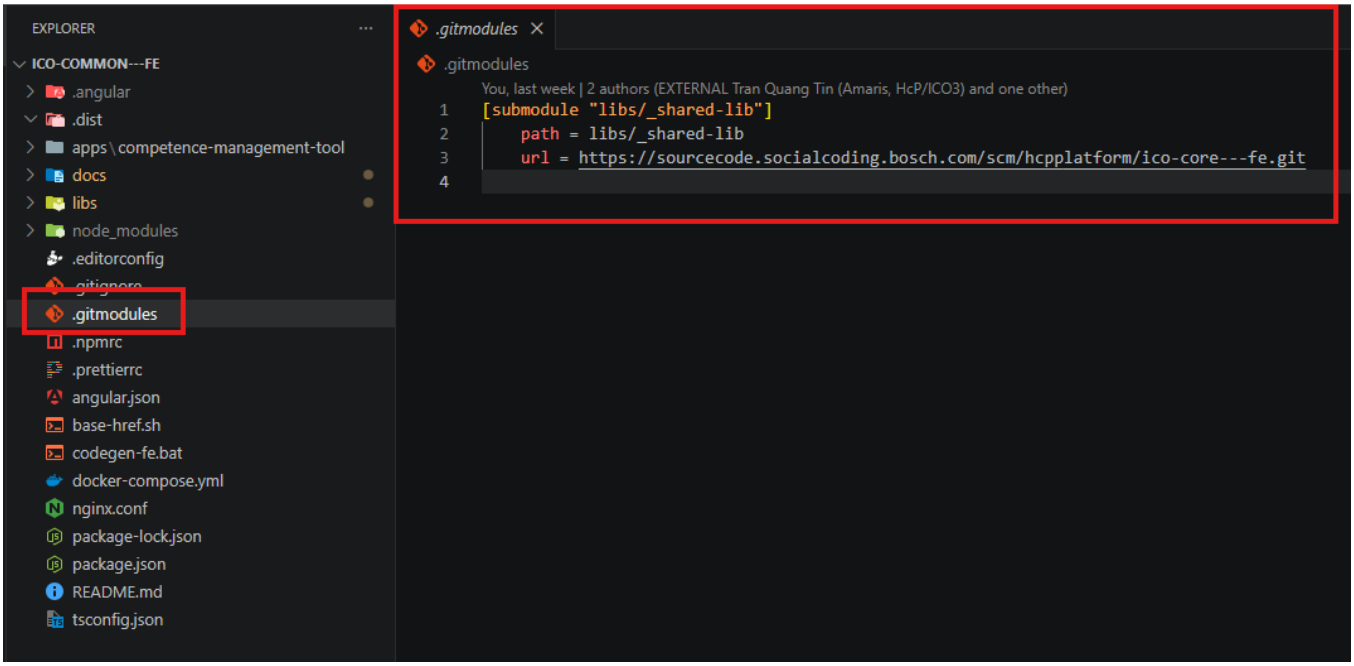
Step 4. Fill thông tin Get source



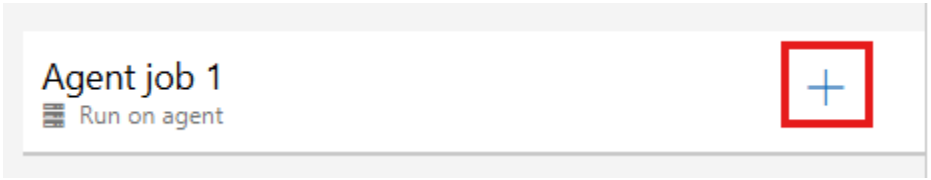
- Authorized using connection: Chọn “FE” (Git connection đã được tạo ở các step trước”
- Select default branch sẽ được build

- Tick chọn “Checkout submodules” nếu sử dụng submodules và chọn “Top level submodules only” tại Recursion level.

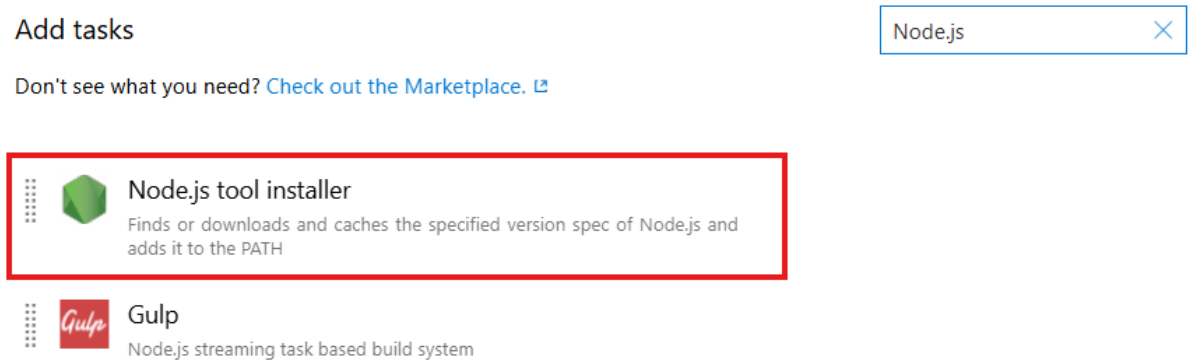
* Lưu ý: Khi sử dụng submodule, cần khai báo .gitsubmodule tại folder root.



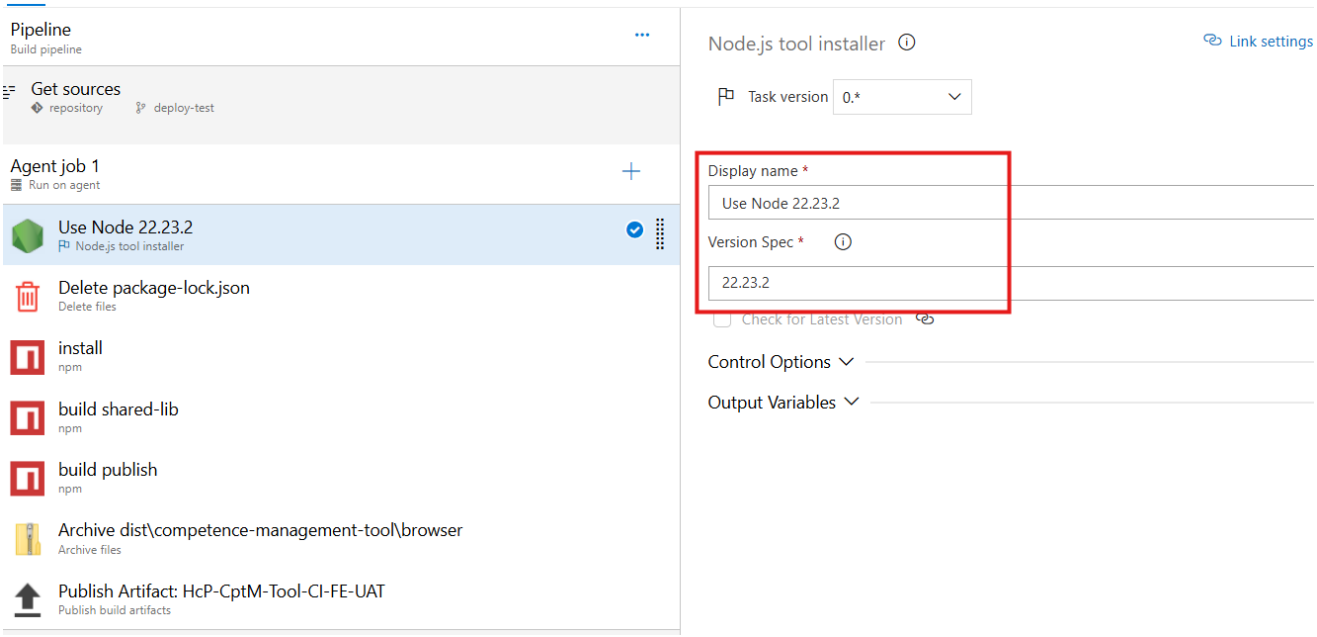
Step 5. Tại Agent job, click + button.



Tìm và add task “Node.js tool installer”

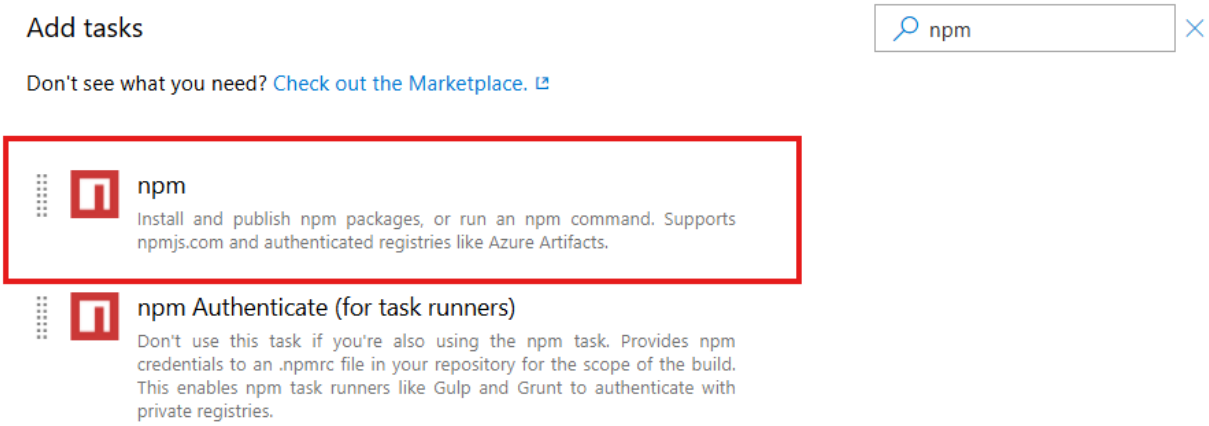


Sau đó, fill thông tin Node.js version phù hợp với dự án. Đối với dự án sử dụng Angular 22, có thể sử dụng Version 22.23.2.

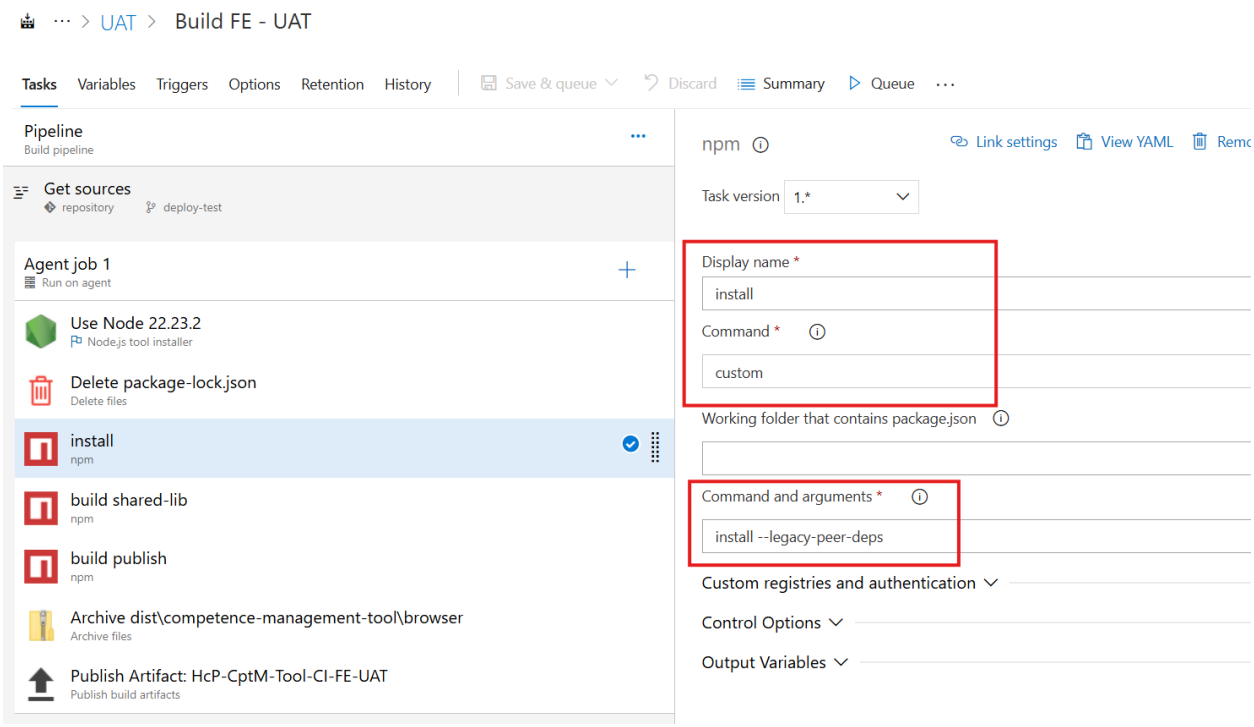


Step 6. Tại Agent job, click + button.

Tìm và add task “npm”.

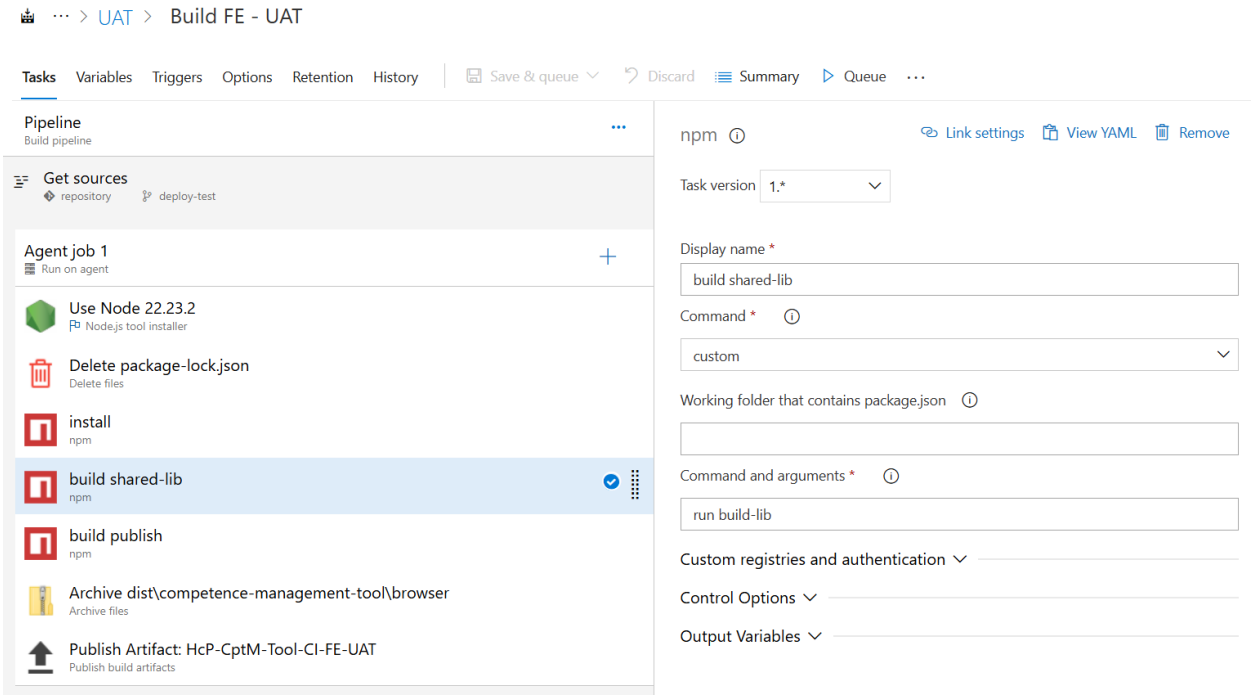


Sau đó, fill các thông tin như sau:



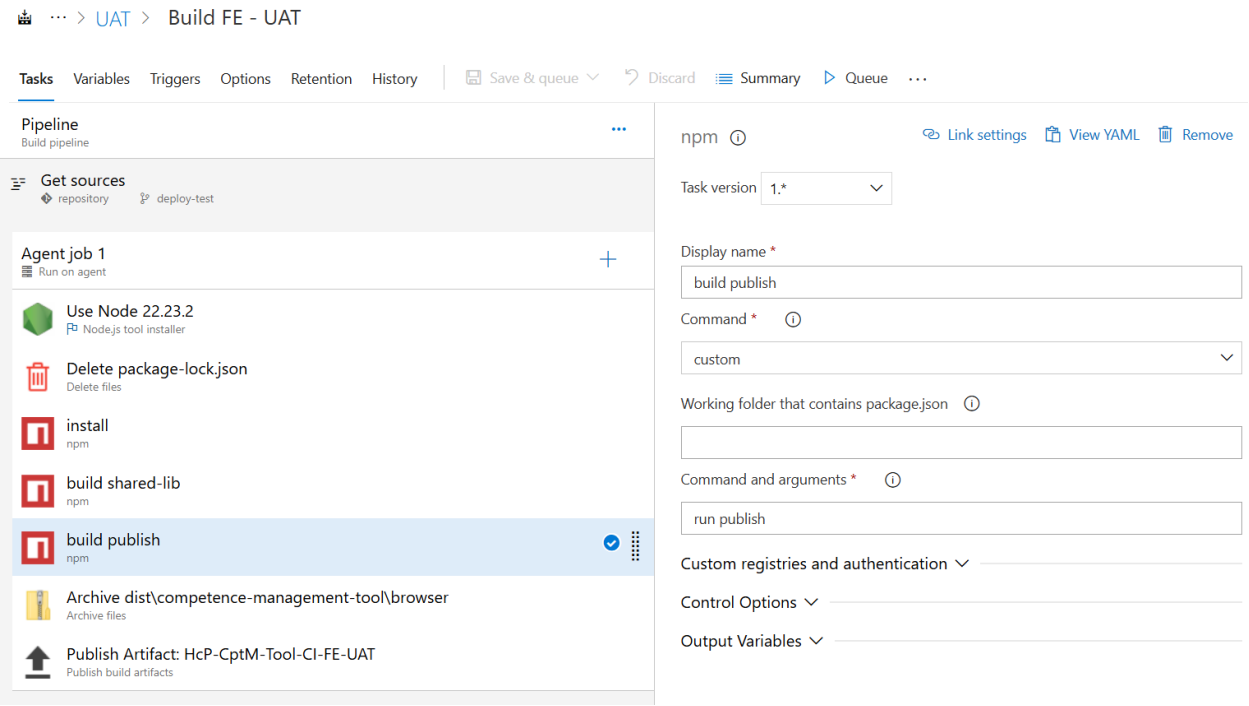
Step 7. Tại Agent job, click + button.

Tìm và tiếp tục add task “npm”. Sau đó, fill các thông tin như sau:



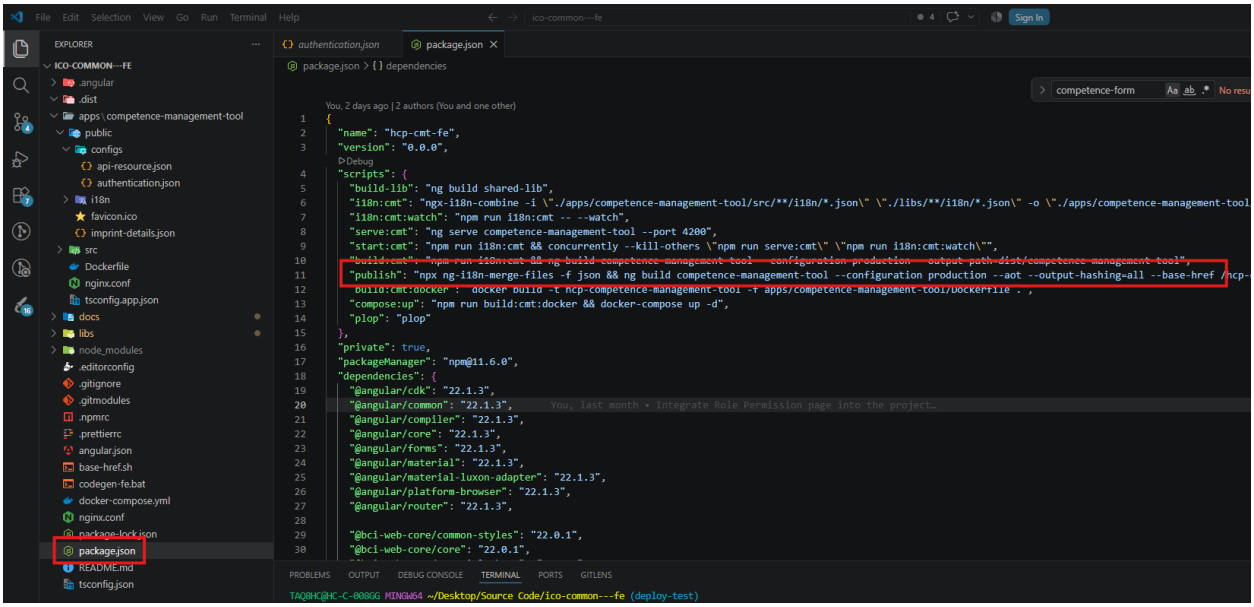
Step 8. Tại Agent job, click + button.

Tìm và tiếp tục add task “npm”. Sau đó, fill các thông tin như sau:



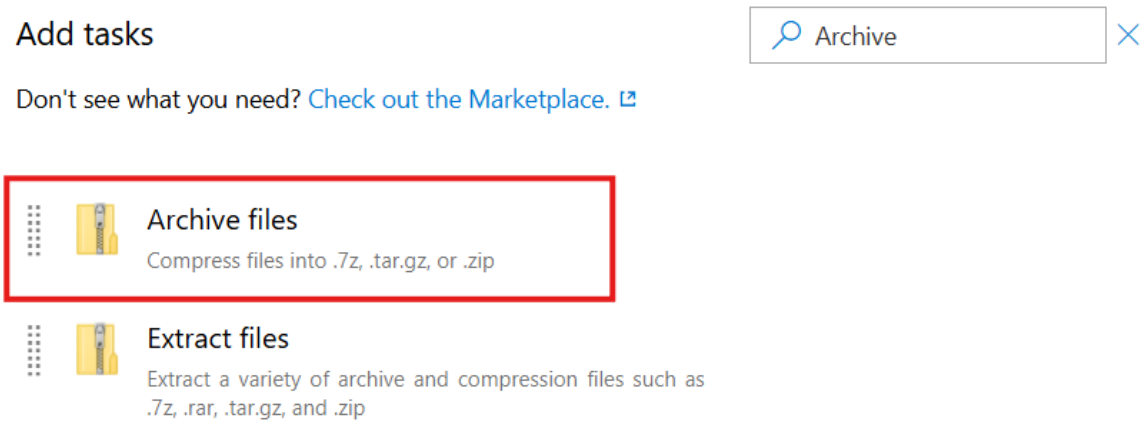
* Lưu ý: publish script cần phải được define trong package.json như sau:

"publish": "npx ng-i18n-merge-files -f json && ng build competence-management-tool --configuration production --aot --output-hashing=all --base-href /hcp-cpt/",

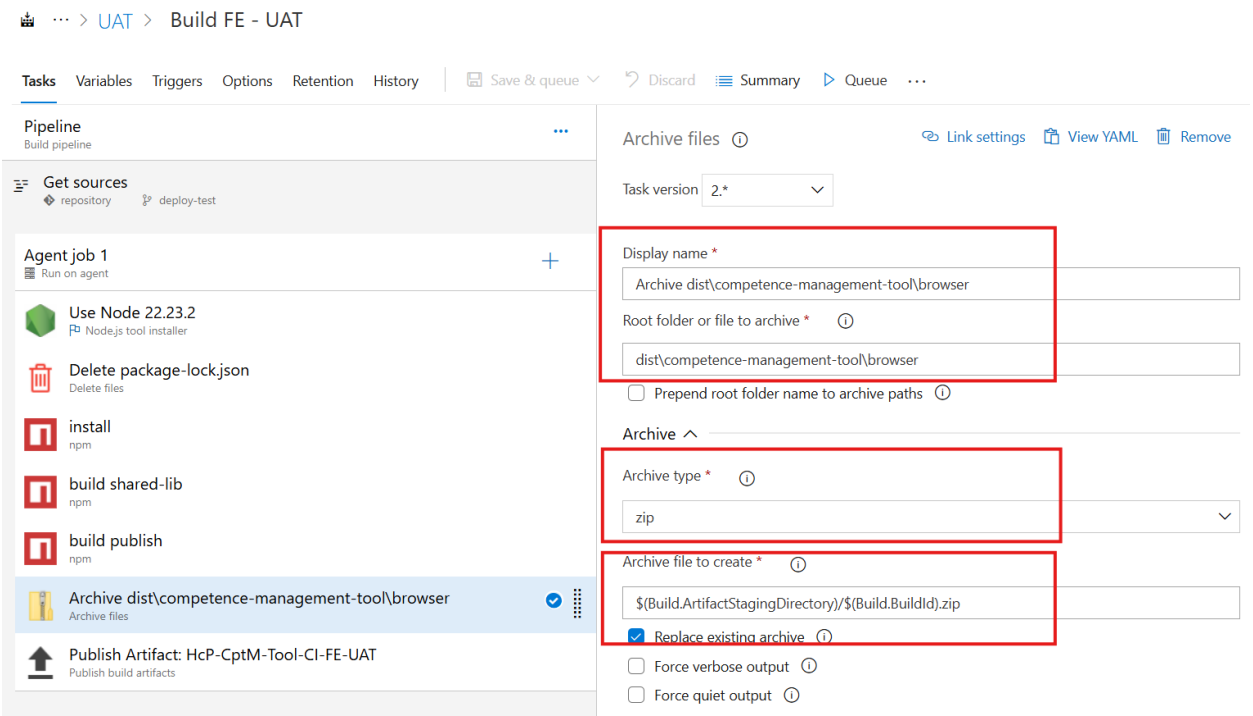


Step 9. Tại Agent job, click + button.

Tìm và add task “Archive files”.



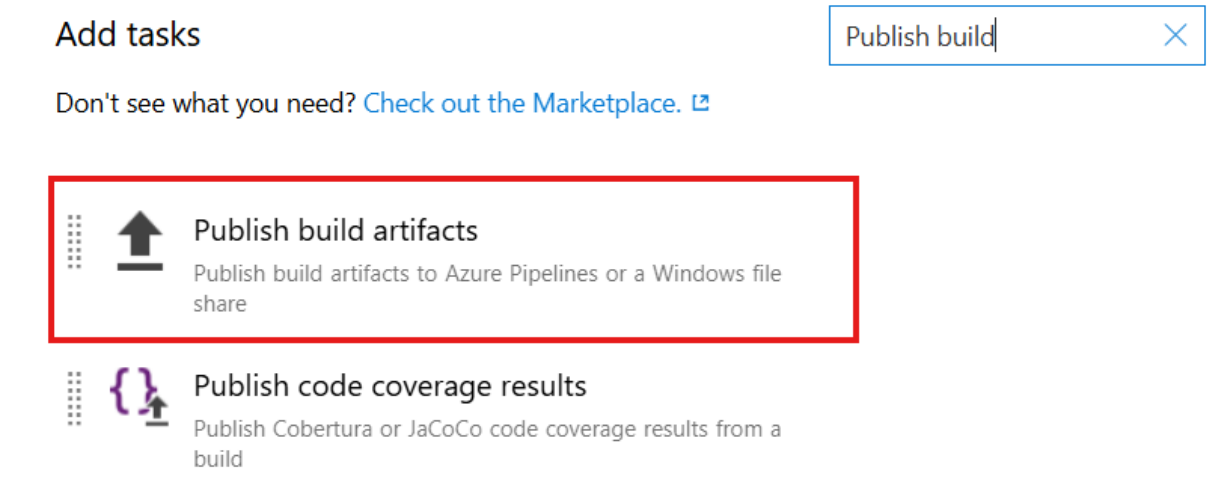
Sau đó, fill các thông tin như sau:



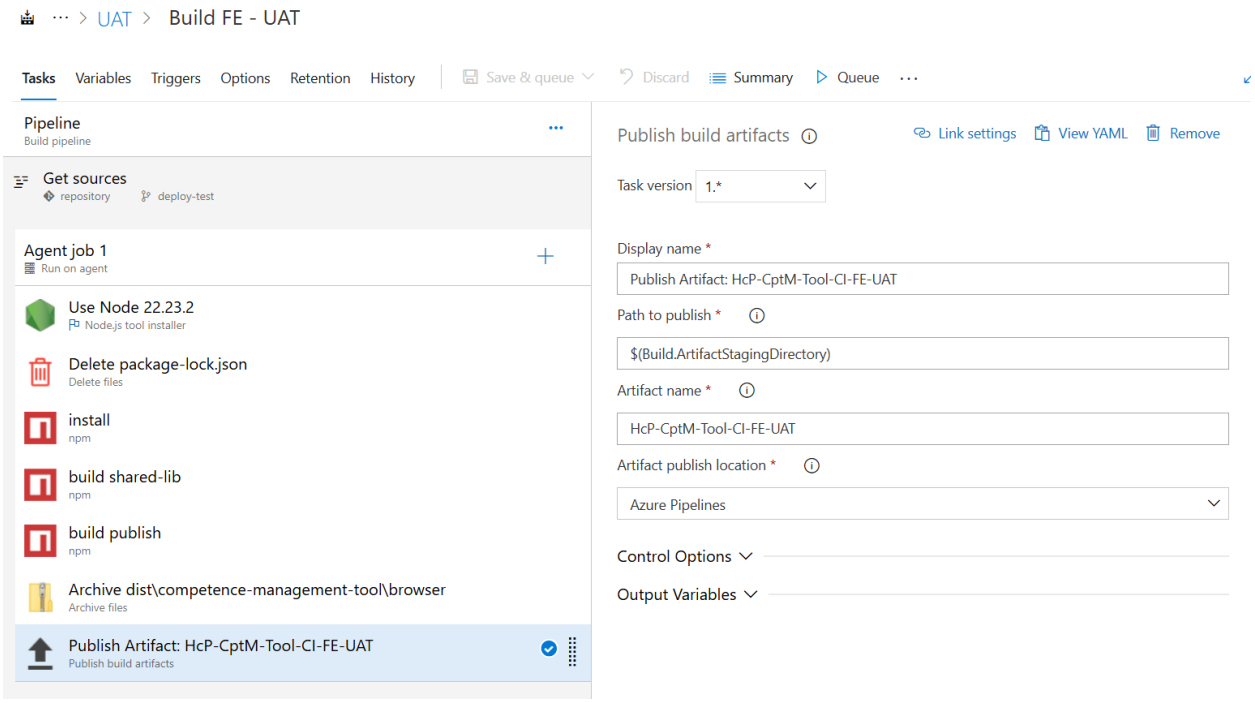
Hãy điều chỉnh lại tên folder đúng với tên dự án.

Step 10. Tại Agent job, click + button.

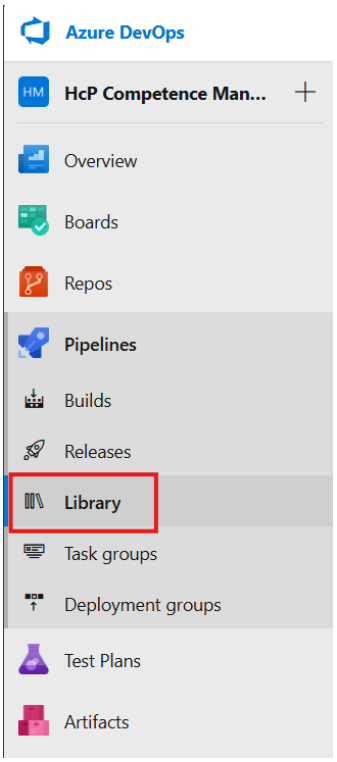
Tìm và add task “Publish build artifacts”.



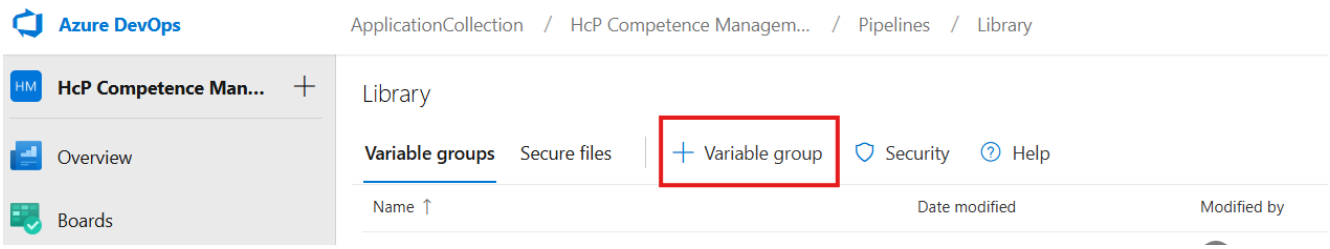
Sau đó, fill các thông tin như sau:



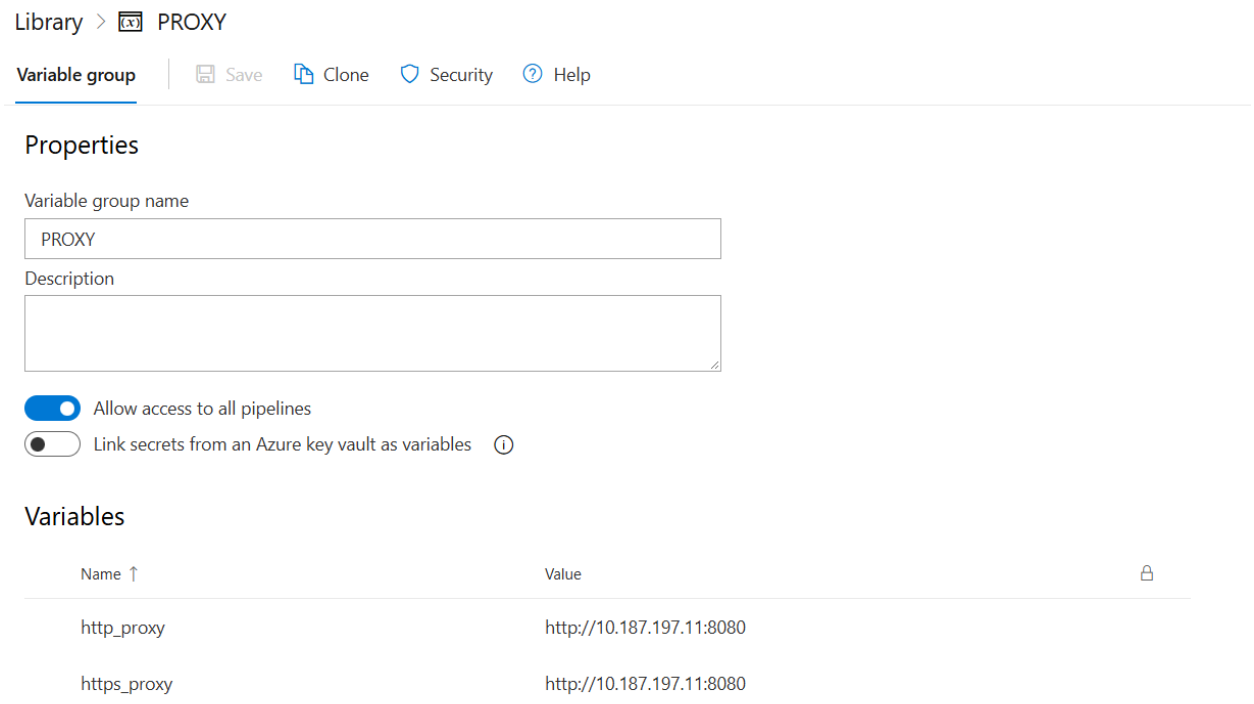
Step 11. Tại left menu, chọn Pipelines > Library.



Sau đó, thêm “Variable group”.

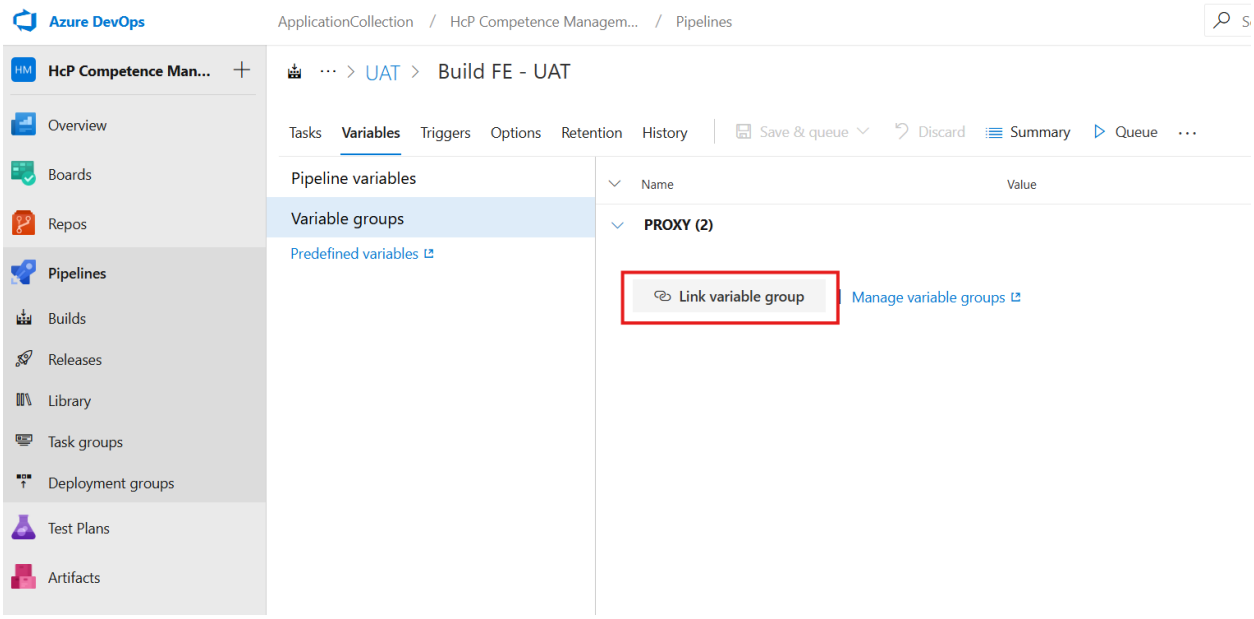


Fill các thông tin như sau:

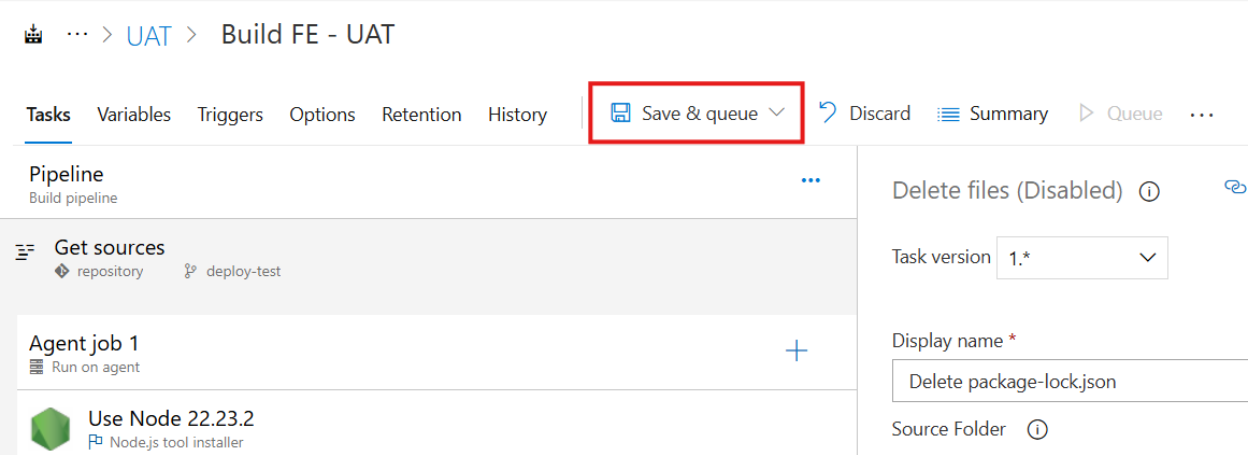


- http_proxy: http://10.187.197.11:8080
- https_proxy: http://10.187.197.11:8080

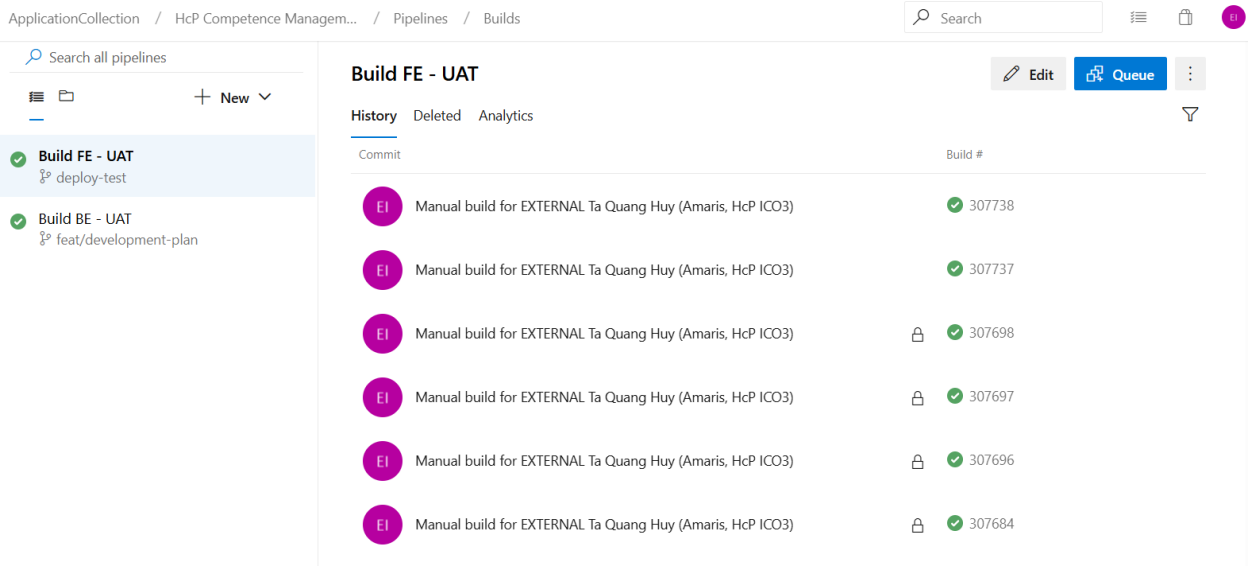
Step 12. Tại Build FE Pipeline, chọn tab “Variables” sau đó Link variable group PROXY vừa được tạo ở Step trên vào.



Step 13. Chọn “Save & queue”.



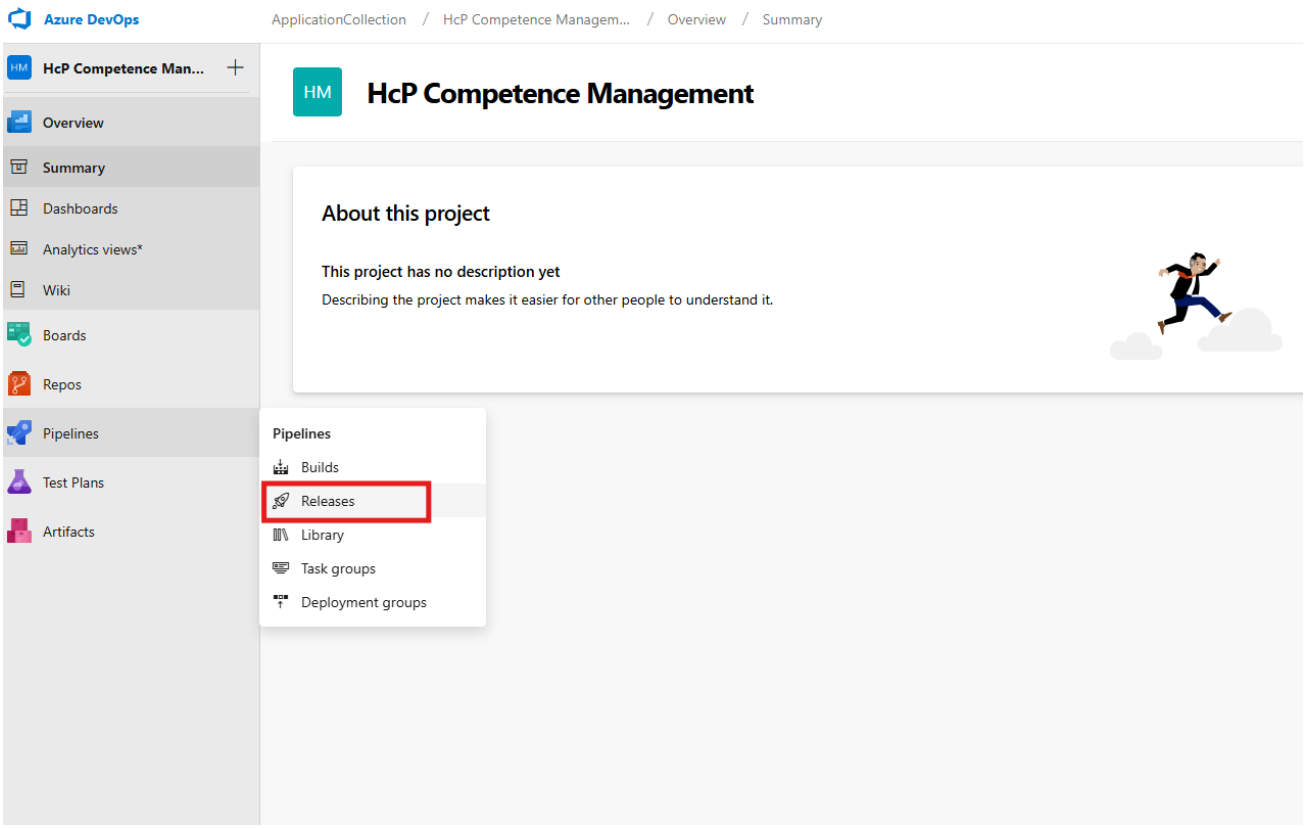
Theo dõi trạng thái build tại Pipelines > Builds



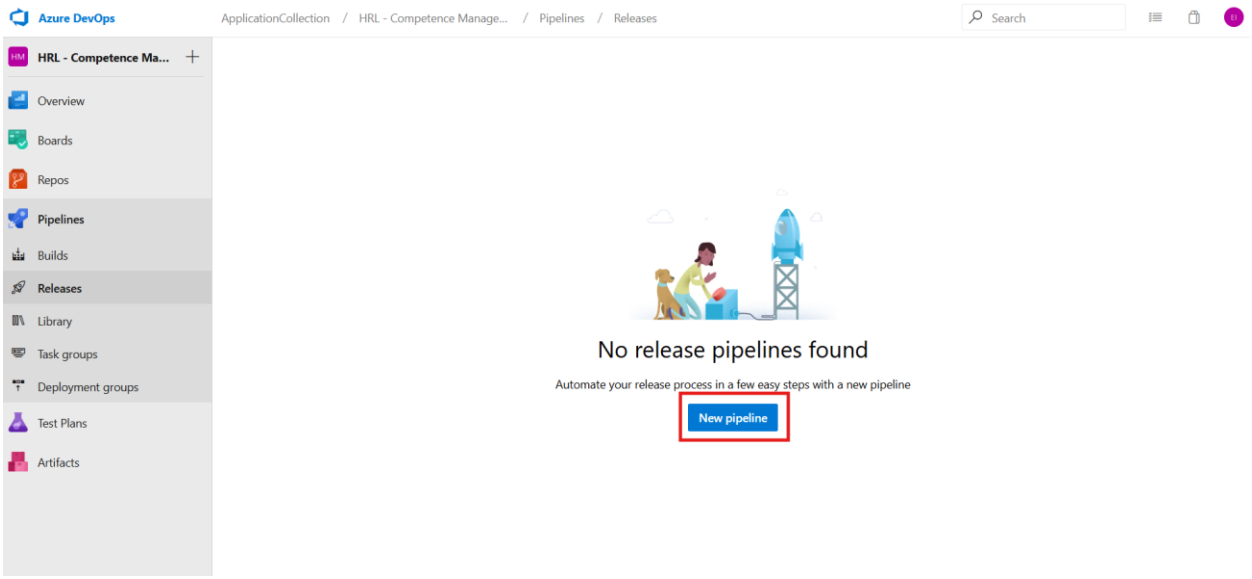
2. Set up Releases (CD)

2.1. Set up CD cho backend

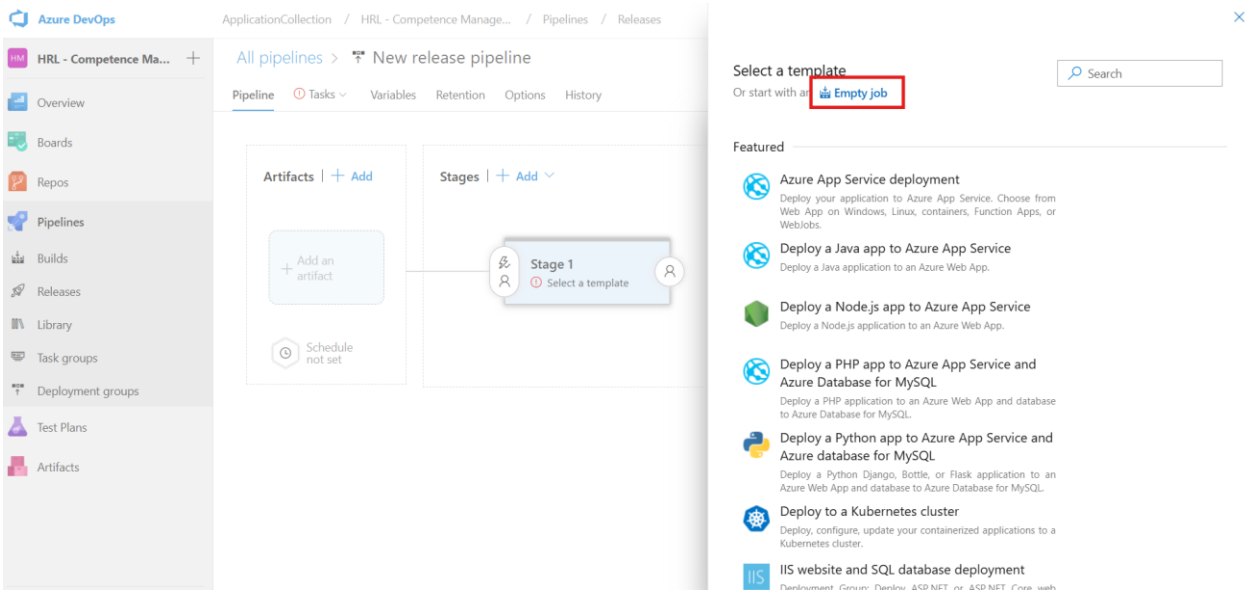
Step 1. Tại left menu, chọn Pipelines > Release



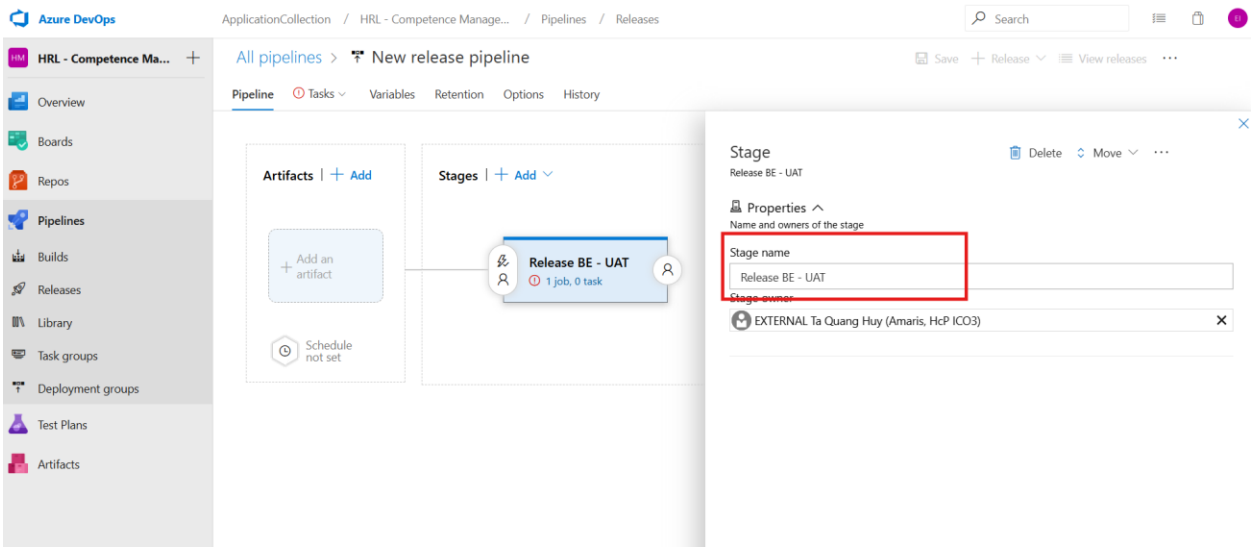
Step 2. Chọn “New pipeline”.



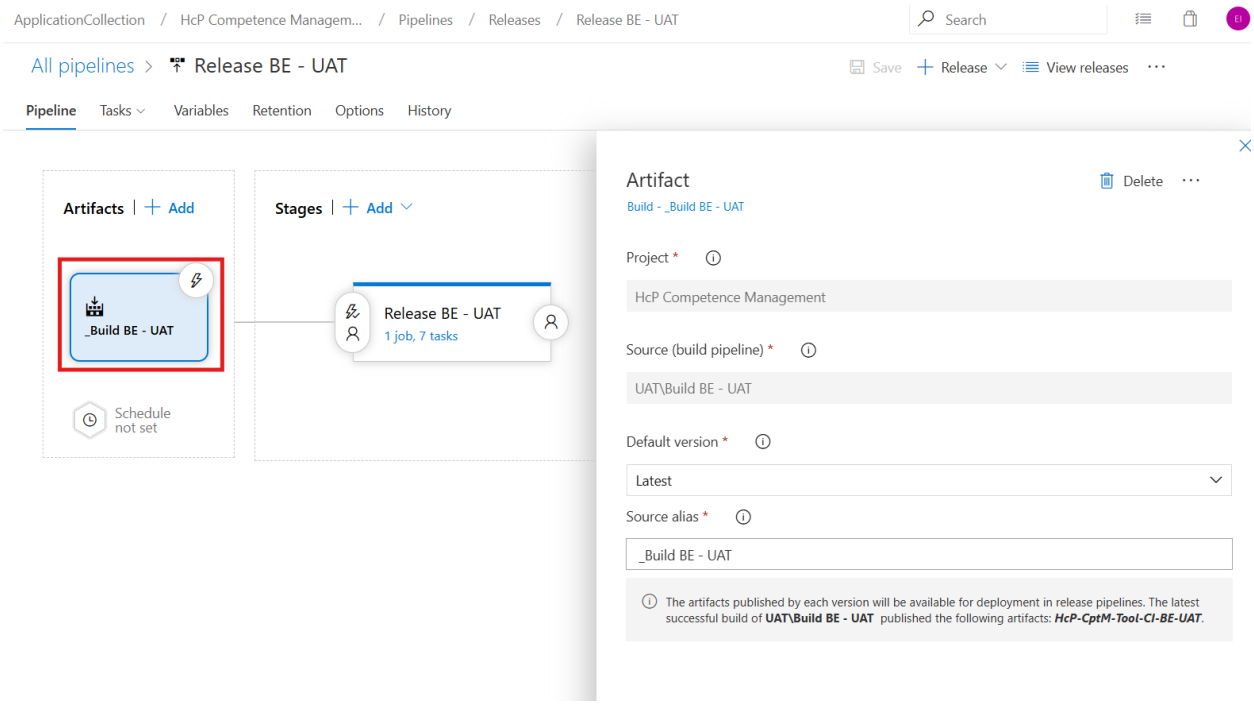
Step 3. Chọn “Empty job”.



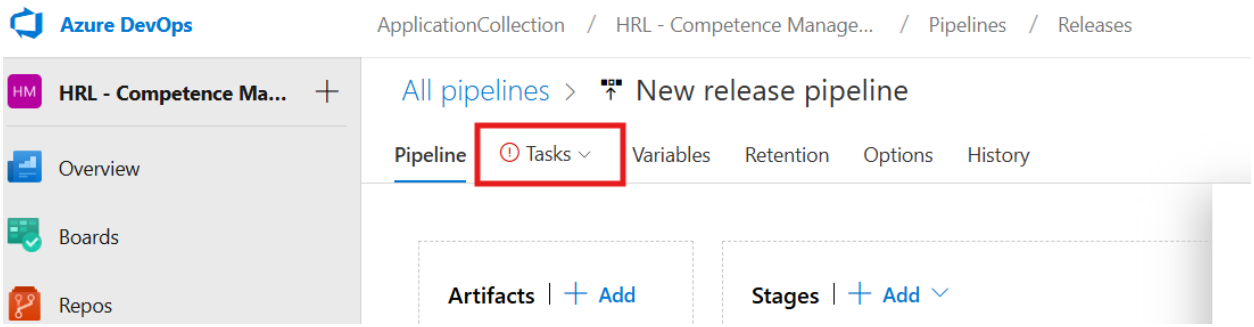
Step 4. Nhập Stage name theo format: “Release BE - <ENVRIONMENT>”.



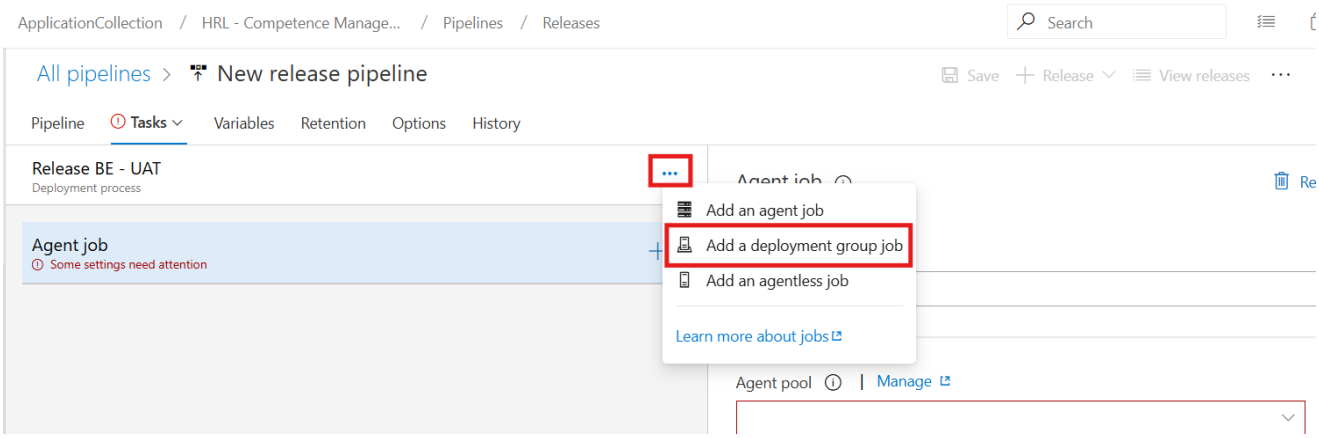
Step 5. Chọn Artifacts với Source là build pipeline đã set up và build thành công ở mục 1.1.



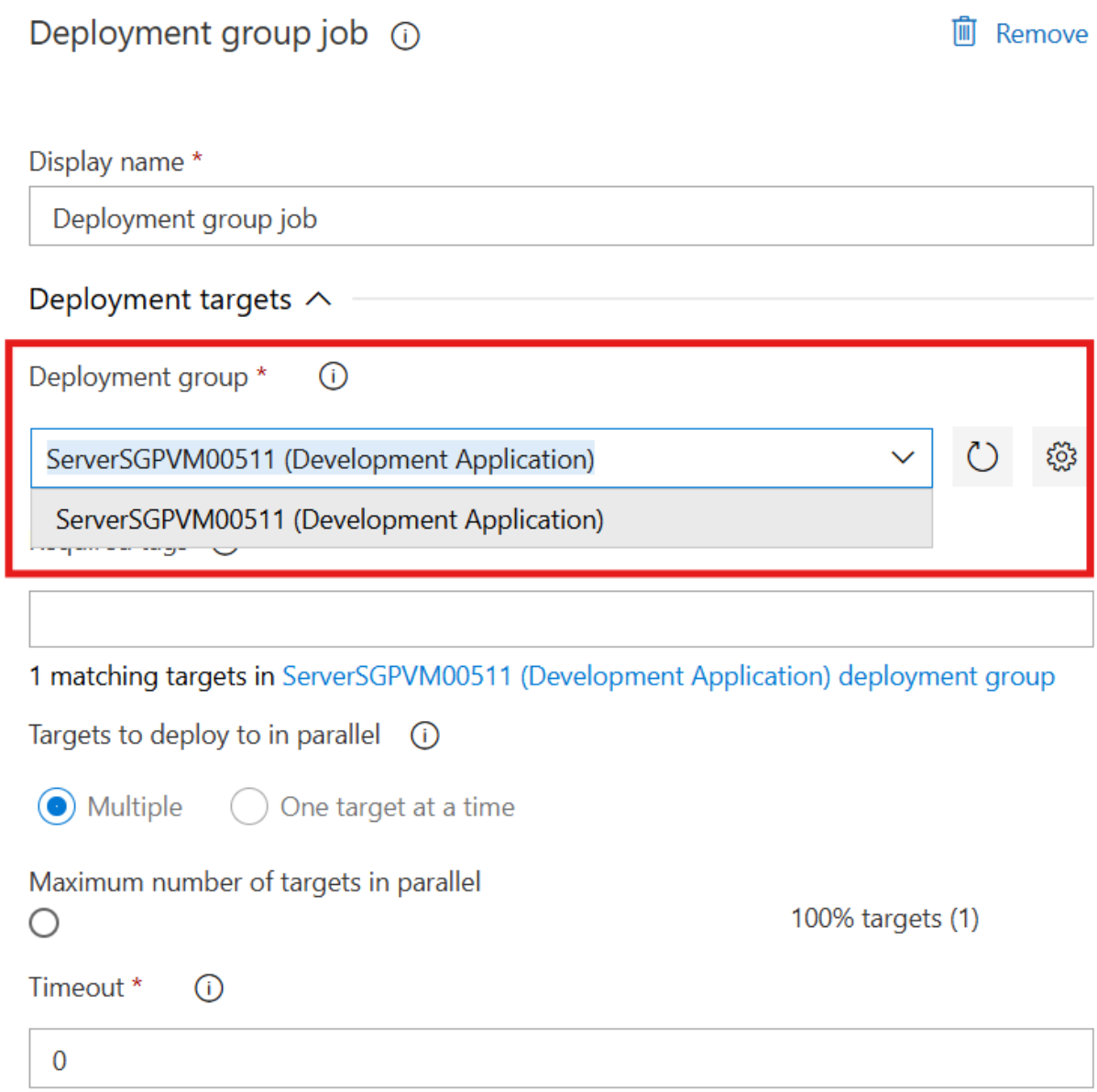
Sau đó click qua tab “Tasks”.



Step 6. Chọn “Add a deployment group job” để như hình sau:

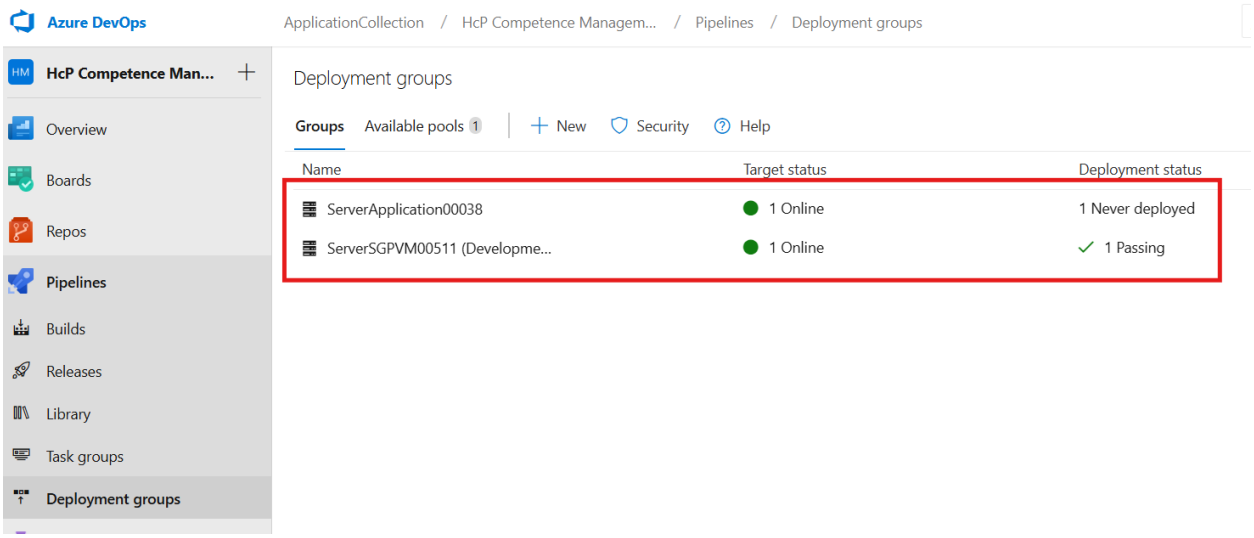


Step 7. Tại Deployment group job, chọn Deployment group (Server) cần deploy.

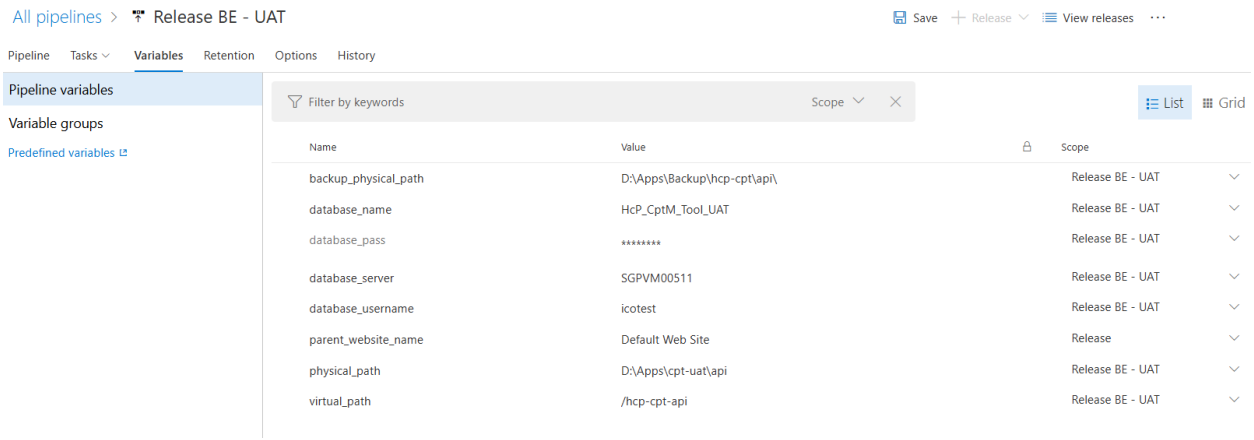


* Note:

- Chỉ người có permission mới chọn được Deployment group
- Deployment group cần được assign cho Project.

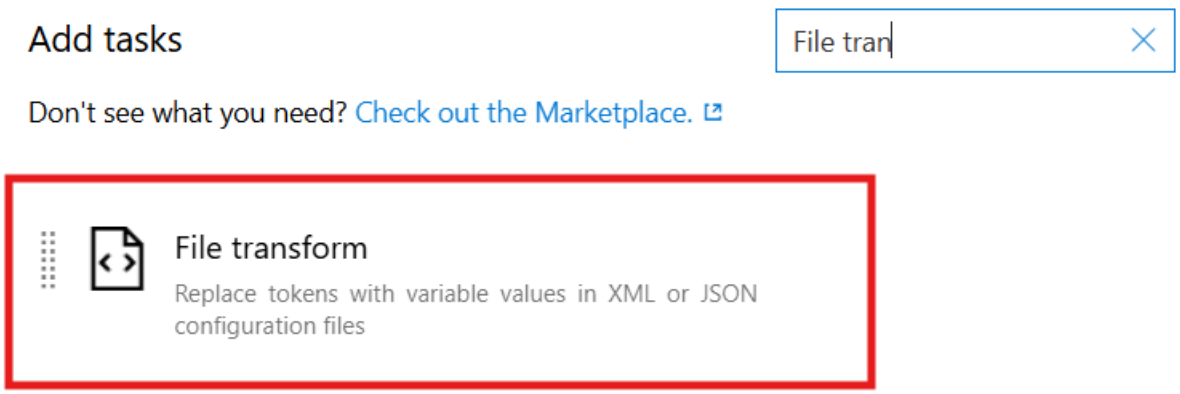


Step 8. Chọn tab "Variables" và config các variables cho quá trình release:



Step 9. Tại Deployment group job, click + button.

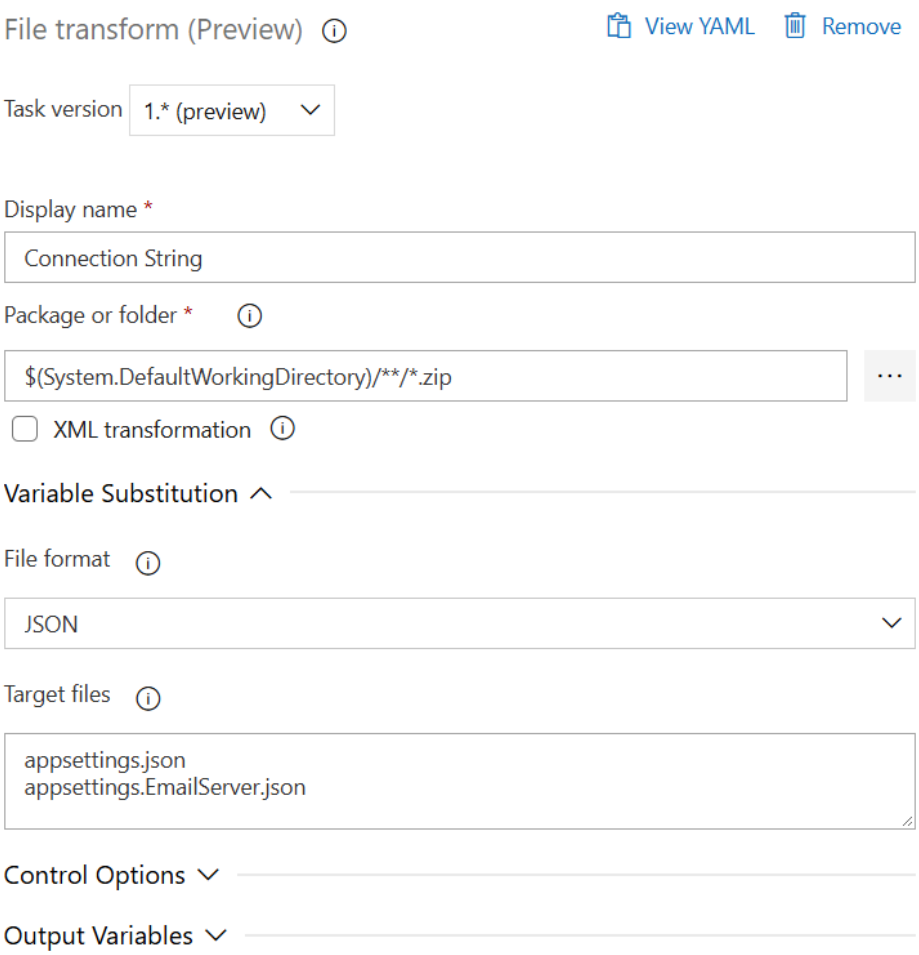
Tìm và add task "File transform".



Sau đó, fill các thông tin như sau:

Với:

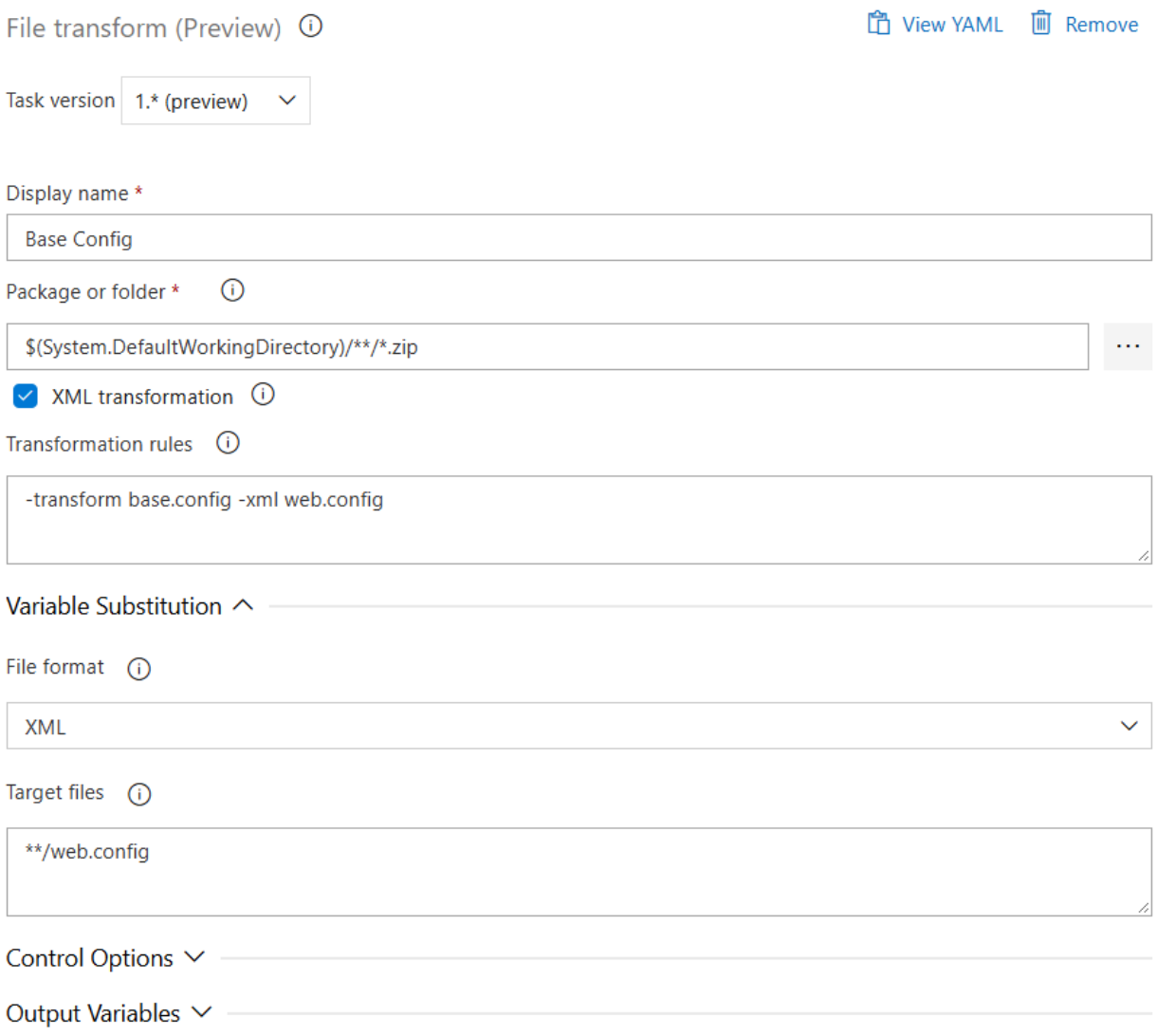
- Target files là các file appsettings trong project.



Step 10. Tại Deployment group job, click + button.

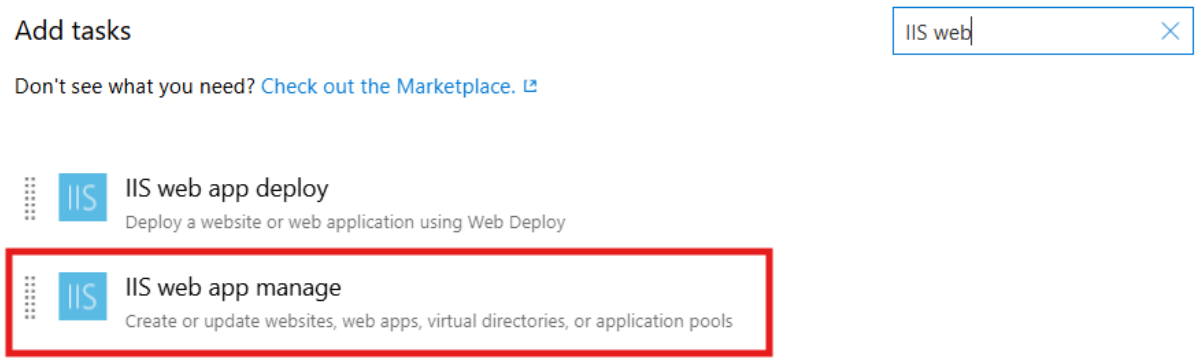
Tiếp tục, tìm và add task “File transform”.

Sau đó, fill các thông tin như sau:

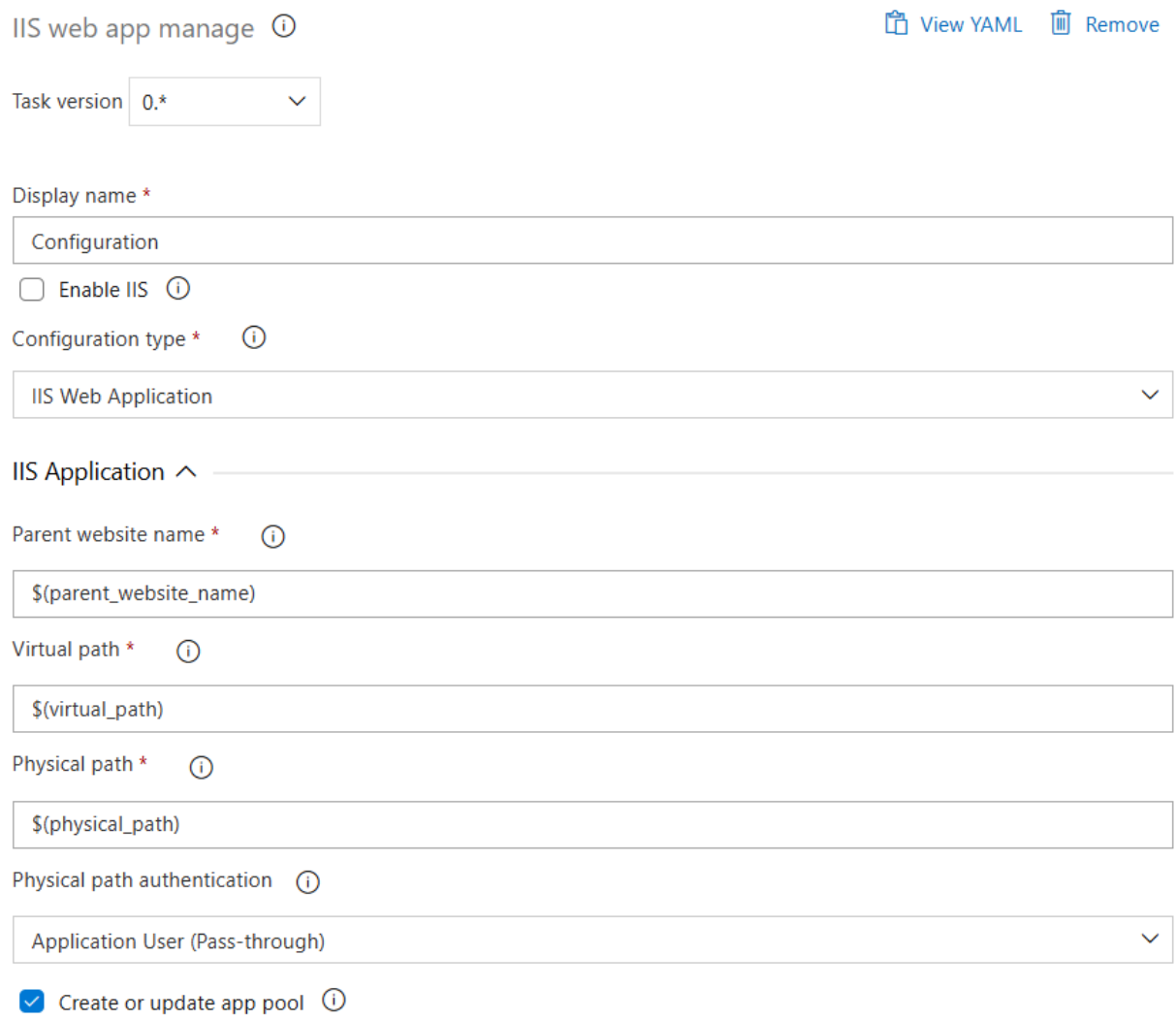


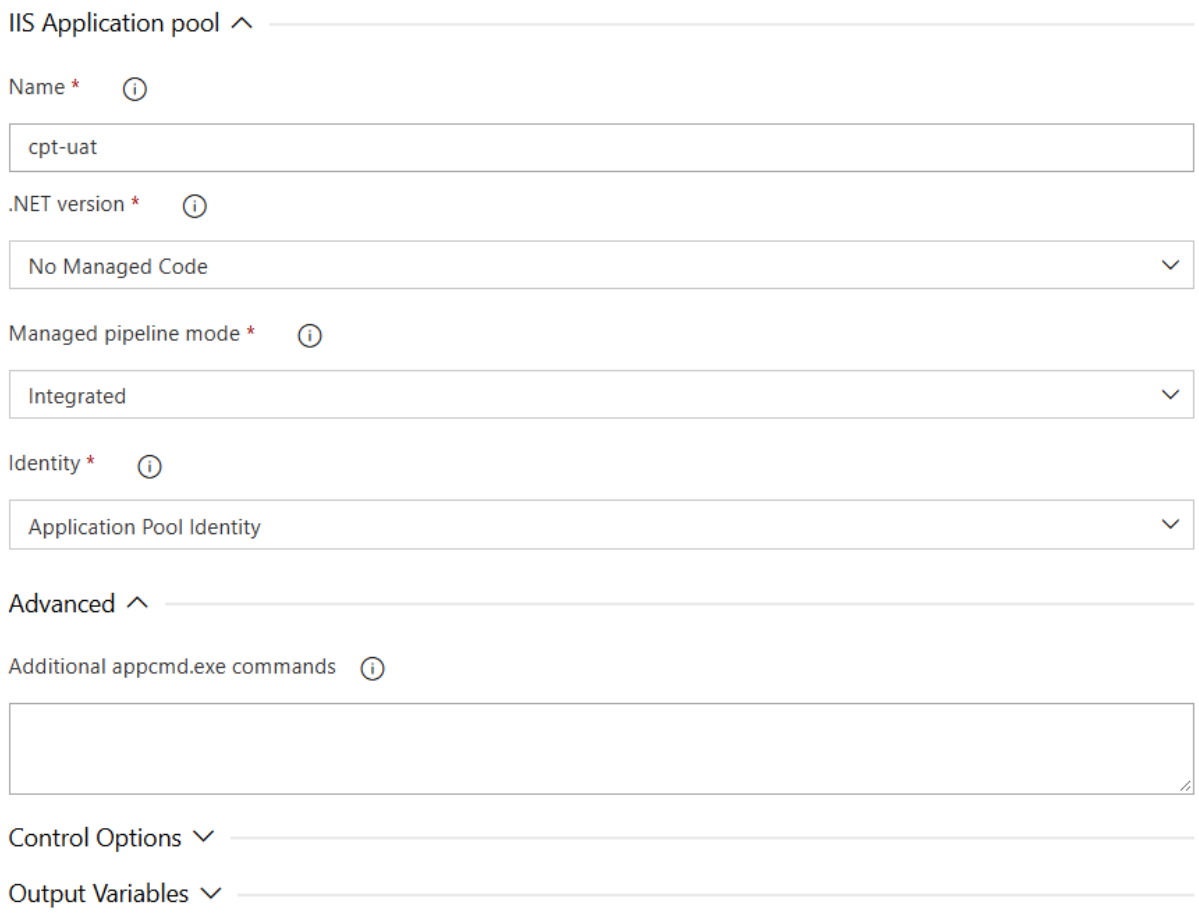
Step 11. Tại Deployment group job, click + button.

Tìm và add task "IIS web app manage".



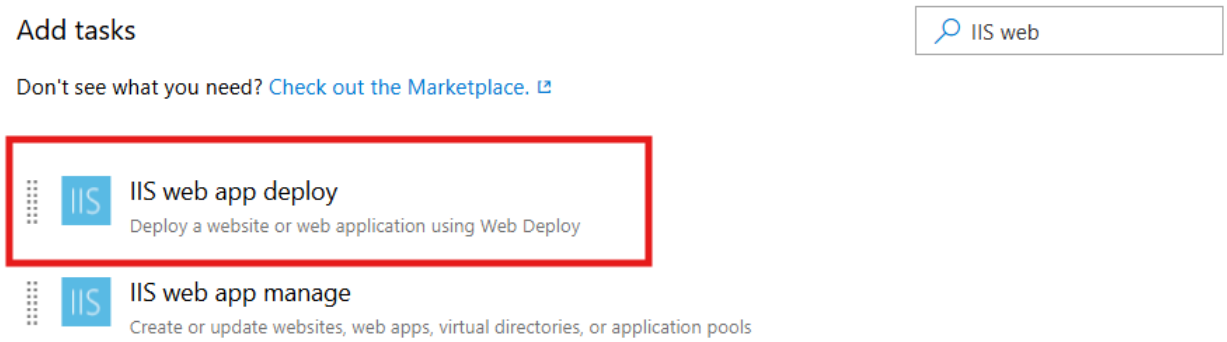
Sau đó, fill các thông tin như sau:



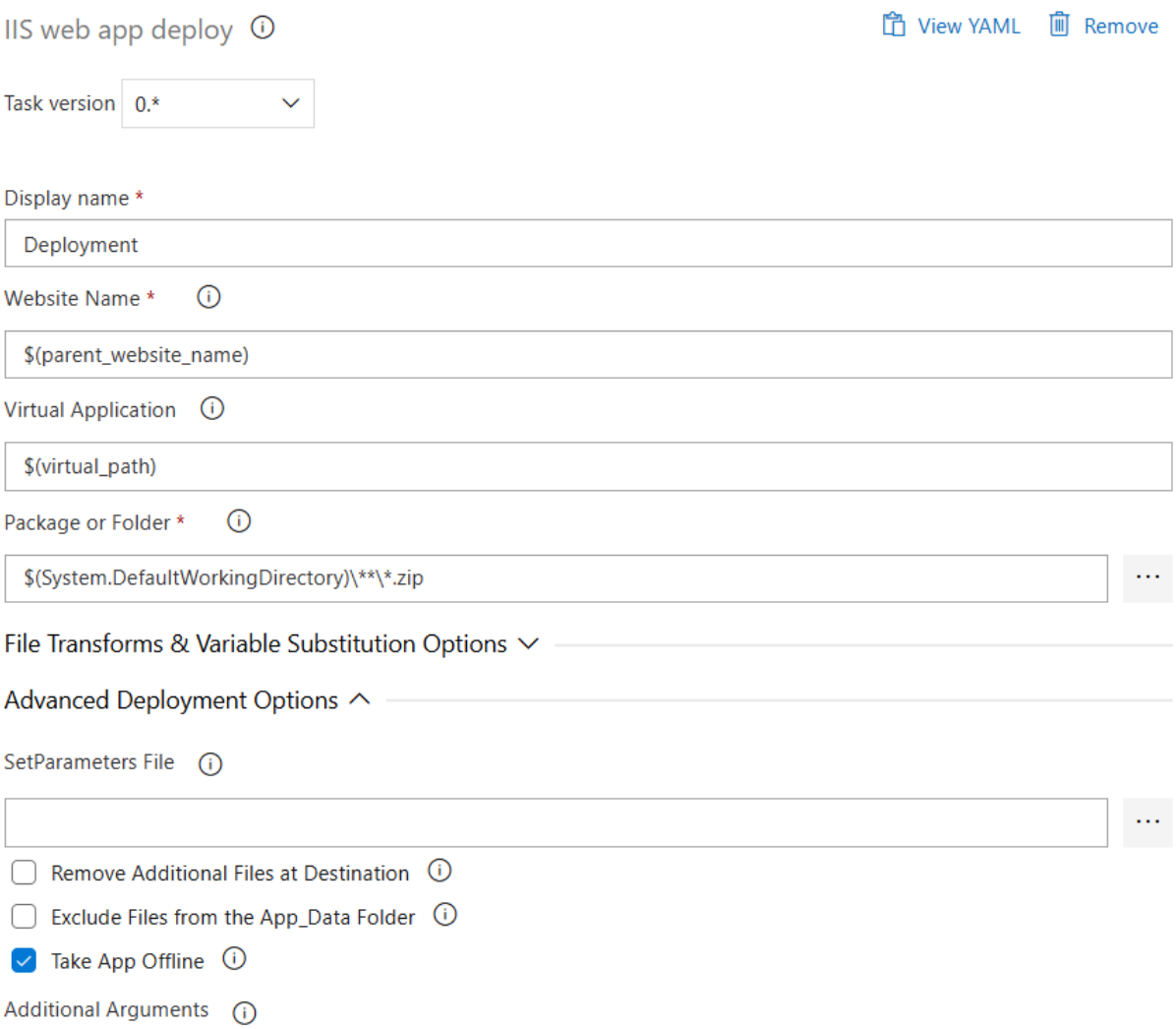


Step 11. Tại Deployment group job, click + button.

Tìm và add task “IIS web app deploy”.



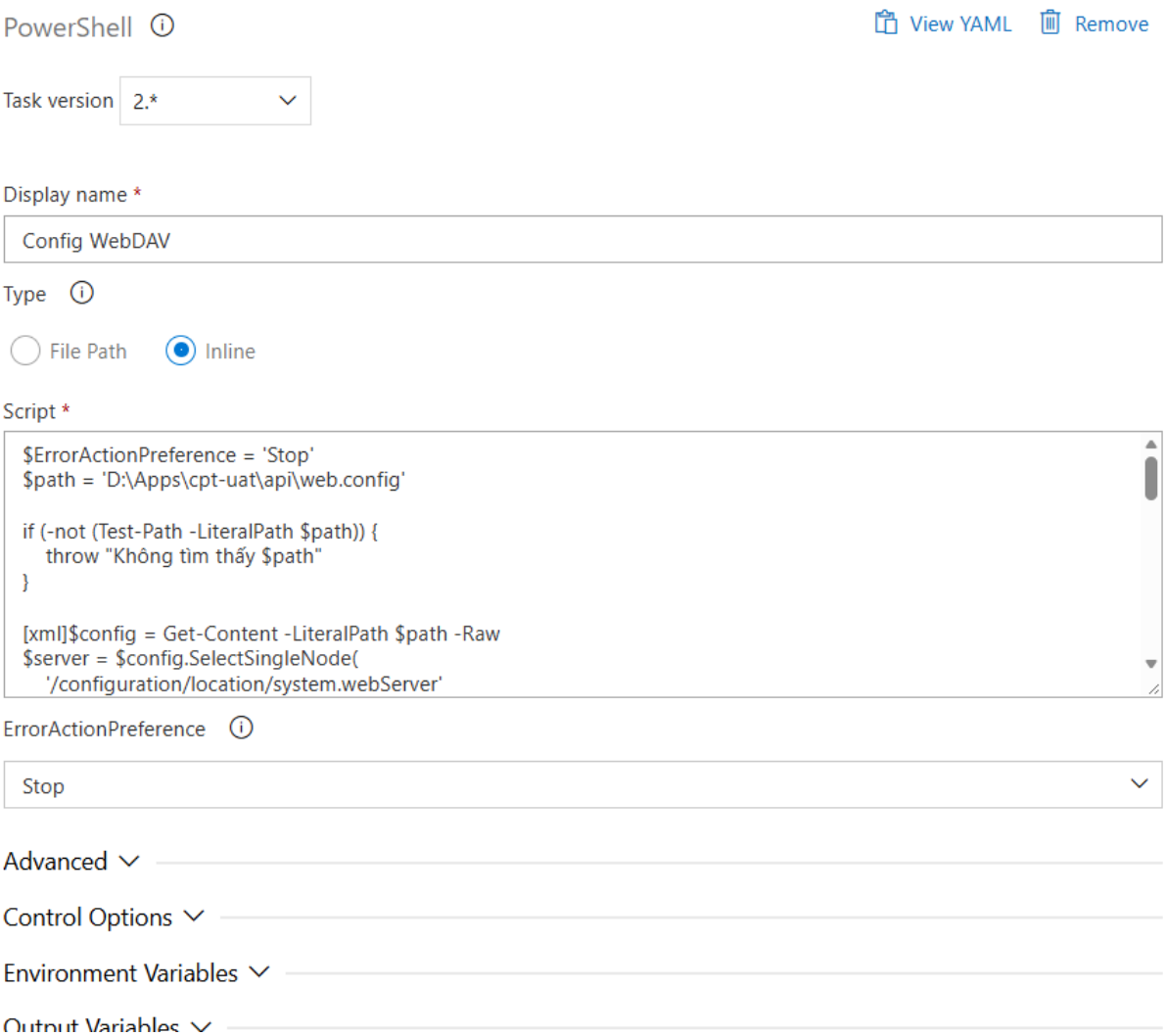
Sau đó, fill các thông tin như sau:



Step 13. Tại Deployment group job, click + button.

Tìm và add task “PowerShell”.

Sau đó, fill các thông tin như sau:



Trong đó, script để config WebDAV như sau:

```
$ErrorActionPreference = 'Stop'
$path = 'D:\Apps\cpt-uat\api\web.config'

if (-not (Test-Path -LiteralPath $path)) {
   throw "Không tìm thấy $path"
}

[xml]$config = Get-Content -LiteralPath $path -Raw
$server = $config.SelectSingleNode(
   '/configuration/location/system.webServer'
)

if ($null -eq $server) {
   throw 'Không tìm thấy system.webServer trong web.config'
```

```
}

$handlers = $server.SelectSingleNode('handlers')
if ($null -eq $handlers) {
   throw 'Không tìm thấy handlers trong web.config'
}

$aspNetCore = $handlers.SelectSingleNode("add[@name='aspNetCore']")
if ($null -eq $aspNetCore -or $aspNetCore.GetAttribute('verb') -ne '*') {
   throw 'Handler aspNetCore chưa được cấu hình với verb="*"'
}

$modules = $server.SelectSingleNode('modules')
if ($null -eq $modules) {
   $modules = $config.CreateElement('modules')
   [void]$server.InsertBefore($modules, $handlers)
}

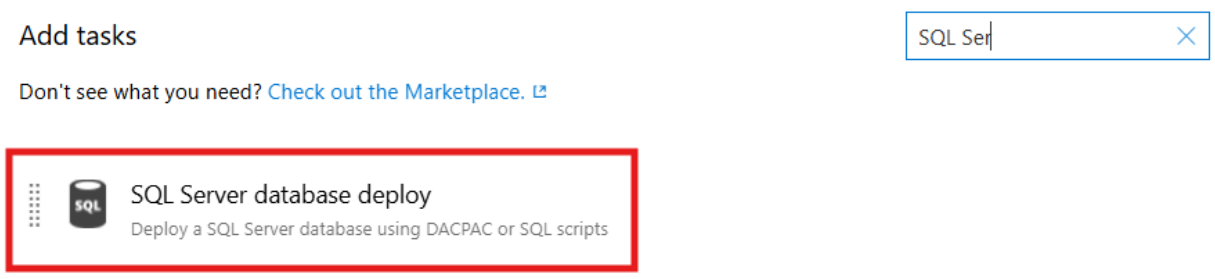
if ($null -eq $modules.SelectSingleNode("remove[@name='WebDAVModule']")) {
   $removeModule = $config.CreateElement('remove')
   $removeModule.SetAttribute('name', 'WebDAVModule')
   [void]$modules.AppendChild($removeModule)
}

if ($null -eq $handlers.SelectSingleNode("remove[@name='WebDAV']")) {
   $removeHandler = $config.CreateElement('remove')
   $removeHandler.SetAttribute('name', 'WebDAV')
   [void]$handlers.InsertBefore($removeHandler, $aspNetCore)
}

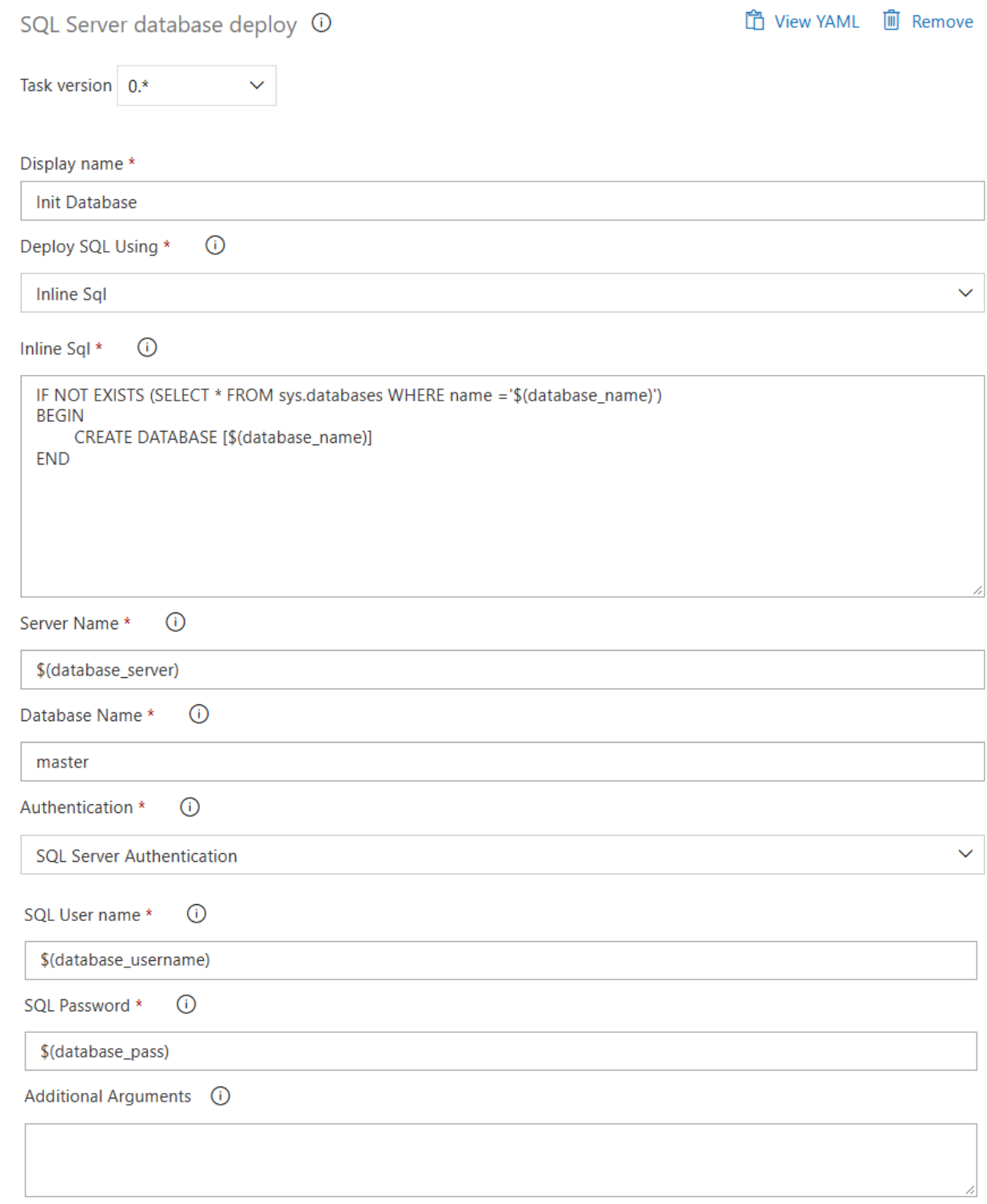
$config.Save($path)
Write-Host "Đã cấu hình loại WebDAV cho API: $path"
```

Step 14. Tại Deployment group job, click + button.

Tìm và add task “SQL Server database deploy”.



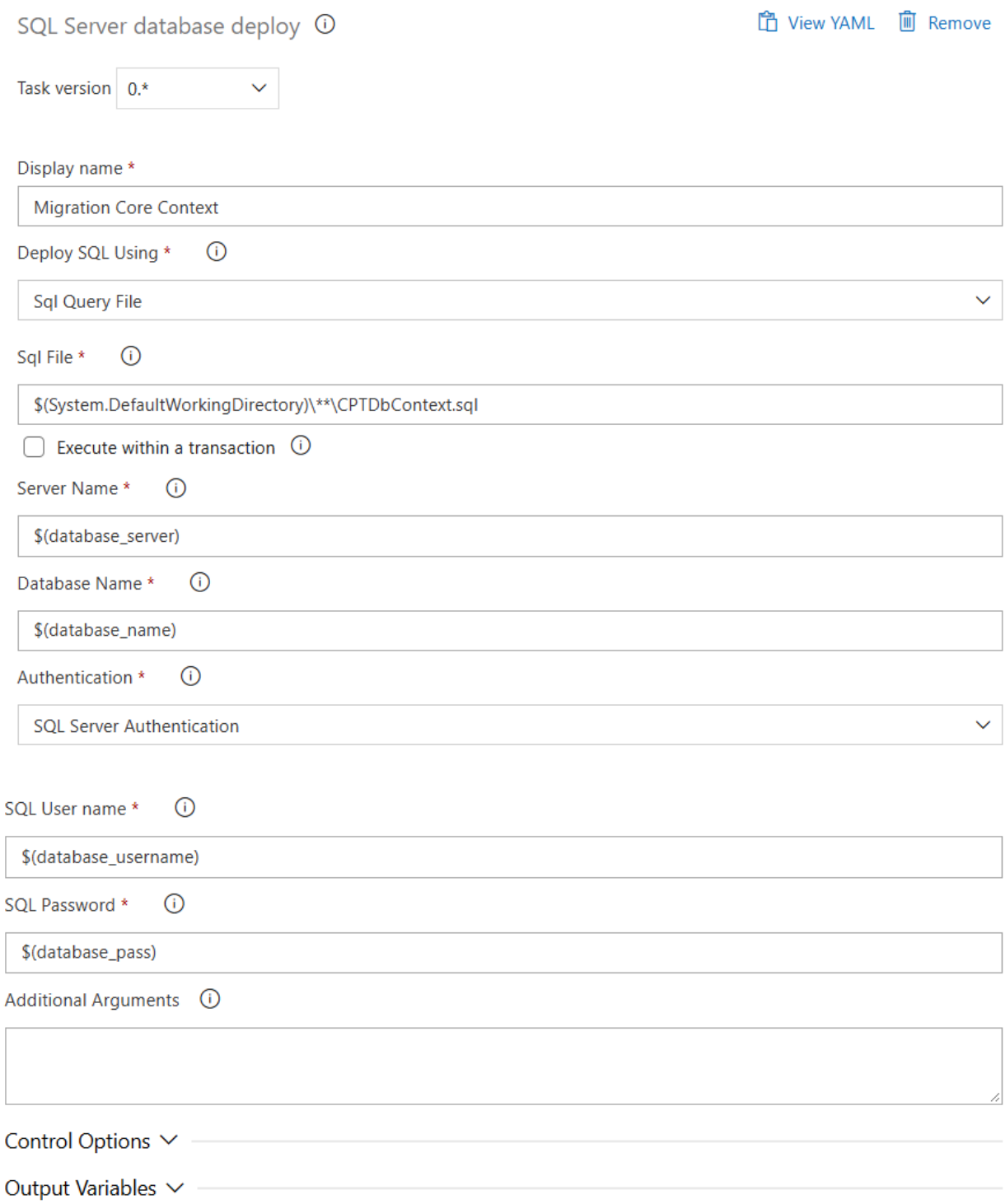
Sau đó, fill các thông tin như sau:



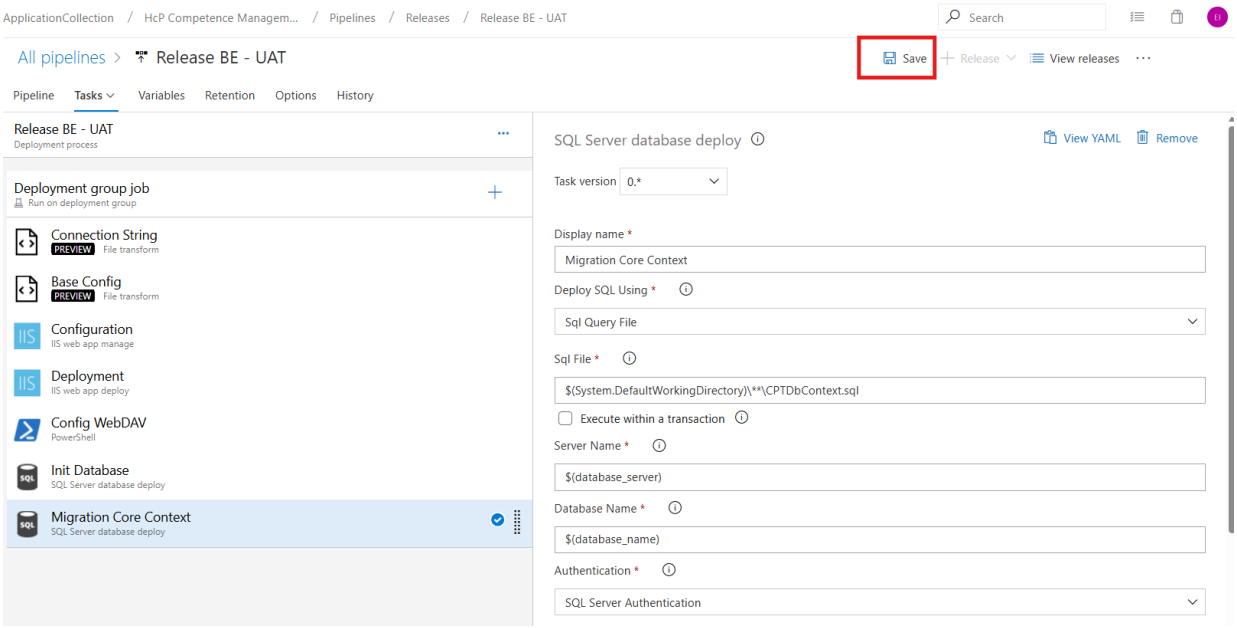
Step 15. Tại Deployment group job, click + button.

Tìm và add task “SQL Server database deploy”.

Sau đó, fill các thông tin như sau:



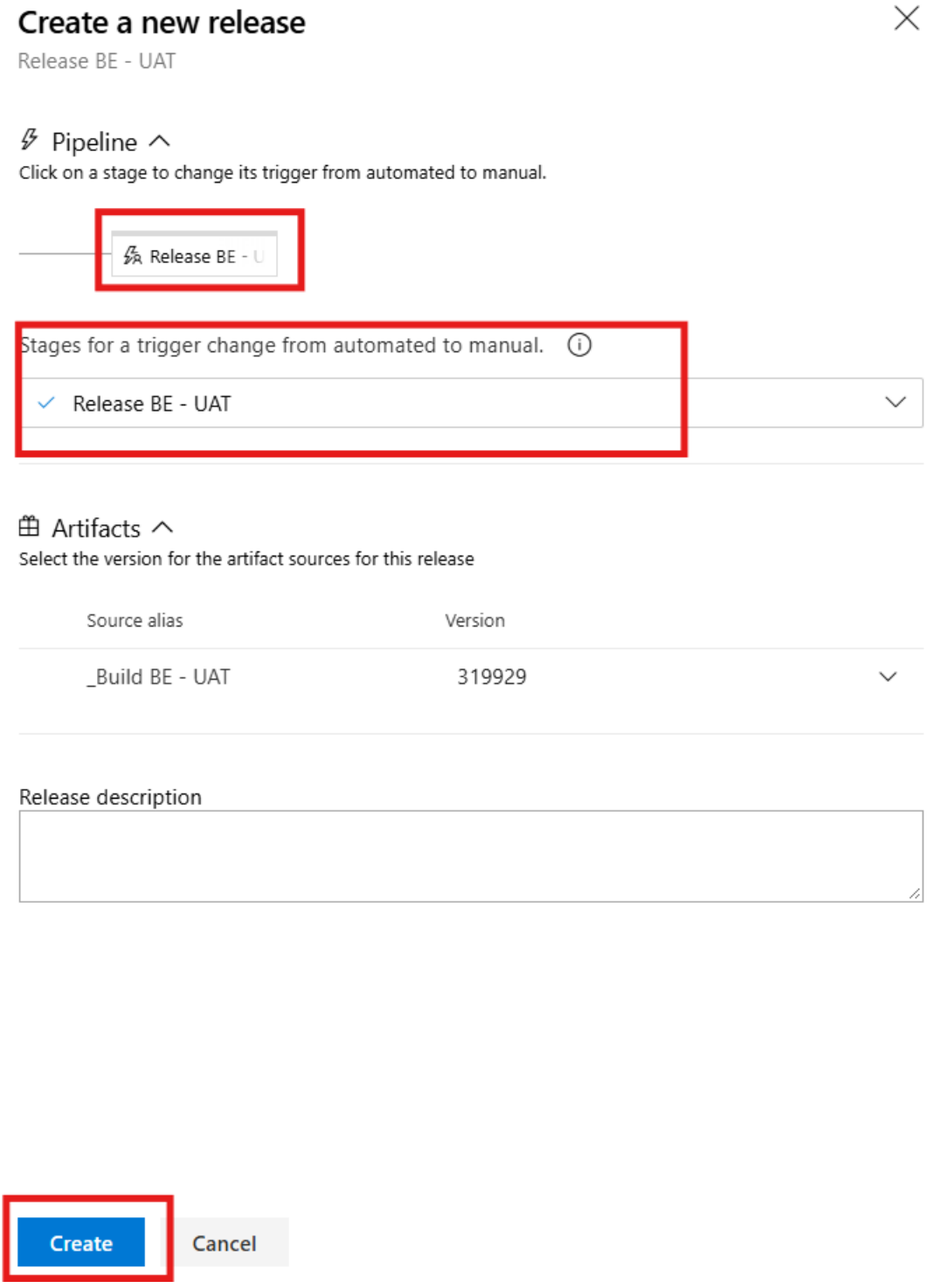
Step 16. Chọn Save để lưu Pipeline.



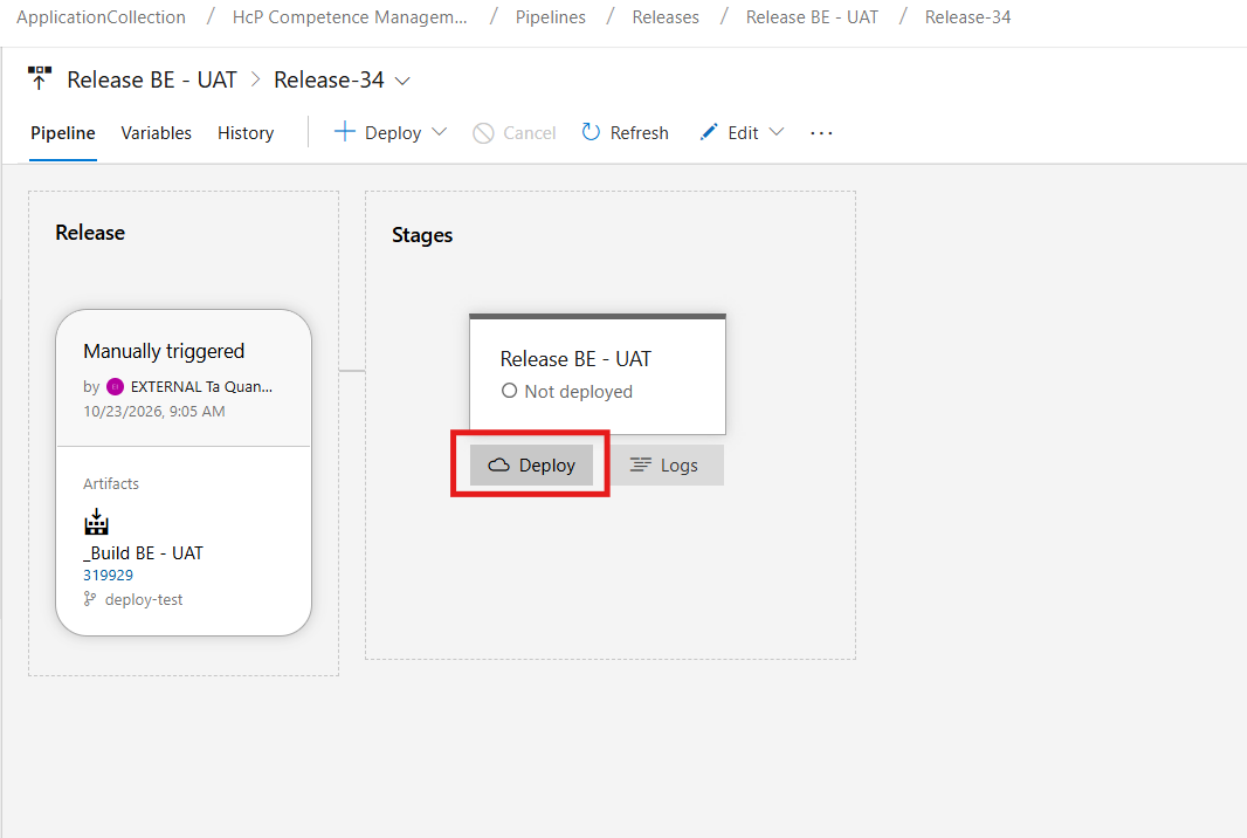
Step 17. Tại Pipeline tab, chọn “Create Release”.



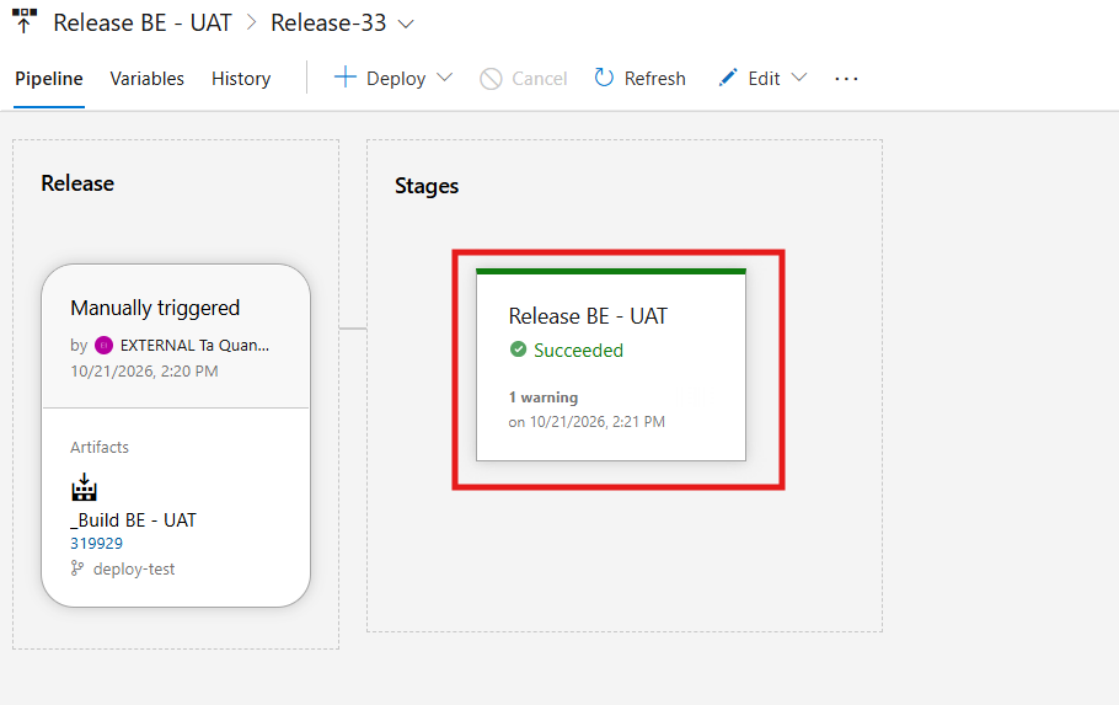
Seletc Pipeline và chọn “Create”.


Create a new release
Release BE - UAT
Pipeline
Click on a stage to change its trigger from automated to manual.
Release BE - U
Stages for a trigger change from automated to manual.
Release BE - UAT
Artifacts
Select the version for the artifact sources for this release
Source alias
Version
_Build BE - UAT
319929
Release description
Create
Cancel

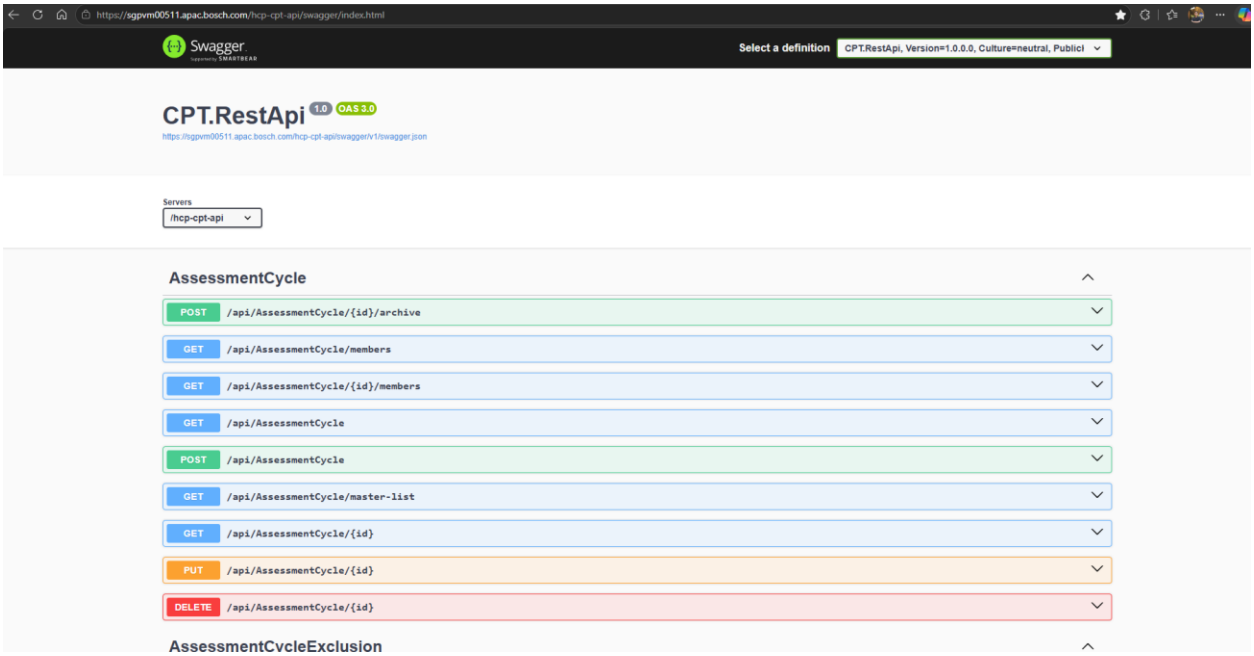
Step 17. Chọn “Deploy” để deploy release lên server.



Step 18. Sau khi deploy thành công Pipeline hiển thị giao diện như sau:



Có thể access vô Swagger thông qua $ virtual_path đã config ở các bước trên để verify các API được deploy.



2.2. Set up CD cho FrontEnd

Step 1. Thực hiện tương tự step 1 đến step step 7 như mục “2.1. Set up CD cho backend” .

Step 2. Tại Deployment group job, click + button.

Tìm và add task “File transform”.

Sau đó fill các thông tin như sau:

Trong đó:

- Target files là các file config trong hệ thống.

Ví dụ:

apps\competence-management-tool\public\configs\api-resource.json

apps > competence-management-tool > public > configs > {} api-resource.json

```
[
  {
    "name": "BASE_API",
    "urlPattern": "https://localhost:7266",
    "audience": "hcp-competence-management-tool-confidential",
    "scope": "hcp-competence-management-tool"
  }
]
```

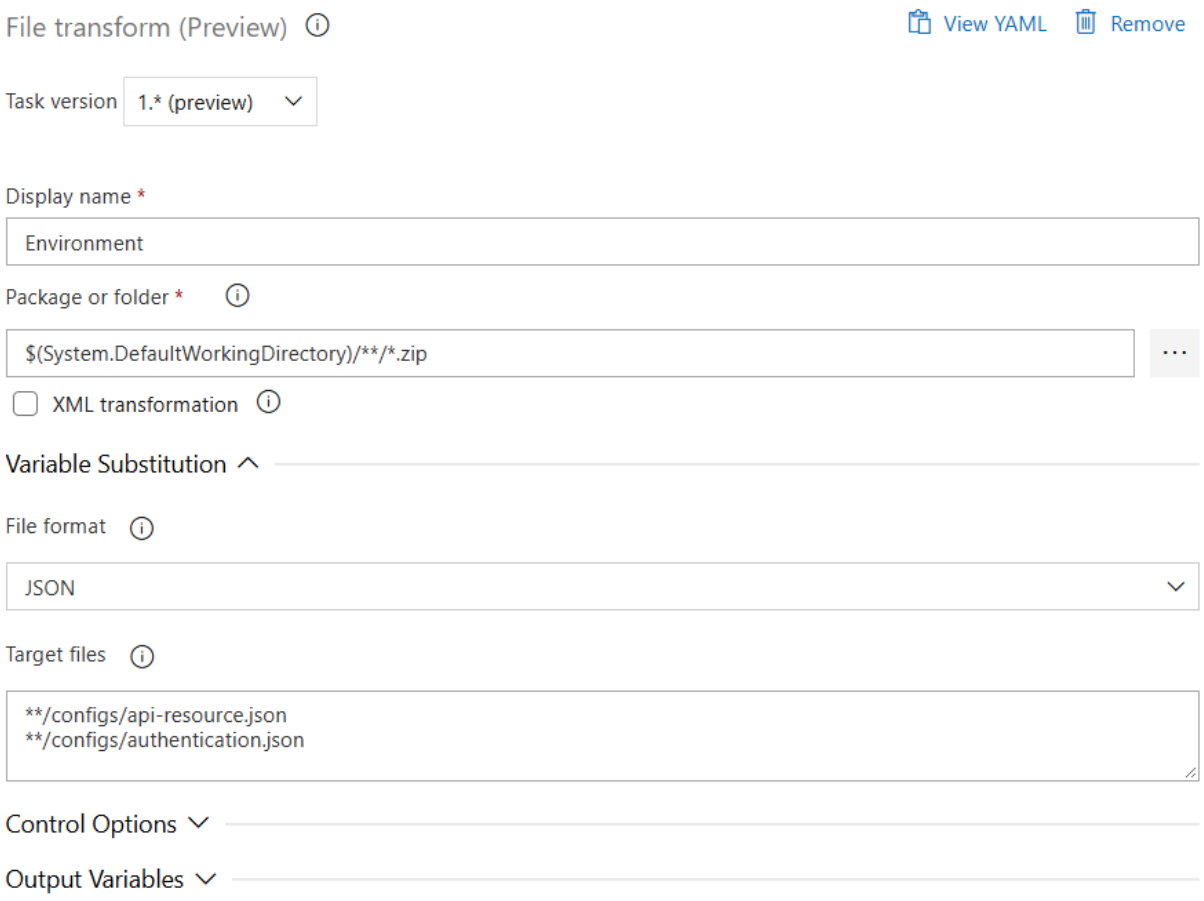
apps\competence-management-tool\public\configs\authentication.json

apps > competence-management-tool > public > configs > {} authentication.json > ...

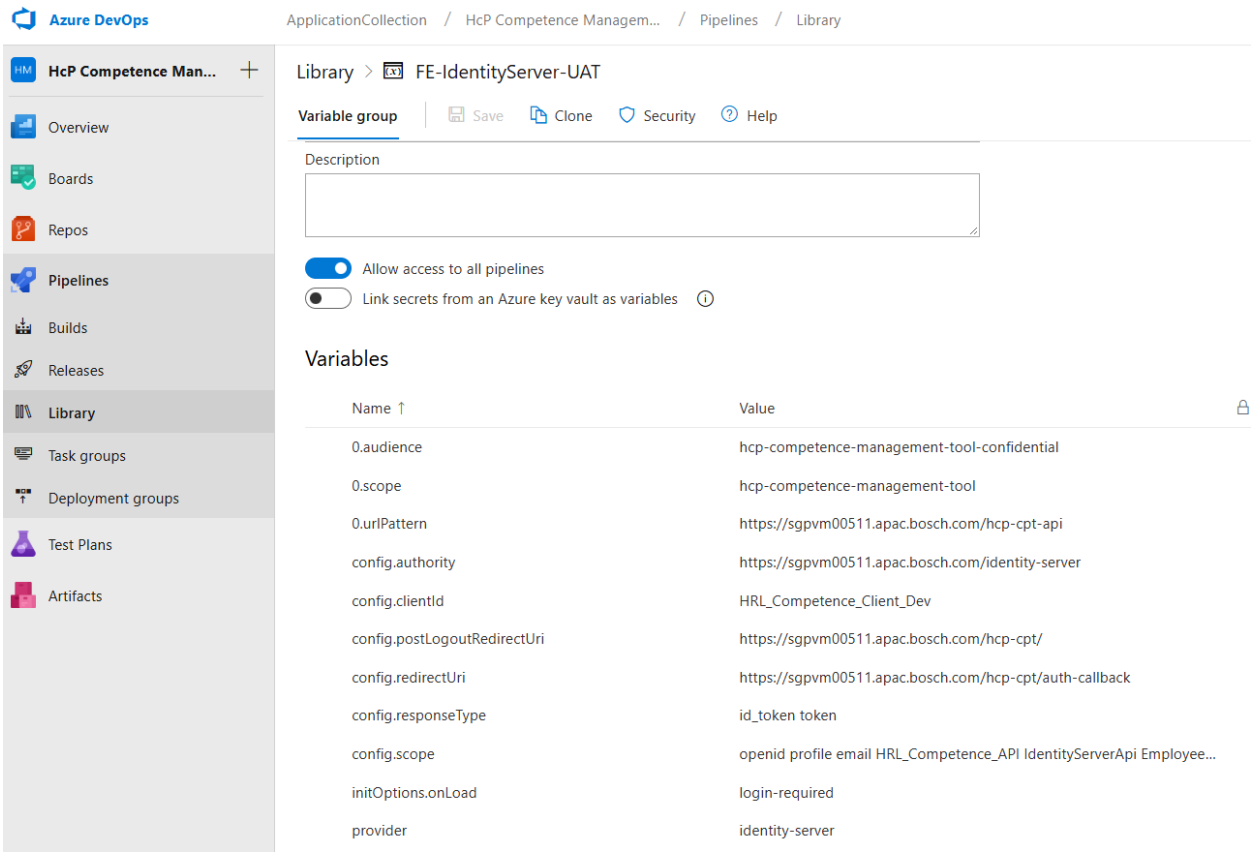
You, yesterday | 2 authors (You and one other)

```
{
  "provider": "identity-server",
  "config": {
    "authority": "https://sgpvm00511.apac.bosch.com/identity-server",
    "clientId": "HRL_Competence_Client_Dev",
    "redirectUri": "http://localhost:4200/auth-callback",
    "postLogoutRedirectUri": "http://localhost:4200/",
    "responseType": "id_token token",
    "scope": "openid profile email HRL_Competence_API IdentityServerApi Employee_API_Lv1",
    "loadUserInfo": false,
    "automaticSilentRenew": false,
    "monitorSession": false
  },
  "initOptions": {
    "onLoad": "login-required"
  }
}
```

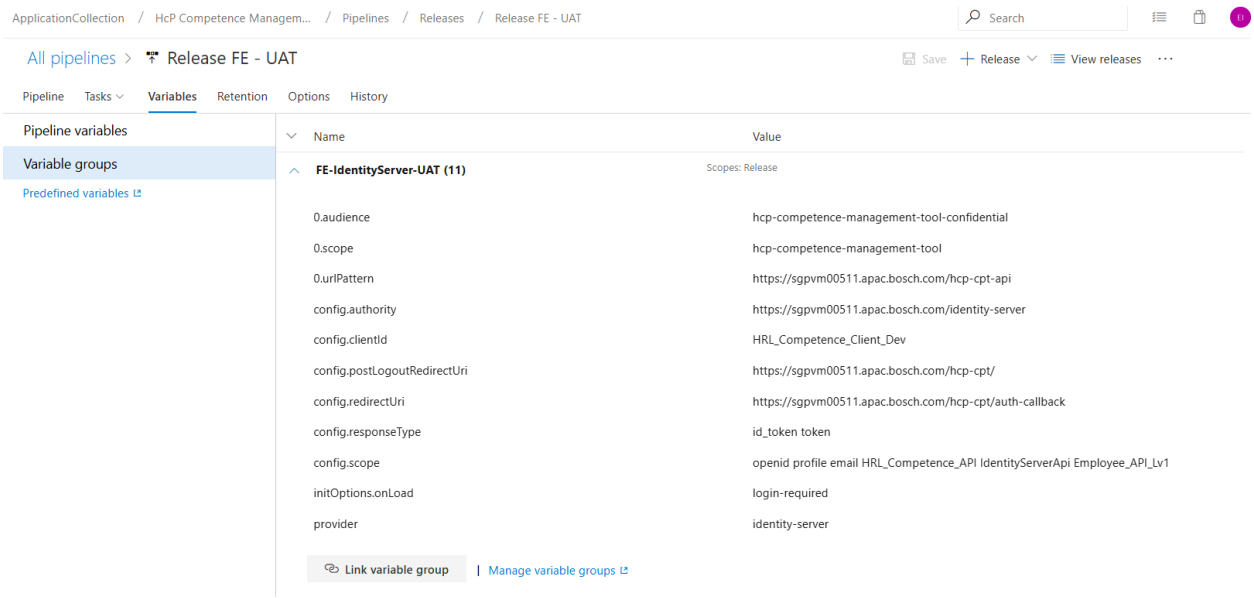
Thì config như sau:



Step 3. Tạo một Variable group để ghi đè các configs ở step trước khi cần thiết (tùy theo môi trường và authentication provider mà các configs có thể có khác nhau).

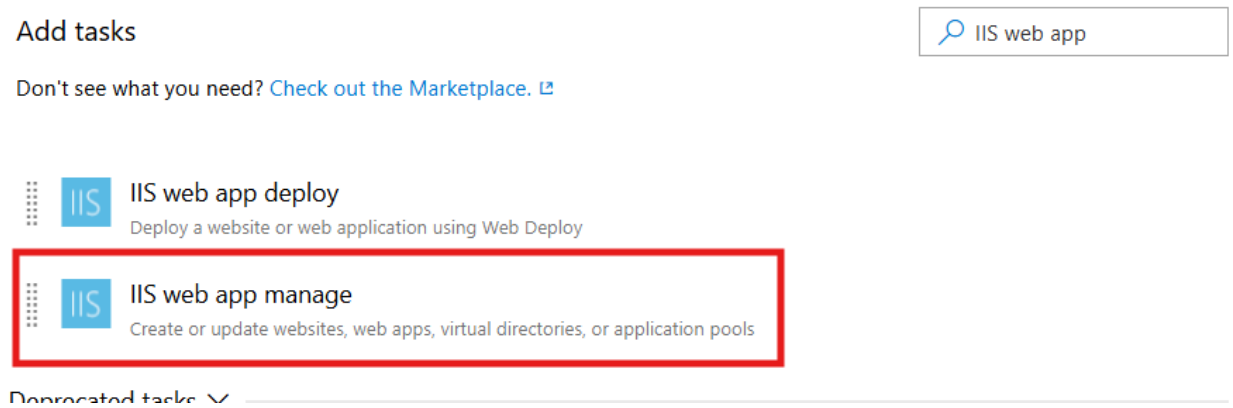


Sau đó link variable group vào Pipeline:

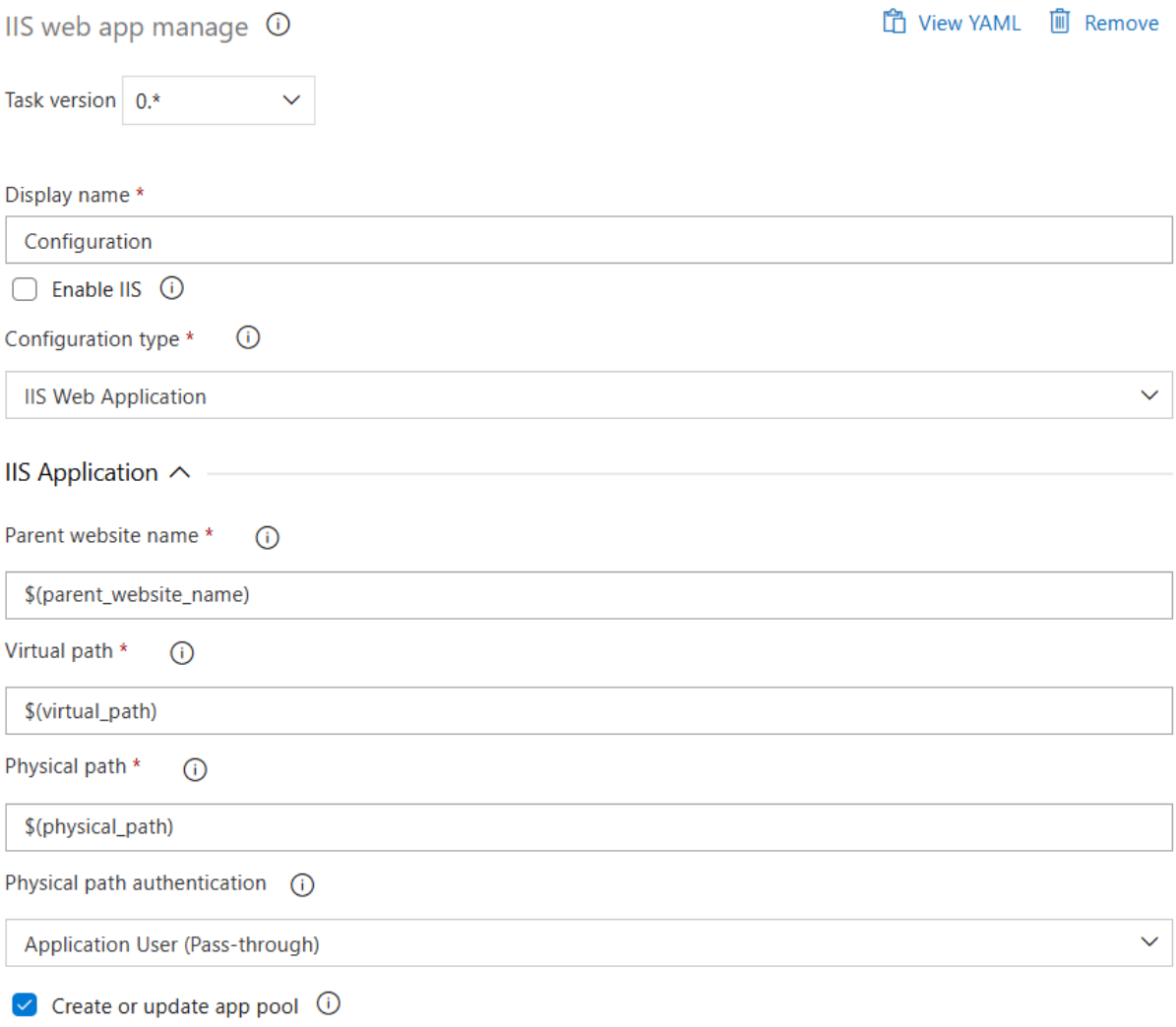


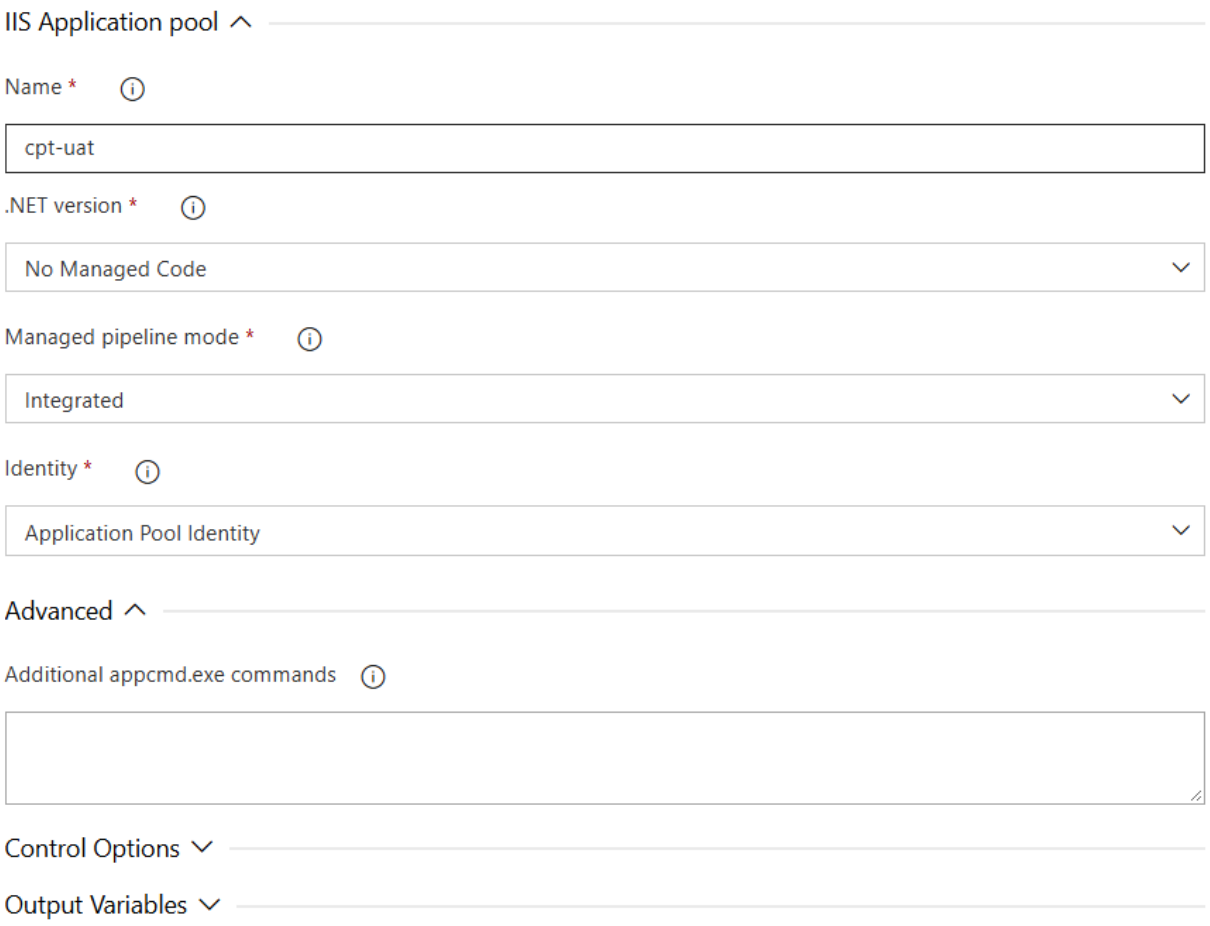
Step 4. Tại Deployment group job, click + button.

Tìm và add task “IIS web app manage”.



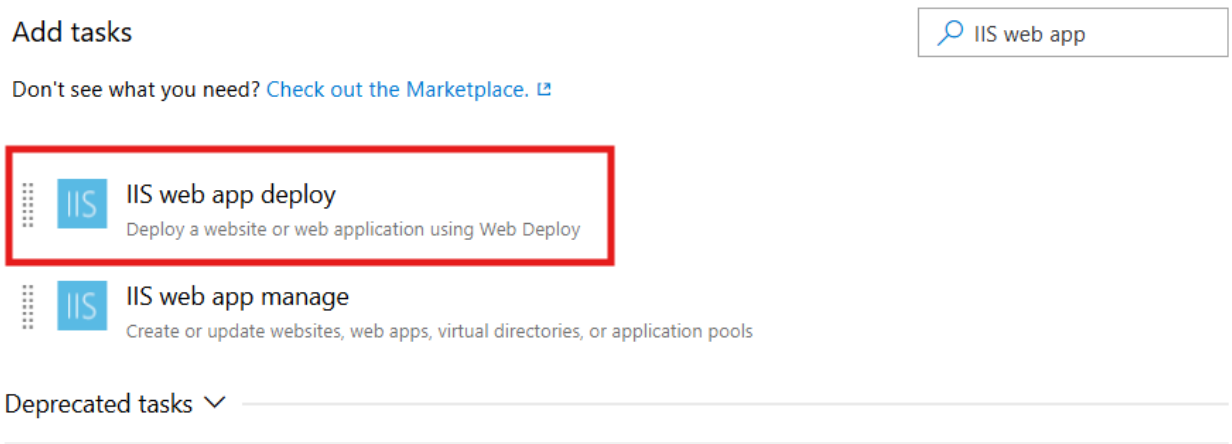
Sau đó fill các thông tin như sau:



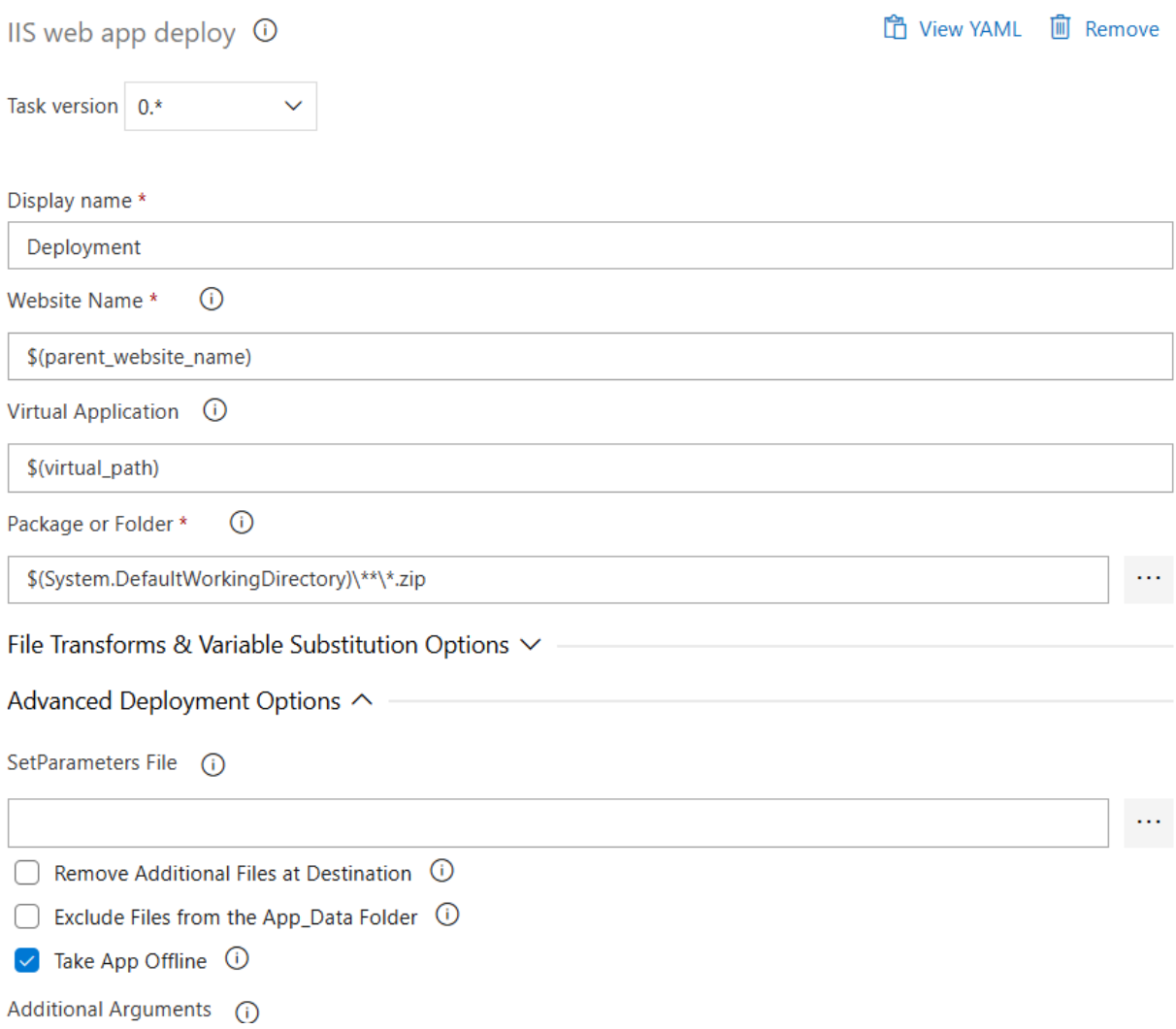


Step 5. Tại Deployment group job, click + button.

Tìm và add task “IIS web app deploy”.



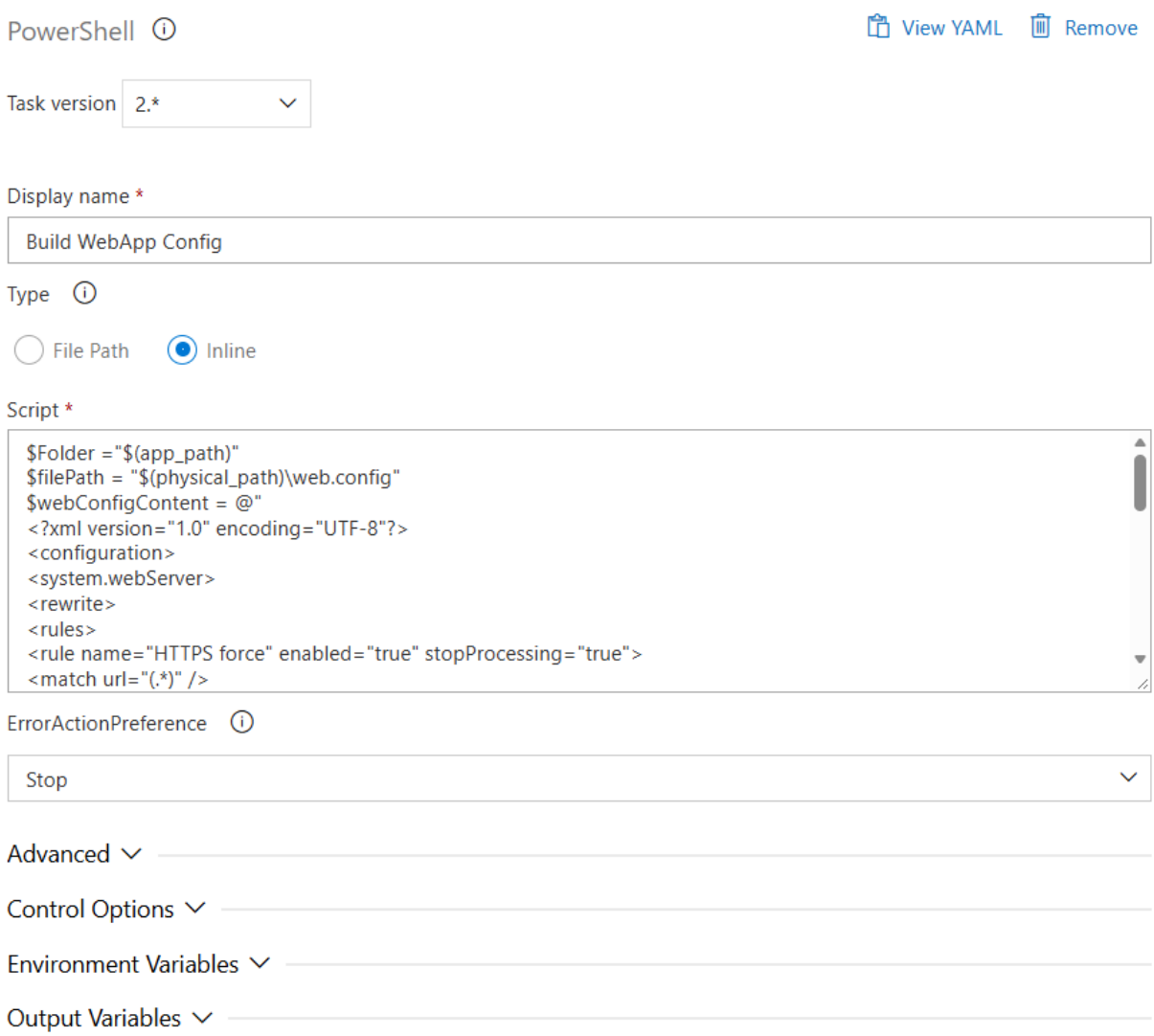
Sau đó fill các thông tin như sau:



Step 6. Tại Deployment group job, click + button.

Tìm và add task “PowerShell”.

Sau đó fill các thông tin như sau:



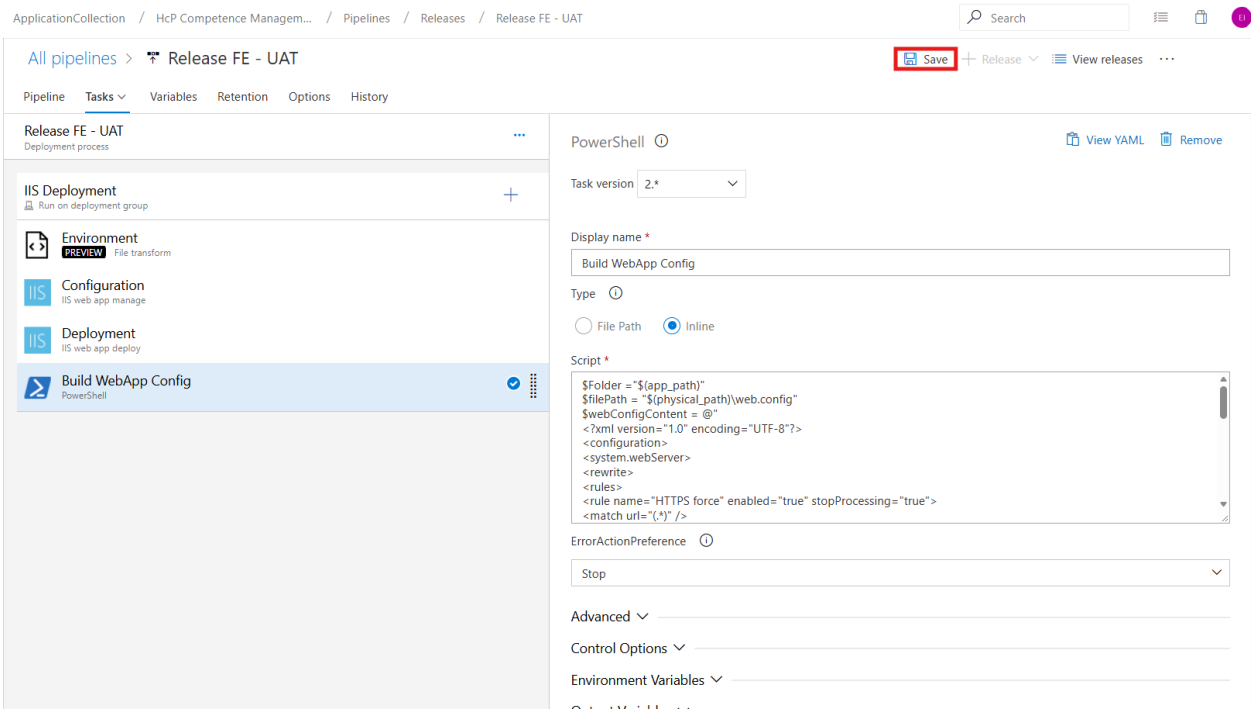
Trong đó, script được define như sau:

```
$Folder ="$(app_path)"
$filePath = "$(physical_path)\web.config"
$webConfigContent = @"
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
<system.webServer>
<rewrite>
<rules>
<rule name="HTTPS force" enabled="true" stopProcessing="true">
<match url="(.*)" />
<conditions>
<add input="{HTTPS}" pattern="^OFF$" />
</conditions>
```

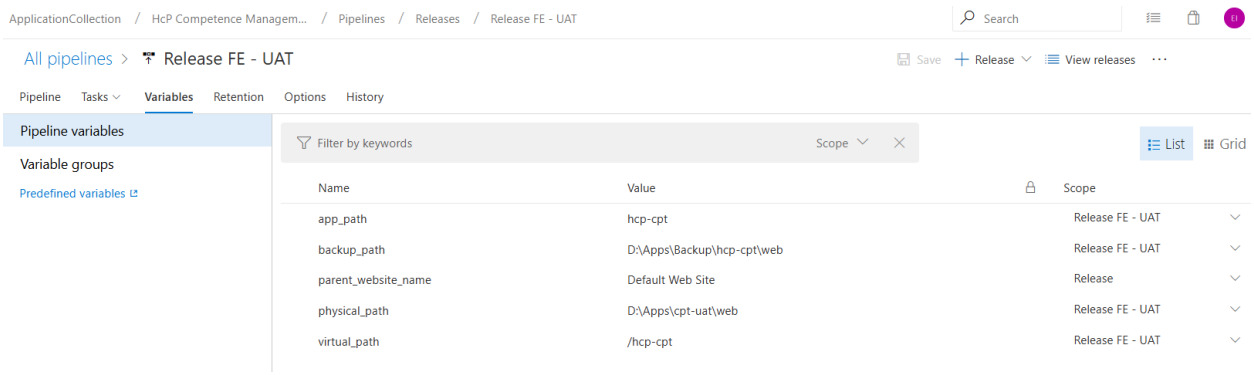
```
<action type="Redirect" url="https://{HTTP_HOST}{REQUEST_URI}"
redirectType="Permanent" />
</rule>
<rule name="Angular Routes" stopProcessing="true">
<match url=".*" />
<conditions logicalGrouping="MatchAll">
<add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
<add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
<add input="{REQUEST_URI}" pattern="^/api/.*$" negate="true" />
</conditions>
<action type="Rewrite" url="/$Folder/index.html" />
</rule>
</rules>
</rewrite>
</system.webServer>
</configuration>
"@

$FileExists = Test-Path -Path $filePath
if (!$FileExists) {
$webConfigContent | Out-File -FilePath $filePath -Encoding UTF8
Write-Host "web.config file generated at: $filePath"
}
```

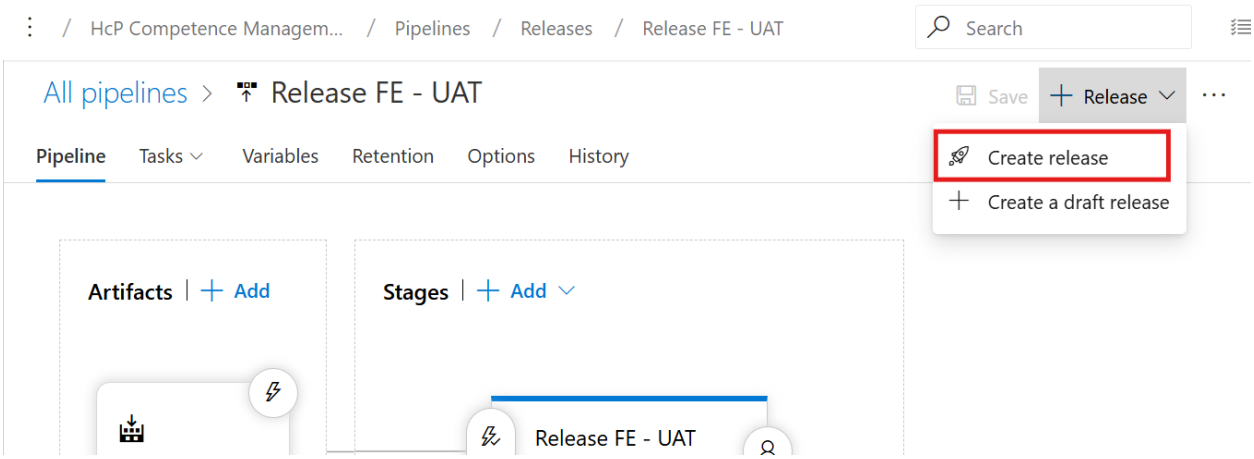
Step 7. Chọn Save để lưu pipeline



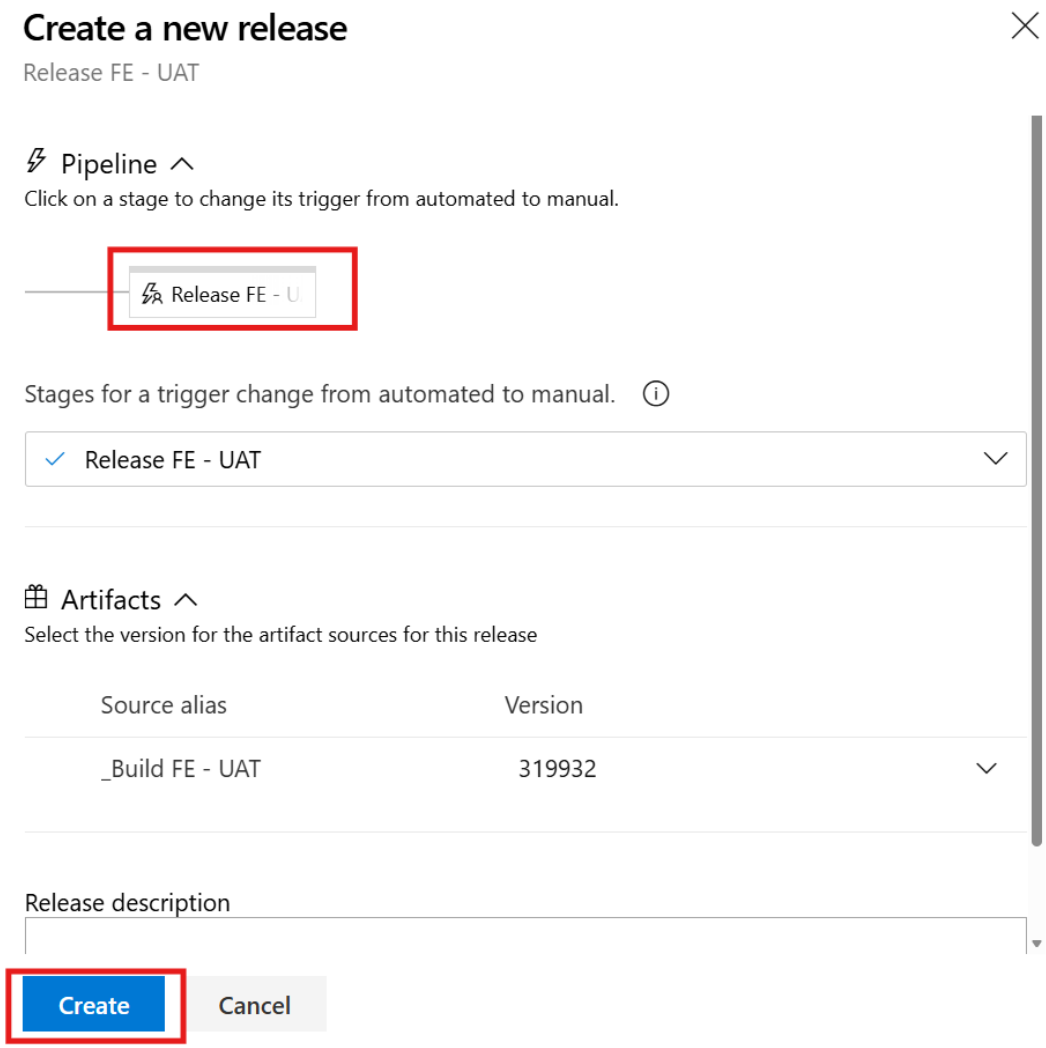
Step 8. Chọn Variables tab và config các variables cần thiết cho quá trình Release.



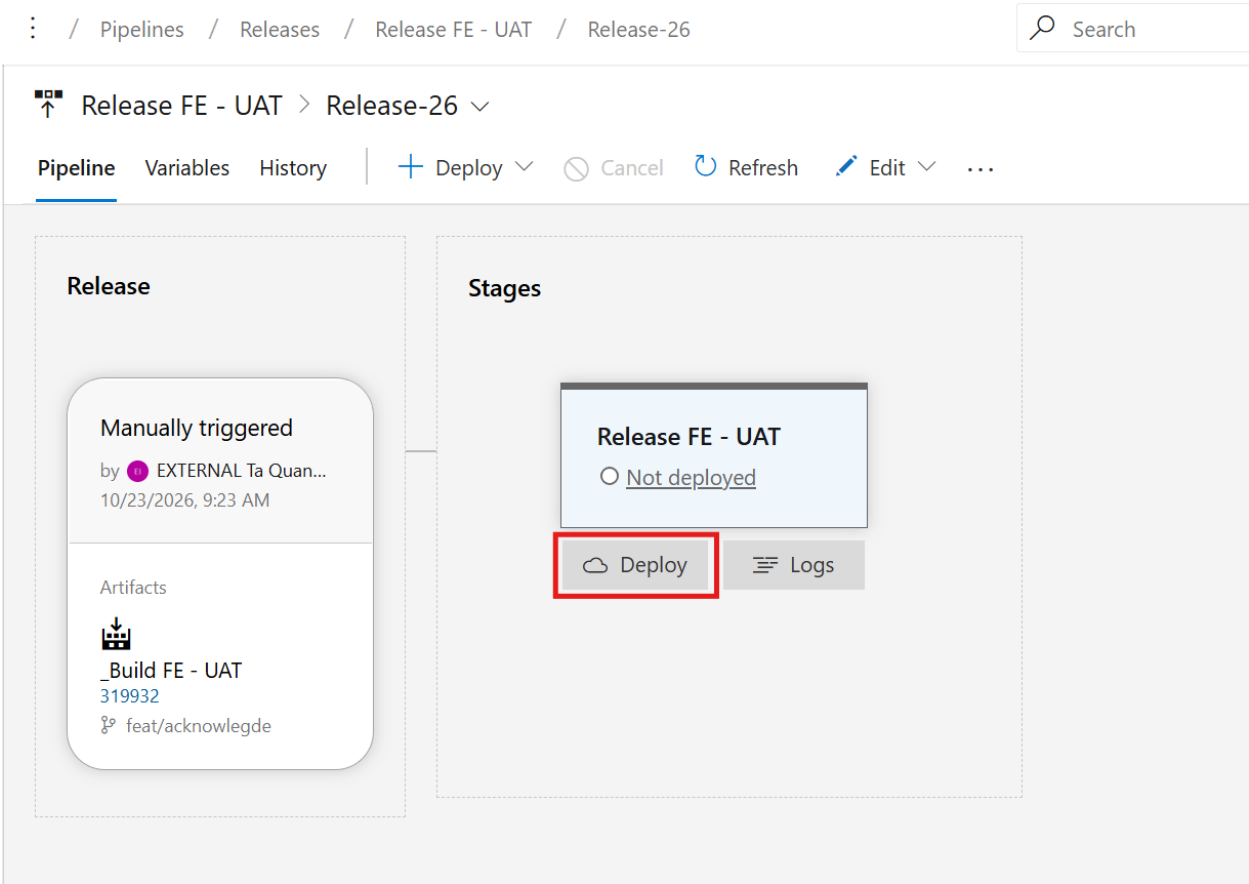
Step 8. Chọn “Create release”.



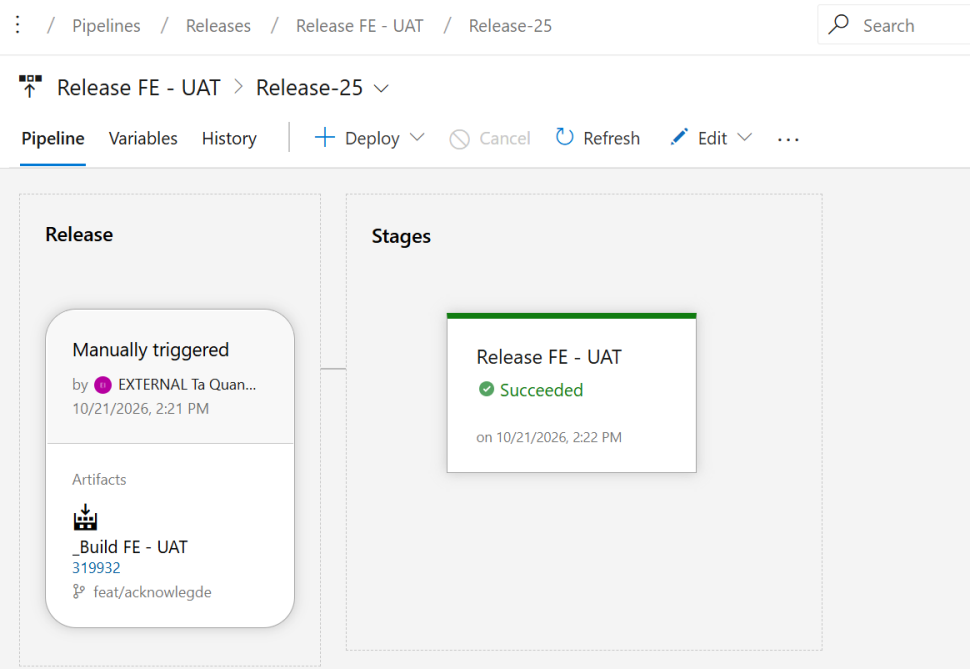
Chọn pipeline và chọn “Create”.



Step 9. Chọn deploy để deploy front end lên server.



Step 18. Sau khi deploy thành công Pipeline hiển thị giao diện như sau:



Có thể access vô front end thông qua $ virtual_path đã config ở các bước trên để verify các package đã được deploy.

